

Linguagens Formais - Notas de Aula

Ricardo Vargas Dorneles

26 de dezembro de 2021

- Introdução à Compilação
- Termos Básicos
- Expressões Regulares
- Autômatos Finitos
 - Algoritmo para obter E.R. a partir de A.F.
 - JFLAP
 - Autômatos Finitos Não-Determinísticos
 - Conversão de um AFN F em um AFD F'
 - Algoritmo para minimizar estados de um DFA
- Gramáticas
 - Classificação de Gramáticas
 - Algoritmo para gerar gramática não ambígua
 - Recursão à Esquerda
 - Conversão de G.R. para E.R
 - Fatoração
- Lista de Tópicos da Área 1

- **Análise Sintática Top-Down**
 - **Análise Recursiva com Retrocesso**
 - **Análise Recursiva Preditiva**
 - **Análise Tabular Preditiva**
- **Análise Redutiva**
 - **Construção de Analisadores SLR**
 - **Construção das tabelas de parsing LALR**
 - **Construção das tabelas de parsing LR(1)**
 - **Analisadores de Precedência de Operadores**
- **Exercícios do POSCOMP**
- **Questões do ENADE**
- **Os geradores de analisadores Flex e Bison**
- **Provas Anteriores - Área 1**
- **Provas Anteriores - Área 2**
- **Lista de Tópicos - Área 2**
- **Comentários sobre a Gramática de Referência**

- Apresentação da terminologia e de conceitos básicos relativos a linguagens formais.
- Estudo e análise dos formalismos clássicos para especificação de linguagens.
- Utilização de formalismos para especificação de linguagens.
- Estudo e análise dos algoritmos para reconhecimento de linguagens.
- Aplicação de algoritmos a problemas de reconhecimento.

Objetivo

- Instrumentalizar o aluno para que ele possa implementar um analisador léxico-sintático para uma linguagem de programação simples.

Conteúdo Programado

1. Introdução: Alfabeto, Sentença, Linguagem, Geração e Reconhecimento de Linguagens, Gramáticas, Derivação
2. Linguagens Regulares
 - 2.1 Gramáticas regulares
 - 2.2 Expressões regulares
 - 2.3 Autômato finito
 - 2.3.1 Determinístico
 - 2.3.2 Não determinístico
 - 2.3.3 Com movimentos vazios
 - 2.4 Conversão entre autômatos, gramáticas e expressões regulares
3. Introdução à Compilação
4. Análise Léxica

Conteúdo Programado

5. Linguagens Livres do Contexto

5.1 Gramática livre de contexto

5.2 Árvore de derivação

5.3 Ambigüidade

5.4 Operações com gramáticas (fatoração, eliminação de recursão etc.)

5.5 Autômato de pilha

5.6 Conversão entre autômatos de pilha e gramáticas

6. Análise Sintática

6.1 Análise Sintática Descendente Recursiva

6.2 Análise Sintática Descendente Preditiva

6.3 Análise Sintática Top Down

6.4 Análise Sintática Bottom Up

- As aulas serão teórico-expositivas com a resolução de diversos exercícios práticos durante as aulas. O professor orientará o aluno no desenvolvimento, ao longo do semestre, do projeto de um analisador léxico e sintático de alguma linguagem ou subconjunto de linguagem.

Avaliação

- O desempenho do aluno será avaliado no decorrer do semestre através de duas provas (P1 e P2), ambas com peso 1, e um trabalho prático (T1), também com peso 1 (média harmônica).
- Será considerado aprovado o aluno que atingir, no mínimo, média 6 (seis) nas atividades de avaliação e frequência igual ou superior a 75%.
 - Se esta média não for alcançada, o aluno poderá recuperar uma das duas provas, P1 ou P2, na data prevista no cronograma.
 - A nota obtida na recuperação substituirá a nota anterior correspondente. Notas de trabalhos não serão recuperados.
- O conceito final será expresso segundo as normas regimentais da Instituição.

Bibliografia

■ Básica

- AHO, A.V. Compiladores - Princípios, Técnicas e Ferramentas. Rio de Janeiro: Livros Técnicos e Científicos, 1995.
- MENEZES, P.B. Linguagens Formais e Autômatos. Porto Alegre: Editora Sagra-Luzzatto. 4ª ed. 2001.

■ Bibliografia Complementar

- HOPCROFT, John E.; ULLMAN, Jeffrey D. Introduction to automata theory, languages, and computation. Massachusetts: Addison-Wesley, 1979. 418 p.
- AHO, A. V. et alii. Compilers principles, techniques and tools. New Jersey: Prentice-Hall, 1988.
- AHO, A. V. and ULLMAN, J. The theory of parsing, translation and compiling. New Jersey: Prentice-Hall, 1972.
- APPEL, A.W. Modern Compiler Implementation in C. Cambridge: Cambridge University Press. 1998.
- WIRTH, N. Compiler Construction. Englewood Cliffs: Addison-Wesley, 1996.

Introdução à Compilação

Tipos de tradutores mais usados:

- Montador : linguagem assembly para linguagem de máquina
- Compilador : linguagem de alto nível para linguagem assembly ou de máquina
- Interpretador : Traduz e executa simultaneamente

[Voltar para o índice](#)

Fonte → Compilador → Programa Objeto
→ Mensagens de Erro
→ Tabelas Auxiliares

Tabela: Compilador

[Voltar para o índice](#)

Modelo Análise x Síntese

A compilação pode ser dividida em duas partes, com tarefas distintas em cada uma : análise e síntese

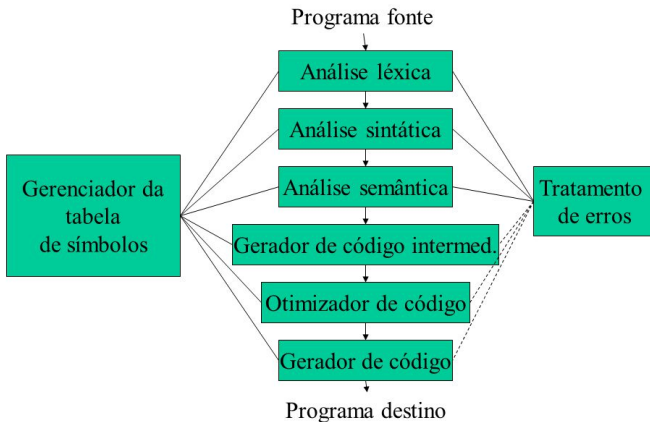
■ Análise:

- Reconhece cada uma das partes componentes do programa fonte e sua estrutura
- Cria uma representação interna
- A sintaxe a ser analisada é descrita através de gramáticas ou autômatos

■ Síntese

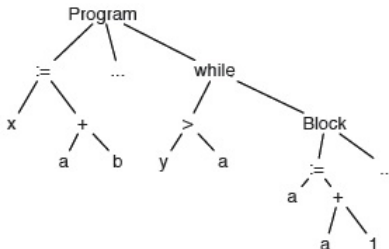
- Gera um programa objeto a partir da representação interna
- Depende da semântica do programa

Fases de um compilador (II)



■ Exemplo de representação interna : Árvore Sintática

```
x := a + b;  
y := a * b;  
while (y > a) {  
    a := a + 1;  
    x := a + b  
}
```



- Sintaxe - descreve a forma de um comando (forma de escrita)
- Semântica - descreve o significado de um comando (o que o comando faz)

[Voltar para o índice](#)

- Análise do Programa Fonte é feita em três fases:
 - Análise **léxica**
 - Análise **sintática**
 - Análise **semântica**

[Voltar para o índice](#)

Análise léxica:

- Lê seqüências de caracteres que formam o programa fonte
- Agrupa os caracteres em tokens que são as unidades léxicas, os átomos que compõem o programa fonte (ex: números, identificadores, operadores)
- Elimina os caracteres não significativos, como o fim de linha e espaços em branco

[Voltar para o índice](#)

Ex:

elemento := inicial + fator * 60



Análise léxica



(id,1)(op,:=)(id,2)(op,+)(id,3)(op,*)(cte,60)

[Voltar para o índice](#)

- O analisador léxico normalmente trabalha "por demanda".
- As informações obtidas na análise léxica são armazenadas na tabela de símbolos.

Ex: Identificador
Tipo
Valor
Linha onde foi definido

[Voltar para o índice](#)

Análise Sintática

- Agrupa os tokens em sentenças da linguagem
- Verifica se os tokens formam sentenças corretas da linguagem de acordo com as regras de sintaxe da linguagem
 - $\text{Atrib} \rightarrow \text{id} = \text{exp}$
 - $\text{Exp} \rightarrow \text{exp} + \text{exp}$
 - $\text{Exp} \rightarrow \text{cte} \mid \text{id}$
- Ex: $x3 = y + 3$

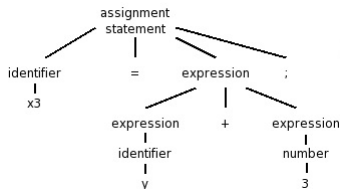


Figura: Árvore de derivação

- Efetua verificações que não podem ser feitas pela análise sintática
 - Verifica se as combinações entre operandos e operadores são válidas (posso aplicar o operador % a float?)
 - Verificação de tipo (posso atribuir um float a um char?)
 - Declarações (duplicadas, variáveis ainda não declaradas...)
 - Alguma coisa sobre estruturas de controle (esse break está dentro de um switch, for ou while?, entradas duplicadas em um switch...)
 - Complementa a tabela de símbolos

Fases de um compilador

Programa Fonte → An. Léxica → An. Sintática → An. Semântica →
ANÁLISE

Ger. de cód. intermediário → Otimização de Código → Ger. de código Objeto
SÍNTESE

[Voltar para o índice](#)

Ferramentas para a geração de compiladores

- Em C:
 - Geradores de analisadores léxicos : LEX
 - Geradores de analisadores sintáticos : YACC
- Em Java:
 - Javacc : gerador de analisadores léxicos e sintáticos

[Voltar para o índice](#)

Termos Básicos

- **Vocabulário** ou **alfabeto** : É o conjunto de símbolos que podem ser utilizados na construção de sentenças. Ex: $\{0,1\}$, $\{a,b,c\dots z\}$
- **Sentença** : Qualquer seqüência finita composta de zero ou mais símbolos. '0', '01', '001'
- **Linguagem** : É um conjunto qualquer de sentenças.
 - $\{1,100,1000\}$ por enumeração
 - $\{01,0011,000111,\dots\}$ 0^n1^n - define-se regras

[Voltar para o índice](#)

Operações com Sentenças

- Concatenação ou produto: Se $x='01'$ e $y='233'$ então $xy='01233'$
- Potenciação ou exponenciação: X^n é a concatenação de n ocorrências de x . Ex: se $x='01'$ então $x^2='0101'$
- Tamanho da Sentença: $|x|$, representa o número de elementos da sequência x . Ex: $|'01'| = 2$. Obs: a sentença vazia tem tamanho zero e é representada por ϵ .

[Voltar para o índice](#)

Operações com Linguagens

- União: $L \cup M$ é a linguagem formada pelas sentenças de L e M
- Diferença : $L - M$: sentenças de L que não estão em M
- Concatenação : LM : sentenças geradas pela concatenação de sentenças de L com sentenças de M . $LM = \{xy \mid x \in L \text{ e } y \in M\}$
- Potenciação da Linguagem: L^n é a linguagem resultante da concatenação de n sentenças de L
- Fecho de linguagens: L^* é a linguagem resultante da concatenação de qualquer número de sentenças de L . $L^* = L^0 \cup L^1 \cup L^2 \cup L^3 \cup \dots$
- Fecho positivo: L^+ é igual a L^* mas sem a sentença vazia

1.Considere as seguintes sentenças:

$$x=a$$

$$y=ab$$

$$z=bcd$$

Apresente o resultado das operações a seguir sobre as sentenças:

$$a) xy =$$

$$b) yx =$$

$$c) (xy)z =$$

$$d) y^2 =$$

$$e) z^3 =$$

$$f) z^0 =$$

$$g) |xy| =$$

$$h) |z^3| =$$

$$i) |z^2| =$$

$$j) |z^1| =$$

$$k) |z^0| =$$

2) Considere as seguintes linguagens:

$$A = \{a, b\}$$

$$B = \{aa, ab, bb\}$$

$$C = \{b, bb, bbbb\}$$

Apresente o resultado das operações que seguem sobre as linguagens (por enumeração ou por regra):

a) $A \cup B =$

b) $B \cup C =$

c) $A - B =$

d) $(A - B) - C =$

e) $A^3 =$

f) $B^2 =$

g) $A^* =$

h) $C^* =$

Notação Algébrica para Linguagens

Estas operações podem ser usadas para especificar linguagens.

$$\begin{aligned}\text{Ex: } \{ 10^n \mid n \geq 0 \} &= \{ 1, 10, 100, 1000 \} \\ \{ 0^n 1^n \mid n \geq 0 \} &= \{ \epsilon, 01, 0011 \}\end{aligned}$$

A linguagem sobre $V = \{0, 1\}$

$\{ x00y \mid x \text{ e } y \in V^* \}$ (qualquer sentença que pertença a V^* e que contenha ao menos uma seqüência 00)

[Voltar para o índice](#)

Componentes de Sentenças

- **Prefixo** de x : É obtido removendo-se 0 ou mais símbolos do final de x .

Ex.: $x = 001010$ prefixos: ϵ , 0, 00, 001, 0010, 00101, 001010

- **Sufixo** de x : obtido removendo-se 0 ou mais símbolos do início de x .

Ex.: $x = 001010$ sufixos: ϵ , 0, 10, 010, 1010, 01010, 001010

- **Subcadeia** : obtida removendo-se um sufixo e um prefixo.

Ex.: $x = 001010$ algumas subcadeias: ϵ , 001, 010, 101, 1010, etc.

- **Prefixo próprio, sufixo próprio e subcadeia própria**: são prefixos, sufixos e subcadeias diferentes de x .

3. Considere as seguintes sentenças:

$x = abba$

$y = abab$

$z = bcd$

a) apresente os prefixos de x

b) apresente os sufixos de y

c) apresente as subcadeias de z

d) apresente os prefixos próprios de yz

[Voltar para o índice](#)

Expressões regulares

- Uma expressão regular sobre um alfabeto V é uma expressão construída a partir de símbolos deste alfabeto e da sentença vazia combinados a partir da união, concatenação e fecho. O conjunto de sentenças formadas a partir da expressão regular forma uma linguagem.

Para o alfabeto $\{0,1\}$

1) Os símbolos do alfabeto constituem uma expressão

Expressão	Linguagem
0	$\{ 0 \}$
1	$\{ 1 \}$
ϵ	$\{ \epsilon \}$

2) Duas expressões r e s podem ser concatenadas na forma rs :

Expressão	Linguagem
01	$\{01\}$

3) União : $(r \mid s)$

Expressão	Linguagem
$0 \mid 1$	$\{0,1\}$
$0 \mid 00 \mid 000$	$\{0,00,000\}$
$(0 \mid 1)(0 \mid 1)$	$\{00,01,10,11\}$ - expressão com concatenação e duas uniões

4) Fecho : r^*

$$0^* = \{ \epsilon, 0, 00, 000, \dots \}$$

[Voltar para o índice](#)

Precedência :

1. Fecho
2. Concatenação
3. União

[Voltar para o índice](#)

1) Escrever 4 sentenças (se houver) p/ cada uma das expressões abaixo sobre o alfabeto $V = \{a, b\}$

a) $(ab)b =$

b) $\epsilon b b \epsilon a =$

c) $a|b|ab|ba =$

d) $(a|b)(b|a) =$

e) $(a|\epsilon)bbb =$

f) $aa(b|\epsilon)aa =$

g) $(a|b)a^* =$

h) $a^*ba^* =$

i) $(a^*)(b^*) =$

j) $(ab)^* =$

k) $(a|bb)^* =$

l) $(b^*ab^*)^* =$

Reescrita de expressões regulares

É a forma pela qual uma ER gera sentenças. Tem a forma:

$$r \rightarrow s$$

onde a expressão r pode ser substituída pela expressão s .

As regras para reescrita são as seguintes:

$$1) (r\epsilon) \rightarrow r$$

$$2) (\epsilon r) \rightarrow r$$

$$3) (r|s) \rightarrow r$$

$$4) (r|s) \rightarrow s$$

$$5) (r^*) \rightarrow \epsilon$$

$$6) (r^*) \rightarrow (r^*)r$$

[Voltar para o índice](#)

A partir da expressão $0(0|1)^*$ gerar a sentença 0101.

$$0(0|1)^* \rightarrow (6)$$

$$0(0|1)^*(0|1) \rightarrow (4)$$

$$0(0|1)^*1 \rightarrow (6)$$

$$0(0|1)^*(0|1)1 \rightarrow (3)$$

$$0(0|1)^*01 \rightarrow (6)$$

$$0(0|1)^*(0|1)01 \rightarrow (5)$$

$$0 \in (0|1)01 \rightarrow (4)$$

$$0 \in 101 \rightarrow (1)$$

$$0101$$

[Voltar para o índice](#)

2) Considere a expressão regular $(ab)^*(a|b)^*$. Apresente duas seqüências de aplicação de regras de reescrita que devem ser aplicadas a esta expressão para gerar a sentença **ababaaab**.

[Voltar para o índice](#)

Regras algébricas para expressões regulares

- As expressões regulares são expressões algébricas sobre as quais se podem fazer algumas operações (união, concatenação, fecho).
- Essas operações devem obedecer a um certo número de regras.
- Podemos agrupá-las em regras comutativas, regras associativas, regras distributivas, identidades e anuladores, regras de fecho.
- Duas expressões regulares são equivalentes se elas denotam a mesma linguagem.
- Quando se simplifica uma expressão regular, obtêm-se sucessivamente expressões regulares equivalentes.

Regras comutativas e associativas

- Sejam L , M e N expressões regulares.
 - A união de duas expressões regulares é comutativa
 $L \mid M = M \mid L$
 - A união de linguagens é associativa
 $(L \mid M) \mid N = L \mid (M \mid N)$
 - Se queremos fazer a união de três conjuntos, poderemos unir os primeiros dois e depois unir o resultante com o terceiro; ou podemos unir os dois últimos, unindo depois o primeiro com o conjunto resultante.

[Voltar para o índice](#)

Regras comutativas e associativas

- A concatenação de expressões regulares é associativa
- $(LM)N = L(MN)$
- A concatenação de três cadeias pode fazer-se indiferentemente de dois modos: concatenar as duas primeiras e depois a terceira à direita; ou concatenar as duas últimas e depois a primeira à esquerda.
- Mas não é comutativa
 - $LM \neq ML$.

[Voltar para o índice](#)

Regras distributivas

- A concatenação tem alguma analogia com o produto algébrico e a união com a adição.
- A concatenação é distributiva à esquerda em relação à união:
$$L (M \mid N) = LM \mid LN$$
- Por exemplo $aba(bba+abb)=ababba+abaabb$. Atendendo ao significado de $+$, união, temos que concatenar o prefixo aba com cada uma das cadeias dentro do parêntese. O resultado é a união dessas concatenações.
- E também distributiva à direita
$$(M \mid N)L = ML \mid NL$$

Regras distributivas

- A união não é distributiva em relação à concatenação nem à direita nem à esquerda
 - $(MN)|L \neq (M|L)(N|L)$
 - $L|(MN) \neq (L|M)(L|N)$

[Voltar para o índice](#)

Identities and annihilators (zeros)

- Numa certa álgebra, a identidade é o elemento que operado com qualquer outro elemento resulta nesse elemento, tal como 1 na multiplicação numérica.
 - O conjunto vazio, $\emptyset$, é a identidade para a união porque
$$\emptyset \cup L = L \cup \emptyset = L$$
 - Já para a concatenação, a identidade é a cadeia vazia porque
$$\epsilon L = L \epsilon = L$$
- O anulador é o elemento que operado com outro elemento dá sempre o conjunto vazio.
 - No caso da concatenação é o conjunto vazio
$$\emptyset L = L\emptyset = \emptyset$$
 - Para a união não existe anulador.

Regras do fecho

- O fecho é uma operação muito importante em expressões regulares quando se trata de linguagens infinitas.
- Ele possui as seguintes propriedades:
 - (i) $(L^*)^* = L^*$
 - (ii) $L^+ = LL^* = L^*L$
 - (iii) $L^* = L^+|\epsilon$
 - (iv) $\emptyset^* = \epsilon$
 - (v) $\emptyset^* = \emptyset$
 - (vi) $(L|M)^* = (L^*M^*)^*$

[Voltar para o índice](#)

Definições regulares

- Uma **definição regular** introduz um nome para uma expressão regular na forma

- $D \rightarrow r$

- onde D é um nome

Ex: $\text{Min} \rightarrow a|b|c|d|e|\dots|z$

$\text{Mai} \rightarrow A|B|C|D|E|\dots|Z$

$L \rightarrow \text{Min} \mid \text{Mai}$

[Voltar para o índice](#)

Especificação léxica através de expressões regulares

$Dig \rightarrow 0|1|\dots|9$

$Letra \rightarrow a|b|c|\dots|z|A|\dots|Z$

-Identificador em Pascal

$IdPas \rightarrow Letra(Letra|Dig|_)*$

$IdC \rightarrow (Letra|_)(Letra|Dig|_)*$

Um identificador ADA é semelhante ao de pascal mas não pode haver dois `_`'s seguidos e o identificador não pode terminar em `_`.

$IdADA \rightarrow Letra(Letra|Dig|_ (Letra|Dig))^*$

[Voltar para o índice](#)

- $\text{SimbEsp} \rightarrow := \mid = \mid < \mid <=$
- $\text{PalReserv} \rightarrow \text{AND} \mid \text{BEGIN} \mid \text{CONST} \mid \dots$
- Número inteiro sem sinal : $\text{Intsem} \rightarrow \text{DigDig}^*$
- Número inteiro com sinal : $\text{Intcom} \rightarrow (+ \mid - \mid \epsilon) \text{DigDig}^*$
- Número em ponto flutuante :
 - 3.2
 - 3E10
 - 3.2E10
 - +3.2

[Voltar para o índice](#)

- Um comentário em linguagem C é uma seqüência de caracteres delimitada por /* e */. Entre os delimitadores pode aparecer qualquer caracter, exceto a seqüência "/". Use o nome Y para representar "qualquer caracter exceto * e /".

Coment →

Obs : Tokens : são símbolos básicos (as "palavras") de um programa na linguagem.

[Voltar para o índice](#)

Exercícios

- Escreva uma expressão regular sobre o alfabeto $\{a, b, c\}$ que gere a linguagem cujas sentenças contenham no mínimo 1 e no máximo 3 símbolos a .
- Considere o alfabeto $\{a, b\}$. Construa expressões regulares sobre esse alfabeto que representem cada uma das linguagens abaixo:
 - a) Sentenças que contem no máximo uma ocorrência do substring bbb ;
 - b) Contem exatamente uma ocorrência do substring aaa e uma ocorrência do substring bbb (em qualquer ordem)
- Escreva uma expressão regular que gere todas as sentenças em $\{0, 1\}^*$ em que o número de 0's é divisível por 3.

- Desenvolva expressões regulares e gramáticas regulares que gerem as seguintes linguagens sobre $\Sigma = \{a, b\}$
 - a) $\{w \mid w \text{ tem no máximo um par de } a\text{'s como subpalavra e no máximo um par de } b\text{'s como subpalavra}\}$
 - b) $\{w \mid \text{qualquer par de } a\text{'s antecede qualquer par de } b\text{'s}\}$
 - c) $\{w \mid w \text{ não possui aba como subpalavra}\}$
- Especifique expressão regular que gere todas as sentenças de $\{0, 1\}^*$ formadas por 0's e 1's alternados.
- Especifique uma expressão regular equivalente que gere todas as sentenças em $\{0, 1\}$ com no mínimo três símbolos e que o terceiro da direita para a esquerda (antepenúltimo) seja 0.

- Descreva em palavras as linguagens geradas pelas seguintes expressões regulares:
 - a) $(aa + b)^*(a + bb)$
 - b) $(b + ab)^*(\epsilon + a)$
 - c) $(aa + bb + (aa + bb)(ab + ba)(aa + bb))^*$

[Voltar para o índice](#)

- Construa expressões regulares que gerem as seguintes linguagens sobre o alfabeto $\{0,1\}$
 - a) O conjunto de todos strings que terminam em 00.
 - b) O conjunto de todos os strings com 3 0's consecutivos.
 - c) O conjunto de todos strings tal que a cada 3 caracteres consecutivos pelo menos dois são 0.
 - d) O conjunto de todos os strings tal que o penúltimo símbolo é 0.

[Voltar para o índice](#)

(2014 - Técnicas 68) Considere a expressão regular a seguir.
 $(c^*a[abc]^*b[abc]^*)|c^*$ Assinale a alternativa que descreve, corretamente, todas as cadeias geradas por essa expressão regular.

- a) Cadeias sobre o alfabeto $\{a, b, c\}$ onde o primeiro **a** precede o primeiro **b**.
- b) Cadeias sobre o alfabeto $\{a, b, c\}$ com um número par de **a**'s.
- c) Cadeias sobre o alfabeto $\{a, b, c\}$ contendo a substring **baa**.
- d) Cadeias sobre o alfabeto $\{a, b, c\}$ contendo um número ímpar de **c**'s.
- e) Cadeias sobre o alfabeto $\{a, b, c\}$ terminadas por **c**.

(2012 - Fundamentos 42) Assinale a alternativa que apresenta, corretamente, uma expressão regular que denota todas as strings de a's e b's que têm pelo menos dois b's consecutivos.

a) $(a^* + bb)(a + ba)^*(a + b)^*$

b) $(a + ba)^*bb(ba + a)^*$

c) $(a + b)^*ba^*b(a + b)^*$

d) $(a + bb)^*(bb + a)^*$

e) $(a + ba)^*bb(a + b)^*$

[Voltar para o índice](#)

11) (2004 - Fund - 36) As seguintes expressões regulares denotam as linguagens P, Q, L e R, respectivamente: $(1 + 10)^*$, $(0 + 01)^*$, $(0 + 1)^*$, $0(11)^* + 1(00)^*$. Não se pode afirmar que: (obs: o operador " $\setminus$ " representa a operação de diferença de conjuntos)

- (a) $P \cap Q \neq \emptyset$
- (b) $P \cup Q \neq L$
- (c) $P \cap Q = \{\epsilon\}$
- (d) $(1 + 0)^* \setminus P = Q$
- (e) $R \subset L \setminus (P \cup Q)$

[Voltar para o índice](#)

10) (2004 - Fund - 21) Seja $\Sigma = \{a, b\}$. Uma expressão regular denotando a linguagem $L = \{w \in \Sigma^* \text{ tal que toda ocorrência de "a" em } w \text{ é imediatamente seguida de "b"}\}$ é:

(a) $(a^*b)^*$

(b) $(b + ab)^*$

(c) a^*b

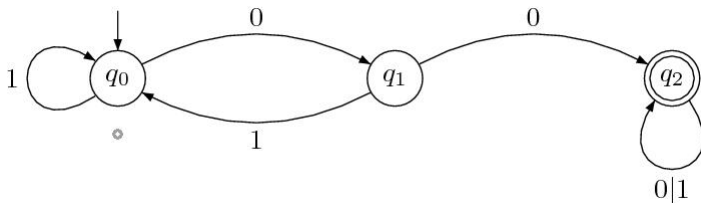
(d) $b + (ab)^*$

(e) $(ab)^*$

[Voltar para o índice](#)

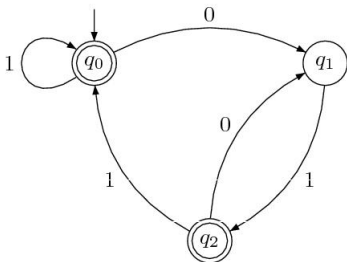
Autômatos Finitos

- Descrição de linguagens :
 - por geração
 - por reconhecimento
- Um **Autômato Finito** (A.F.) é um reconhecedor de sentenças regulares. Dada uma sentença ele responde sim ou não se a sentença pertence à linguagem. Um autômato finito é formado por:
 - Uma fita com uma sentença a ser reconhecida
 - Um cabeçote de leitura que aponta para o símbolo corrente
 - Uma memória que armazena o estado atual
- O símbolo corrente e o estado atual determinam qual o próximo estado (transição). Se ao final o autômato está em um estado aceitável, a sentença foi reconhecida.



- O autômato acima reconhecerá as sentenças que tiverem dois zeros seguidos em alguma posição, ou seja, reconhece as sentenças geradas pela expressão regular $(0|1)^*00(0|1)^*$.
- O autômato mostrado não reconhece a sentença 0110 porque ao terminar a sentença o autômato está no estado q_1 que não é um estado final.

1. Considere o seguinte automato finito:



■ Apresente a sequência de transições e indique se o autômato reconhece ou não cada uma das sentenças:

- a) 111010111
- b) 10101
- c) 01010
- d) 1100

Definição Formal de um autômato finito

- É definido por:
 - (V, S, q_0, F, d) onde
 - $V =$ Vocabulário
 - $S =$ Conjunto de estados
 - $q_0 =$ Estado inicial
 - $F =$ Conjunto de estados finais
 - $d =$ Função de transição no formato : $d(q,a) = p$

[Voltar para o índice](#)

Ex: O autômato anterior é definido pela quintupla (V, S, q_0, F, d)

$$V = \{0, 1\}$$

$$S = \{q_0, q_1, q_2\}$$

$$F = \{q_0, q_2\}$$

$$d(q_0, 0) = q_1$$

$$d(q_0, 1) = q_0$$

$$d(q_1, 1) = q_2$$

$$d(q_2, 0) = q_1$$

$$d(q_2, 1) = q_0$$

[Voltar para o índice](#)

- Matriz de transição : É uma forma reduzida de representar a função de transição

	Símbolos	
	0	1
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_2

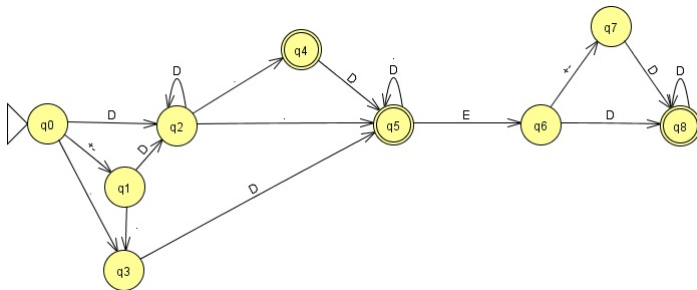
[Voltar para o índice](#)

- Faça um AF sobre o vocabulário $V=\{0,1\}$ que reconheça sentenças com um número par de símbolos e que contenha pelo menos dois símbolos.
- (V, S, q_0, F, d)
- $V=\{0,1\}$
- $S=$
- $F=$

[Voltar para o índice](#)

- Faça um AF sobre o vocabulário $V=\{0,1\}$ que reconheça sentenças com um número par de zeros e um número par de uns.
- Faça AF's para reconhecer:
 - a) Números inteiros sem sinal
 - b) Números inteiros com sinal
 - c) Identificadores em pascal (iniciam por letra e só pode conter letras, dígitos e underlines)
 - d) Números reais

[Voltar para o índice](#)



[Voltar para o índice](#)

- Faça um AF que reconheça sentenças sobre o alfabeto $\{0,1\}$ que não possuam prefixos onde a diferença entre os números de 0's e 1's é superior a 1.

[Voltar para o índice](#)

- Construa autômatos finitos que aceitem as seguintes linguagens sobre o alfabeto $\{0,1\}$
 - a) O conjunto de todos strings que terminam em 00.
 - b) O conjunto de todos os strings com 3 0's consecutivos.
 - c) O conjunto de todos strings tal que a cada 3 caracteres consecutivos pelo menos dois são 0.
 - d) O conjunto de todos os strings tal que o penúltimo símbolo é 0.

[Voltar para o índice](#)

- Desenvolva autômatos finitos determinísticos que reconheçam as seguintes linguagens sobre $\Sigma = \{a, b\}$
 - a) $\{w \mid w \text{ possui } aaa \text{ como subpalavra}\}$
 - b) $\{w \mid \text{o sufixo de } w \text{ é } aa\}$
 - c) $\{w \mid w \text{ possui número ímpar de } a \text{ e } b\}$
 - d) $\{w \mid w \text{ possui número par de } a\text{'s e ímpar de } b\text{'s ou } w \text{ possui número par de } b\text{'s e ímpar de } a\text{'s}\}$.
 - e) $\{w \mid \text{o quinto símbolo da direita para a esquerda de } w \text{ é } a\}$

[Voltar para o índice](#)

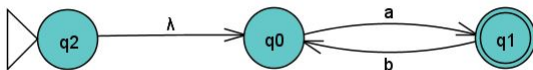
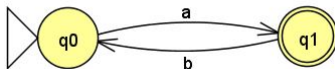
Conversão de AF para ER

- A conversão de um autômato finito para uma expressão regular é útil porque a ER é um formalismo mais sucinto e usado em geradores de analisadores léxicos e outros programas.
- Dois algoritmos usados para fazer essa conversão são:
 - Método de eliminação de estados
 - Método de Arden
- Começaremos pelo método de eliminação de estados

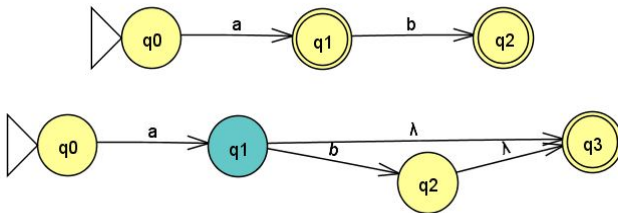
Método de eliminação de estados

- Se baseia em ir simplificando o autômato até chegar em um autômato com um estado inicial e um final, ligados por uma transição equivalente à expressão regular.
- As simplificações no autômato são mostradas a seguir:

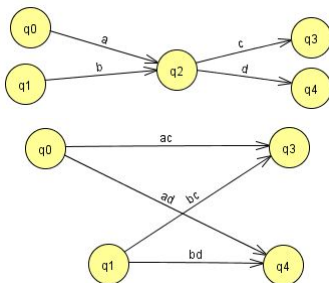
- O estado inicial não pode ter nenhuma transição chegando a ele. Se houver, é criado um novo estado inicial e acrescentada uma transição vazia do novo estado inicial ao estado inicial original. O mesmo deve ser feito se o estado inicial for um estado final.



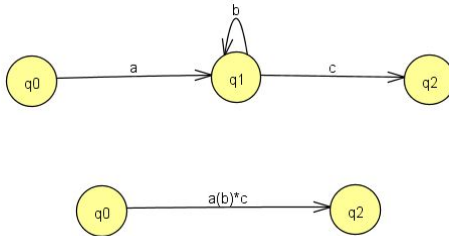
- O estado final deve ser único e não pode haver transições saindo dele. Se houver mais de um, é criado um novo estado final e puxadas transições ϵ (transições associadas a uma expressão regular ϵ) de cada estado final para o novo estado final. O mesmo ocorre se houver transições saindo do estado final.



- Elimina-se cada um dos estados do autômato, exceto o inicial e o final.
- Suponha que o estado X a ser eliminado tenha N transições chegando nele e M transições saindo. Combina-se cada transição chegando com cada transição saindo, gerando $N \times M$ transições.

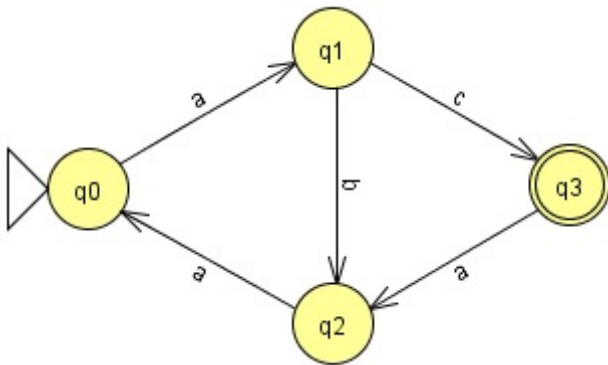


- Se o estado X tem uma transição para ele próprio, coloca-se o fecho dessa transição dentro de cada transição resultante das $N \times M$ combinações.

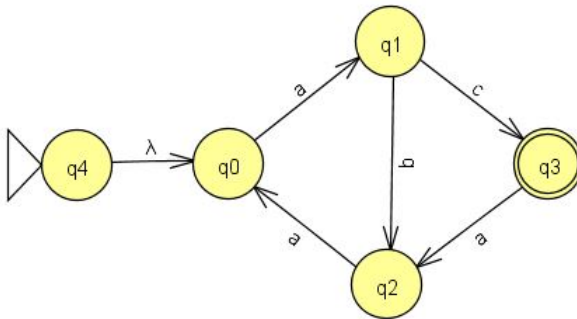


- Transições paralelas são substituídas pela transição com a união das transições paralela.

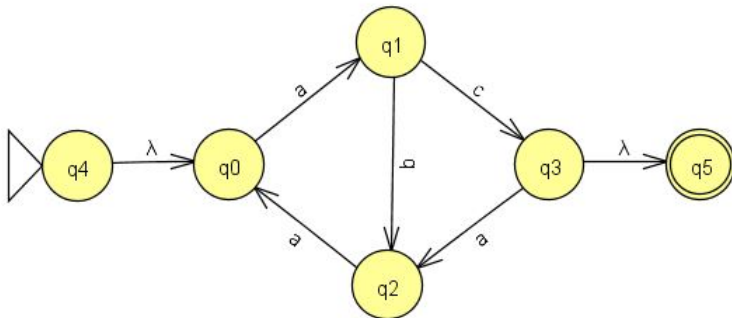
- Construa, para o autômato abaixo, a expressão regular equivalente:



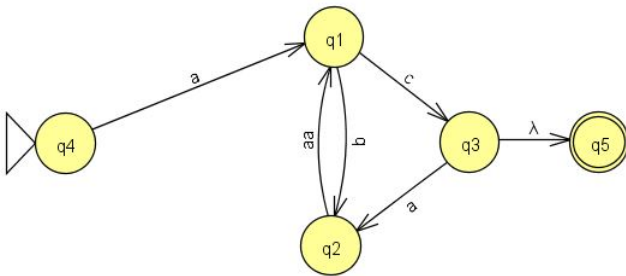
- Primeiro passo: Como o estado inicial tem uma transição chegando, gera-se um outro estado inicial:



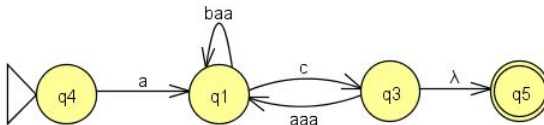
- Segundo passo: Como o estado final tem transições saindo, gera-se um outro estado final:



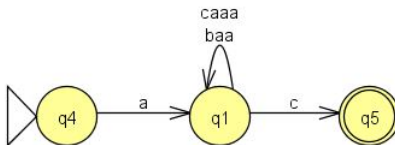
- Passo 3: Elimina-se q_0 (podia ser qualquer estado). A sequência de transições λ - a entre q_4 e q_1 torna-se a transição a entre q_4 e q_1 , a sequência de transições a - a entre q_2 e q_1 torna-se a transição aa entre q_2 e q_1 , e o estado q_0 é removido.



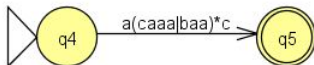
- Passo 4: Elimina-se q2 (podia ser qualquer estado. Escolhi o q2 porque tem menos transições). A sequência de transições b-aa a partir de q1 torna-se a transição **baa** de q1 para q1. A sequência de transições a-aa entre q3 e q1 torna-se a transição **aaa** entre q3 e q1, e o estado q2 é removido.



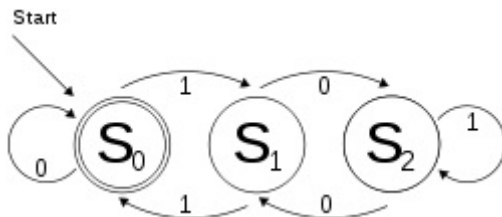
- Passo 5: Elimina-se q3 (podia ser o q1. Escolhi o q3 porque não tem transição para ele mesmo). A sequência de transições $c \rightarrow \lambda$ a partir de q1 torna-se a transição c de q1 para q5. A sequência de transições **c-aaa** de q1 para q1 torna-se a transição **caaaa** de q1 para q1, e o estado q3 é removido.



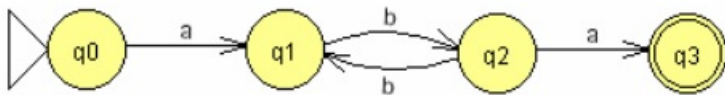
- Passo 6: Elimina-se q1. A sequência de transições a-c torna-se a transição **a(caaa|baa)*c** de q4 para q5 e essa transição é a expressão regular equivalente.



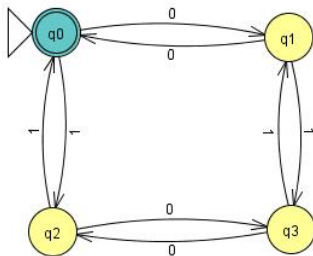
O autômato abaixo reconhece números binários múltiplos de 3.
Encontre a expressão regular equivalente:



Encontre a expressão regular equivalente ao autômato a seguir:

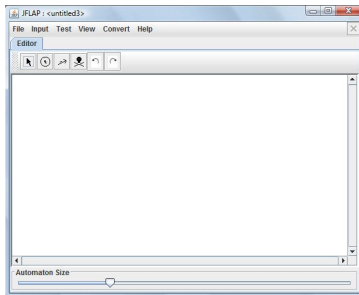


- Encontre uma expressão regular equivalente ao autômato a seguir:



- O JFLAP é um pacote de ferramentas gráficas que podem ser utilizadas como auxílio no aprendizado dos conceitos básicos de Linguagens Formais e Teoria de Autômatos.
- Suas principais funcionalidades são:
 - Criação de autômatos finitos, expressões regulares e gramáticas.
 - Conversão entre os formalismos citados acima.
 - Verificação se sentenças dadas pertencem à linguagem definida.
- Disponível em:
 - <http://www.jflap.com-about.com/download.html>
 - <http://www.jflap.org>
 - Acervo da turma no UCS Virtual

Criação de Autômatos



[Voltar para o índice](#)

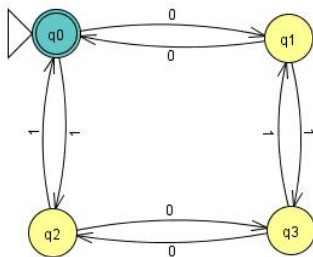
Menu do editor de autômatos



- 1 Definição de atributo de estado
- 2 Criação de estado
- 3 Inclusão de transição
- 4 Remoção de elemento
- 5 Desfazer
- 6 Refazer

[Voltar para o índice](#)

Crie o autômato a seguir no editor do JFLAP:



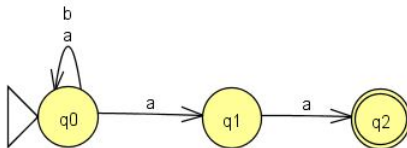
[Voltar para o índice](#)

Autômatos Finitos Não-Determinísticos

- Um **autômato finito não determinístico** (AFN ou NFA) é um autômato em que ocorre mais de uma transição para o mesmo estado e símbolo atual.
- Em um AFN a sentença é reconhecida se existe PELO MENOS UMA seqüência de transições que leva a um estado final.
- Autômatos não determinísticos são utilizados porque simplificam bastante a modelagem da linguagem.

Obs: um autômato em que há no máximo uma transição para cada estado e símbolo é chamado **autômato determinístico** (DFA ou AFD)

- Suponha a expressão regular $(a|b)^*aa$, que descreve "qualquer sentença sobre **a** e **b** que termina em **aa**".
- O autômato abaixo parece uma representação equivalente, bem intuitiva.
- O problema do autômato abaixo é que no estado 0 há duas transições para o símbolo **a**, de modo que o reconhecedor não tem como saber se deve permanecer no estado 0 ou ir para o estado 1.



Um AFN é definido pela quintupla $\{K, V, M, S, Z\}$ onde:

- K = conjunto de estados
- V = Vocabulário
- M = função de transição
- S = Estado inicial
- Z = conjunto de estados finais
- e a função de transição mapeia $K \times V \Rightarrow$ subconjunto de K .
 - ex: $n(q_0, a) = \{q_0, q_1, q_2\}$

[Voltar para o índice](#)

Conversão de um AFN F em um AFD F'

- $V' = V$
- $S' = S$
- $K' =$ o conjunto de estados de F' é o conjunto de estados em que cada estado corresponde a um subconjunto de K
- Ex:
 - $K = \{q_0, q_1, q_2\}$
 - $K' = \{q_0, q_1, q_2, q_0q_1, q_0q_2, q_1q_2, q_0q_1q_2\}$
- Os estados finais de F' são todos os estados que contêm ao menos um estado final de F .

- Ex: Se q_2 é um estado final de F então $\{q_2, q_0q_2, q_1q_2, q_0q_1q_2\}$ são estados finais de F'
- A função de mapeamento M' é definida como:
 - Para cada estado X de F' formado por um único estado X de F , o estado destino da transição $M'(X,T)$ é o estado correspondente ao subconjunto da transição $M(X,T)$ em F .
 - Ex: Se no autômato não-determinístico a transição do estado q_0 para o símbolo 0 são os estados q_0 e q_1 , no autômato determinístico a transição do estado q_0 para o símbolo 0 será o estado q_0q_1 .

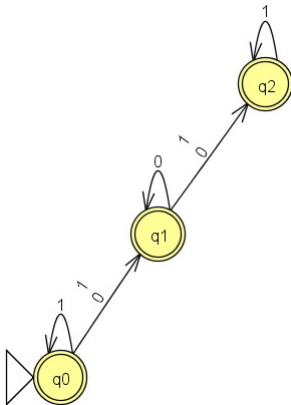
- Para cada estado composto de F' formado pelos estados $X_1, X_2 \dots X_i$ de F , o estado destino da transição $M'(X_1 X_2 \dots X_i, T)$ é o estado correspondente ao subconjunto formado pela união de $M(X_1, T) \cup M(X_2, T) \cup M(X_i, T)$ em F .
- Ex: Se no autômato não-determinístico a transição do estado q_0 para o símbolo 0 são os estados q_0 e q_1 , e a transição do estado q_1 para o símbolo 0 é o estado q_2 , no autômato determinístico a transição do estado $q_0 q_1$ é o estado correspondente ao conjunto para $M(q_0, T) \cup M(q_1, T)$, ou seja, $\{q_0, q_1\} \cup \{q_2\}$, ou seja $\{q_0, q_1, q_2\}$, que é o estado $q_0 q_1 q_2$.

- Faça um DFA equivalente ao seguinte NFA. Encontre uma expressão regular equivalente.

- $V = \{0,1\}$
- $K = \{q_0, q_1\}$
- $S = q_0$
- $F = \{q_0, q_1\}$
- $M(q_0, 0) = \{q_1\}$
- $M(q_0, 1) = \{q_0, q_1\}$
- $M(q_1, 0) = \{q_1\}$

[Voltar para o índice](#)

- Idem:
- $V = \{0, 1\}$
- $K = \{q_0, q_1, q_2\}$
- $S = q_0$
- $F = \{q_0, q_1, q_2\}$
- $M(q_0, 0) = \{q_1\}$
- $M(q_0, 1) = \{q_0, q_1\}$
- $M(q_1, 0) = \{q_1, q_2\}$
- $M(q_1, 1) = \{q_2\}$
- $M(q_2, 1) = \{q_2\}$



- Frequentemente o número de estados possíveis (subconjuntos dos estados do autômato original) é grande, e muitos deles são inalcançáveis a partir do estado inicial ou são mortos (a partir deles não se pode chegar a um estado final). Esses estados são inúteis para o autômato (não alteram seu poder de reconhecimento) e podem ser eliminados.
 - Uma alternativa para reduzir o trabalho de fazer o DFA é partir do estado inicial e ir gerando somente os estados alcançáveis a partir do estado inicial.

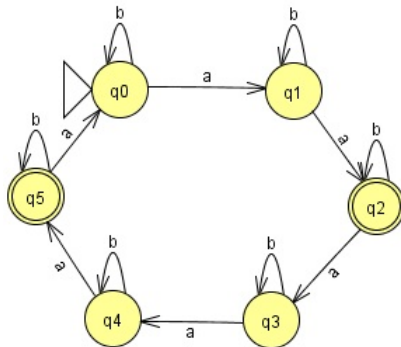
Ex: Encontre um autômato finito determinístico equivalente à expressão regular $a^*b|b^*a$.

Algoritmo para minimizar estados de um DFA

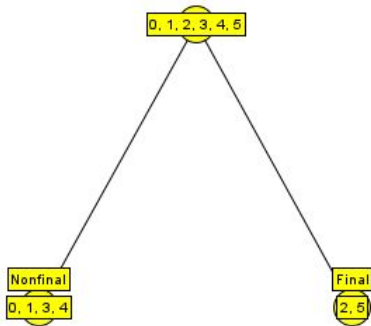
- 1 Construir uma partição inicial Π do conjunto de estados em 2 grupos : os estados finais e os não finais.
- 2 Aplique o procedimento a seguir a Π para construir uma nova partição Π_N
Para cada grupo G de Π faça:
Particione G em subgrupos de tal forma que dois estados s e t de G estão no mesmo subgrupo se e somente se para todos os símbolos de entrada a , os estados s e t têm transições em a para estados do mesmo grupo de Π .
Substitua G em Π_N pelo conjunto de subgrupos formados
- 3 Se $\Pi_N = \Pi$ faça $\Pi_{Final} = \Pi$ e vá para 4
Senão $\Pi \leftarrow \Pi_N$ e vá para 2
- 4 Escolha um estado em cada grupo da partição final como representativo do grupo. Estes serão os estados do DFA mínimo.
- 5 Se M' tem um estado morto, isto é, um estado não final e só tem transições para si mesmo, este estado é removido. Remova também estados que não são alcançáveis a partir do estado inicial.

[Voltar para o índice](#)

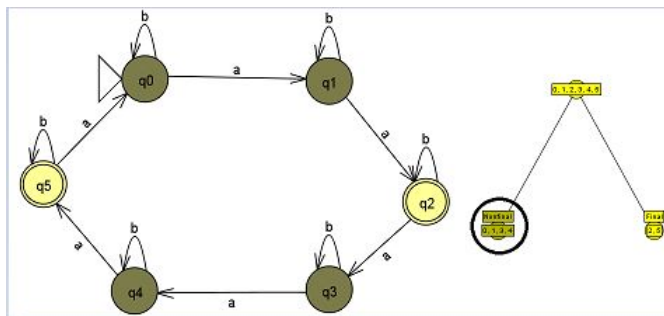
Verifique se o autômato abaixo é mínimo e, se não for, encontre o autômato mínimo equivalente.



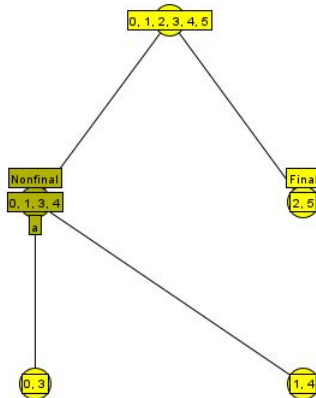
- A funcionalidade de minimização de autômato no JFLAP é um auxiliar na divisão dos grupos.
- O particionamento é feito na forma de uma árvore, onde a cada passo é selecionada uma folha para ser particionada, e qual o símbolo que será utilizado para a divisão do grupo;
- O particionamento inicial é feito em dois grupos: estados finais e estados não-finais.



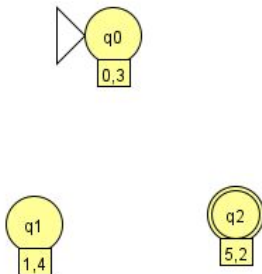
- A árvore acima mostra o particionamento inicial. Ao selecionar-se um grupo que não pode ser subdividido, o JFLAP informa que não é possível.
- Isso ocorre no exemplo acima no conjunto de estados finais.
- Após selecionar uma folha, os estados correspondentes àquela folha são mostrados em destaque:



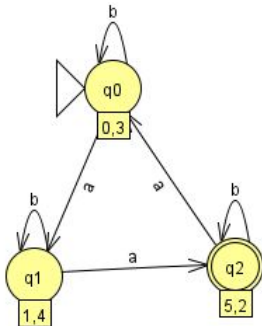
- Pode-se seleccionar um nodo e um símbolo (Set Terminal) para fazer a subdivisão, ou o JFLAP pode completar a subdivisão.



- Ao clicar em Finish, são mostrados os estados equivalentes a cada grupo.



- E ao clicar em Complete, são acrescentadas as arestas.



Prog. Fonte $\rightarrow$ Analisador Léxico $\rightarrow$ tokens $\rightarrow$ Analisador Sintático

Funções do Analisador Léxico

- Agrupar caracteres do fonte em tokens
- Eliminar comentários, espaços em branco, caracteres de tabulação, nova linha
- Agrupar dígitos

Interface do Analisador Léxico com o fonte:

- Lê seqüência de caracteres do fonte
- Bufferiza a entrada

Interface com o sintático : fornece tokens

- Nem sempre é possível separar a análise léxica da sintática.
 - Ex: Fortran (ignora os espaços em brancos que são eliminados no início da análise léxica)
 - Do 10 I=1.10 { variável Do10I recebe 1.10 }
 - Do 10 I=1,10 { laço DO para I variando de 1 a 10 }
- Ex2. Português estruturado:
 - $a < -b$
 - "a recebe b" ou "a menor que -b"?
 - depende do contexto (se $a < -b$ entao...)

■ **Tokens:** São os terminais da gramática da linguagem

- Identificadores
- Números
- Strings
- Palavras Reservadas
- Símbolos especiais (*, <, <=, etc)

[Voltar para o índice](#)

■ Representação

- É definido um código para cada token existente. O token deve conter:

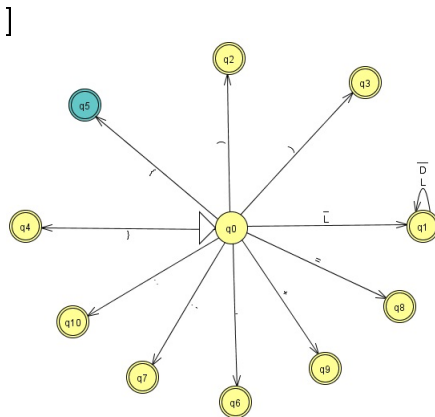
- Código
- Valor (valor numérico, entrada na tabela de símbolos, string do identificador)
- Linha e coluna onde aparece

■ Implementação do analisador léxico

- É feita através de um AFD onde:
 - O alfabeto é composto dos caracteres da linguagem
 - Termina quando reconhece um token

Lista de Tokens

```
#define TKId 1
#define TKVoid 2
#define TKInt 3
#define TKFloat 4
#define TKVirgula 5
#define TKDoisPontos 6
#define TKAbrePar 7
#define TKFechaPar 8
#define TKAttrib 9
#define TKPontoeVirg 10
#define TKAbreChave 11
#define TKFechaChave 12
#define TKSoma 13
```



[Voltar para o índice](#)

```

void main ()
{
int a , b , c , d ;
a=b+c ;
}

```

linha	token	lexema
1	TKVoid	"void"
1	TKID	"main"
1	TKAbrePar	"("
1	TKFechaPar	")"
2	TKAbreCol	"{"
3	TKInt	"int"
3	TKId	"a"
3	TKVirgula	","
...	...	...

[Voltar para o índice](#)

- A função a seguir reconhece sentenças sobre $\{0,1\}$ em que após cada 0 ocorre um símbolo 1.
- A entrada do léxico (a sentença a ser reconhecida) é o string st e a posição corrente do string é dada por pos.
 - Normalmente a entrada do léxico é o arquivo de entrada. Nesse exemplo foi utilizado o string para simplificação.
- A função retorna um dos seguintes valores:
 - 0 - Ocorreu erro no reconhecimento;
 - 1 - A sentença chegou ao final;
 - 2 - A sentença não chegou ao final mas o reconhecimento terminou em um estado final.

```

int rec_equ(char st[], char lex[])
{
    int estado=0, fim=0, pos=0;
    while (!fim)
    {
        lex[pos]=st[pos];
        switch(estado){
            case 0: if (st[pos]=='0')
                    {estado=1; break;}
                    if (st[pos]=='1')
                    {estado=0; break;}
                    lex[pos]='\0';
                    if (st[pos]=='\0') return 1;
                    return 2;
            case 1: if (st[pos]=='1')
                    {estado=0; break;}
                    lex[pos]='\0';
                    return 0;
        } //switch
        pos++;
    } // while
} // função

```



```

#include <stdio.h>
#include <stdlib.h>
int rec_equ(char st[], char lex[]);
int main()
{
    char exp1[20], lex[20];
    scanf("%s", exp1); // string a ser reconhecido
    int res=rec_equ(exp1, lex); // exp = string a ser reconhecido
                                // lex = lexema com o token reconhecido
    if (res==1) // reconheceu o \0 em estado final
        printf("Reconheceu tudo !!! lexema=%s\n", lex);
    else if (res==2) // é estado final, mas não há transição
        printf("Reconheceu até %s", lex);
    else printf("Erro a partir de %s", lex); // não é final e não há transição
    system("pause");
}

```

[Voltar para o índice](#)

Tratamento de Palavras Reservadas

Alternativas:

- Identificadas no autômato
- Reconhece como identificador e verifica se é uma palavra reservada. Pode-se ter uma tabela com palavras reservadas onde é guardado o código da palavra.

[Voltar para o índice](#)

- O autômato também pode ser implementado a partir de uma tabela que diz, para cada estado e símbolo, qual o próximo estado, e para cada estado, se é estado final ou não e qual o código de token que deve ser retornado.
- A tabela para o autômato

[Voltar para o índice](#)

Escreva um analisador léxico que leia um arquivo prog.c e reconheça os seguintes tokens:

- 1 Identificadores (token=1)
- 2 As palavras reservadas:
 - 1 void (token=2)
 - 2 int (token=3)
 - 3 float (token=4)
- 3 os tokens vírgula (5), dois pontos (6) e abre (7) e fecha (8) parênteses, atribuição (9), ponto-e-vírgula (10), abre (11) e fecha (12) chaves e operador de soma (13) .

Use o analisador léxico para reconhecer e listar a sequência de tokens do programa a seguir:

```

void main ()
{
int a , b , c , d ;
a=b+c ;
}

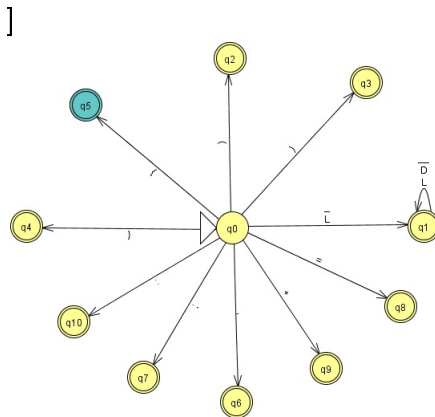
```

linha	token	lexema
1	TKVoid	"void"
1	TKID	"main"
1	TKAbrePar	"("
1	TKFechaPar	")"
2	TKAbreCol	"{"
3	TKInt	"int"
3	TKId	"a"
3	TKVirgula	","
...	...	...

[Voltar para o índice](#)

Lista de Tokens

```
#define TKId 1
#define TKVoid 2
#define TKInt 3
#define TKFloat 4
#define TKVirgula 5
#define TKDoisPontos 6
#define TKAbrePar 7
#define TKFechaPar 8
#define TKAttrib 9
#define TKPontoeVirg 10
#define TKAbreChave 11
#define TKFechaChave 12
#define TKSoma 13
```



[Voltar para o índice](#)

- O arquivo deve ser aberto pelo programa principal e seu descritor acessível pelo léxico
- O programa principal deve constar de um laço onde ele fica chamando o léxico para buscar cada token e exibe o código do token e o lexema identificado.
- A varredura do léxico deve ser encerrada ao ser encontrado o final do arquivo.

[Voltar para o índice](#)

Tabela de símbolos

1.teste

2.a

3.b

4.proc1

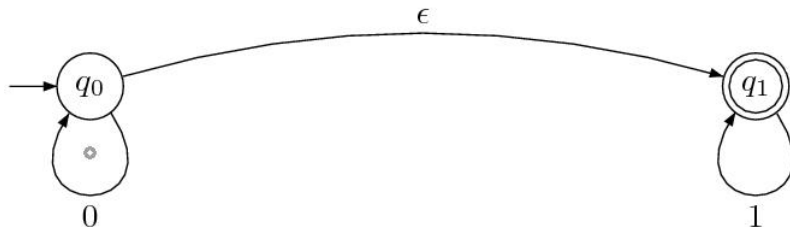
Seqüência de tokens:

<Tprog><id,1><TPV><Tvar><id,2><T2P><Tint><TPV>....

[Voltar para o índice](#)

Autômato Finito com Transições- ϵ

É um autômato finito em que ocorrem transições ϵ . São usadas para representar expressões regulares.



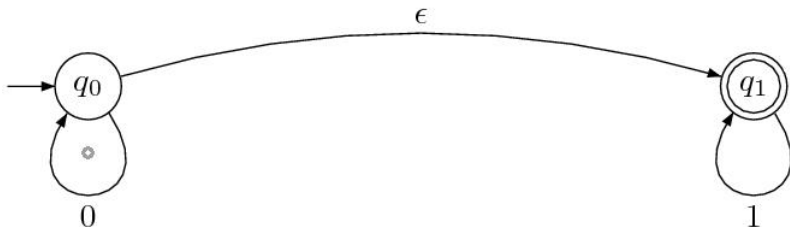
$$= 0^*1^*$$

[Voltar para o índice](#)

Autômato Finito com Transições- ϵ

- É uma 5-tupla (K, V_t, M, S, Z) onde
 - K = conjunto de estados
 - V_t = Vocabulário
 - S = Estado inicial
 - Z = Conjunto de estados finais
 - M = função de transição que mapeia $K \times V_t \cup \epsilon \Rightarrow$ subconjunto de K
- São equivalentes a NDA's sem transições ϵ . Para obter o NFA sem transições ϵ é necessário o conceito de **fecho- ϵ** .
- **Fecho- ϵ** de um estado q é o conjunto de estados alcançáveis através de transições ϵ (inclui o próprio estado q).

No exemplo:



$$\text{Fecho-}\epsilon(q_0) = \{q_0, q_1\}$$

$$\text{Fecho-}\epsilon(q_1) = \{q_1\}$$

[Voltar para o índice](#)

Algoritmo para conversão de um NFA com ϵ para um NFA sem ϵ

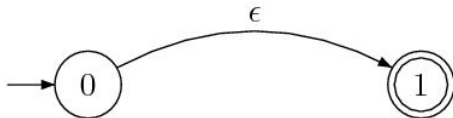
- Considere $F=(K,Vt,M,S,Z)$ um NFA com ϵ .
- F' é um NFA sem transições- ϵ equivalente ($L(F)=L(F')$)
- $Z' = \begin{cases} Z \cup \{S\} & \text{se o fecho-}\epsilon(S) \text{ contem um estado de } Z \\ \text{(estado inicial também é final)} \\ Z & \text{caso contrário} \end{cases}$
- $M'(q,a) =$ todos os estados que podem ser alcançados a partir de q através de uma transição a e qualquer número de transições ϵ .
 - $M'(q_0,0) = \{q_0, q_1\}$
 - $M'(q_0,1) = \{q_1\}$
 - $M'(q_1,0) = \{\}$
 - $M'(q_1,1) = \{q_1\}$

Algoritmo de Thompson para conversão de uma expressão regular em NFA

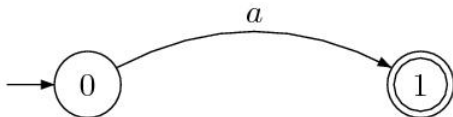
- Entrada : Uma expressão regular r sobre um alfabeto V
- Saída : Um autômato que reconhece $L(r)$.
- Inicialmente a expressão é decomposta nas subexpressões que a compõem e para cada subexpressão é gerado um NFA segundo as seguintes regras.

[Voltar para o índice](#)

1. Para ϵ , construir o NFA



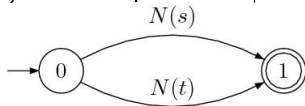
2. Para $a \in V$ construir o NFA



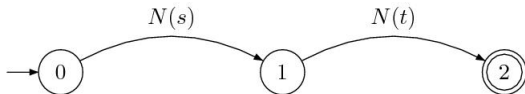
[Voltar para o índice](#)

3. Supondo $N(s)$ e $N(t)$ são NFA's para as expressões s e t

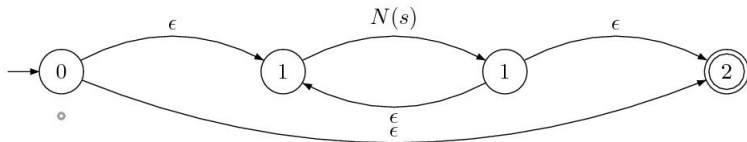
a) Para a expressão $s \mid t$ construir o autômato



b) Para a expressão st construir o autômato



c) Para a expressão s^* construir o autômato



■ Construa NFA's para as expressões regulares a seguir

- a) $(a|b)^*$
- b) $(a^*|b^*)^*$
- c) $((\epsilon|a)b^*)^*$
- d) $(a|b)^*abb(a|b)^*$
- e) $(ab)^*|(ba)^*$

[Voltar para o índice](#)

Algoritmo para construção de um DFA a partir de um NFA com transições- ϵ

Entrada : um NFA N

Saída : um DFA D equivalente Algoritmo :

Inicialmente $fecho = \epsilon(S)$ é o único estado de D e ele não está marcado.

Enquanto existe um estado T não marcado faça

 Marque T

 Para cada símbolo de entrada a faça

$U = fecho - \epsilon(move(T,a))$

 Se U não é estado de D

 Acrescente U como estado não marcado de D

 Fim Se

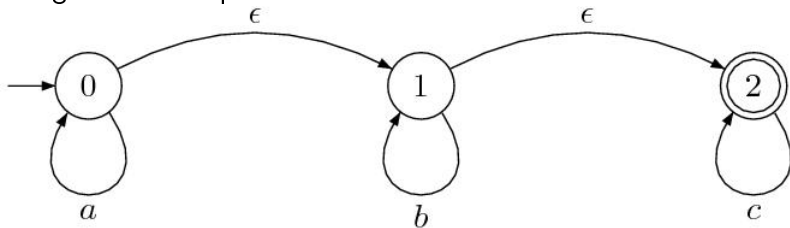
$Dtran(T,a) = U$

 Fim Para

Fim Enquanto

onde $move(T,a)$ é o conjunto de estados para os quais existe uma transição para o símbolo a a partir de um estado em T .

Ex: gere o DFA equivalente a



[Voltar para o índice](#)

$$Fecho - \epsilon(0) = 012$$

T_012:

$$U = fecho - \epsilon(\text{move}(T,a)) = 012$$

$$U = fecho - \epsilon(\text{move}(T,b)) = 12$$

$$U = fecho - \epsilon(\text{move}(T,c)) = 2$$

T_12:

$$U = fecho - \epsilon(\text{move}(T,a)) = \emptyset$$

$$U = fecho - \epsilon(\text{move}(T,b)) = 12$$

$$U = fecho - \epsilon(\text{move}(T,c)) = 2$$

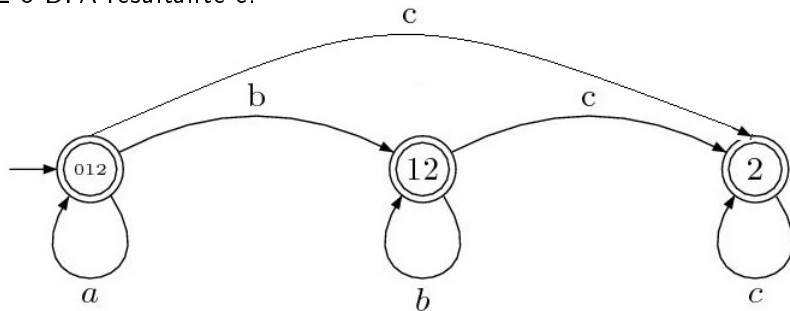
T_2:

$$U = fecho - \epsilon(\text{move}(T,a)) = \emptyset$$

$$U = fecho - \epsilon(\text{move}(T,b)) = \emptyset$$

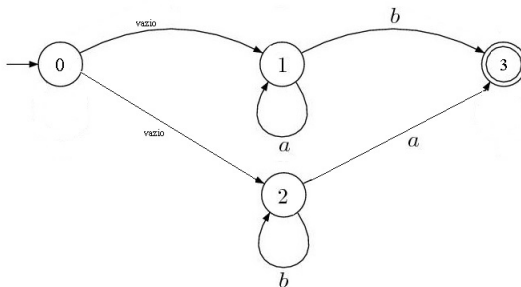
$$U = fecho - \epsilon(\text{move}(T,c)) = 2$$

E o DFA resultante é:



[Voltar para o índice](#)

- Encontre um autômato finito determinístico equivalente à expressão regular $a^*b|b^*a$.



Equivalência de Expressões Regulares

- Pode-se provar que duas expressões regulares são equivalentes mostrando-se que seus DFA's mínimos são o mesmo, exceto pelo nome dos estados.
- Usando esta técnica mostre que as expressões são equivalentes:

a) $(a|b)^*$

b) $(a^*|b^*)^*$

c) $((\epsilon|a)b^*)^*$

[Voltar para o índice](#)

- Uma **gramática** é um dispositivo usado para descrever uma linguagem.
- É definida por uma quádrupla $G = V_N, V_T, S, \phi$ onde V_N e V_T são os conjuntos de símbolos não terminais e terminais da linguagem.
- S é um símbolo inicial ($S \in V_N$) e ϕ é um subconjunto da relação $(V_T \cup V_N)^* V_N (V_T \cup V_N)^* \Rightarrow (V_T \cup V_N)^*$.
- Em geral um elemento (α, β) é escrito na forma $\alpha \rightarrow \beta$ e é chamado uma regra de produção.

Ex: gramática para identificadores em Pascal

$G = (V_N, V_T, S, \phi)$ onde

$V_N = \{I, L, D\}$

$V_T = \{a, b, c, d, \dots, z, 0, 1, \dots, 9\}$

$S = I$

$\phi = \{ \quad I \rightarrow L$

$\quad I \rightarrow IL$

$\quad I \rightarrow ID$

$\quad L \rightarrow a|b|c|\dots|z$

$\quad D \rightarrow 0|1|2|\dots|9\}$

[Voltar para o índice](#)

- Gramáticas são mecanismos mais poderosos do que expressões regulares e autômatos, pois podem descrever uma gama maior de linguagens.
- Estruturas aninhadas, comuns em linguagens de programação, não podem ser descritas por autômatos ou expressões regulares, mas podem ser descritas por gramáticas.
- Ex: $a^m b^m$ não pode ser reconhecido por um autômato pois envolve uma contagem, o que não pode ser feito por um autômato.
- Gramática : $S \rightarrow ab|aSb$

- Diz-se que um string X **produz** um string Y ($X \Rightarrow Y$) se existe uma série de derivações que a partir de X levam a Y .
- Uma **forma sentencial** é qualquer derivação do símbolo S . Uma linguagem L gerada por uma gramática G é o conjunto de todas as formas sentenciais cujos símbolos são terminais.
- Uma BNF (Backus-Naur Form) é uma terminologia usada para descrever gramáticas. Nesta terminologia os símbolos não terminais aparecem entre $\langle \rangle$ e os terminais não. O símbolo $\rightarrow$ de derivação é substituído por $::=$.

Ex:

$$\begin{aligned}\langle \text{identificador} \rangle ::= & \langle \text{letra} \rangle \mid \\ & \langle \text{identificador} \rangle \langle \text{letra} \rangle \mid \\ & \langle \text{identificador} \rangle \langle \text{digito} \rangle\end{aligned}$$
$$\langle \text{letra} \rangle ::= a \mid b \mid c \mid d \mid \dots \mid z$$
$$\langle \text{digito} \rangle ::= 0 \mid 1 \mid \dots \mid 9$$

[Voltar para o índice](#)

Classificação de Gramáticas

As gramáticas podem ser classificadas de acordo com o tipo de produção em 4 grupos (classificação de Chomsky):

- a) Gramáticas do tipo 0, irrestritas ou enumeráveis recursivamente. Não tem restrições nas regras (qualquer tipo de linguagem representável)
- b) Gramática sensível ao contexto (CSG - Context Sensitive Grammar) ou tipo 1. Contem somente produções do tipo $\alpha \rightarrow \beta$ onde $|\alpha| \leq |\beta|$ e $|\alpha|$ é o comprimento de α (ou seja, o número de símbolos terminais e não terminais do lado esquerdo da produção é menor ou igual ao número de símbolos do lado direito). O termo 'sensível ao contexto' vem de que são permitidas produções $\phi_1\alpha\phi_2 \rightarrow \phi_1\beta\phi_2$ (α é reescrito como β no contexto entre ϕ_1 e ϕ_2).

Ex: a linguagem $L(G_2) = \{a^n b^n c^n \mid n \geq 1\}$ é gerada pela seguinte gramática sensível ao contexto:

$G_2 = (\{S, B, C\}, \{a, b, c\}, S, \phi)$ onde ϕ consiste de

1. $S \rightarrow aSBC$

2. $S \rightarrow abC$

3. $bB \rightarrow bb$

4. $bC \rightarrow bc$

5. $CB \rightarrow BC$

6. $cC \rightarrow cc$

Qual a sequência de derivações para a sentença **aabbcc**?

Regra 1	aSBC	(regra 2)
	aabCBC	(regra 5)
	aabBCC	(regra 3)
	aabbCC	(regra 4)
	aabbccC	(regra 6)
	aabbcc	

c) Gramáticas Livres de Contexto (Context Free Grammar ou GLC ou CFG) ou tipo 2. Contem somente produções do tipo $\alpha \rightarrow \beta$ onde $|\alpha| \leq |\beta|$ e $\alpha \in V_N$ (ou seja, em todas as produções o lado esquerdo contem somente um não-terminal, e o lado direito pode conter qualquer sequência de símbolos).

Ex: $L(G_3) = \{a^n ba^n | n \geq 1\}$

$G_3 = (\{S, C\}, \{a, b\}, S, \phi)$ onde ϕ consiste de

1. $S \rightarrow aCa$

2. $C \rightarrow aCa$

3. $C \rightarrow b$

Sentença **aaabaaa**

$S \rightarrow aCa \rightarrow aaCaa \rightarrow aaaCaaa \rightarrow aaabaaa$

d) Gramáticas Regulares : consistem apenas de produções da forma $\alpha \rightarrow \beta$ onde $|\alpha| \leq |\beta|$, $\alpha \in V_N$ e β tem a forma aB ou a onde $a \in V_T$ e $B \in V_N$.

Ex: $L(G_4) = \{a^n b a^m, n, m \geq 1\}$

$G_4 = (\{S, B, C\}, \{a, b\}, S, \phi)$ onde ϕ consiste de

1. $S \rightarrow aS$

2. $S \rightarrow aB$

3. $B \rightarrow bC$

4. $C \rightarrow aC$

5. $C \rightarrow a$

derivação da sentença **aaabaa**

$S \rightarrow aS \rightarrow aaS \rightarrow aaaB \rightarrow aaabC \rightarrow aaabaC \rightarrow aaabaa$

1. Considere a gramática $G=(N,T,P,S)$ onde

$$N = \{S,A,B\}$$

$$T = \{a,b\}$$

$$P = \left\{ \begin{array}{l} S \rightarrow aB \mid bA \\ B \rightarrow b \mid bS \mid aBB \\ A \rightarrow bAA \mid a \mid aS \end{array} \right.$$

Verifique se as sentenças abaixo pertencem a $L(G)$. Caso afirmativo, mostre a derivação correspondente.

a) ababab

b) bbbaaab

c) aabbaabb

[Voltar para o índice](#)

- Uma linguagem regular pode ser reconhecida por um autômato finito.
- Uma linguagem livre de contexto pode ser reconhecida por um autômato de pilha (PDA - Pushdown Automata). Ao contrário dos Autômatos Finitos, autômatos de pilha não-determinísticos são mais poderosos que determinísticos, havendo linguagens livres de contexto que exigem autômatos de pilha não-determinísticos

- Uma linguagem sensível ao contexto pode ser reconhecida por um autômato linear limitado (Máquina de Turing em que a fita está limitada à região que contém a sentença de entrada).
- Uma linguagem enumerável recursivamente pode ser reconhecida por uma máquina de Turing (ou um autômato de duas pilhas, determinístico ou não, que é equivalente à Máquina de Turing).

[Voltar para o índice](#)

- Uma **árvore de derivação** é uma descrição da geração de sentenças durante a derivação.
- Suas propriedades são as seguintes:
 - 1 A raiz é rotulada com o símbolo inicial
 - 2 Cada folha é rotulada com um terminal, não terminal ou ϵ
 - 3 Cada nodo interno é um não terminal
 - 4 Para cada nodo interno A com folhas $X_1, X_2, \dots, X_n$ deve existir na gramática uma produção da forma $A \rightarrow X_1 X_2 \dots X_n$
 - 5 A concatenação dos símbolos das folhas constitui a sentença derivada.

- Ex: Defina uma CFG que gere expressões aritméticas em C. Considere variáveis, constantes e operadores aritméticos como símbolos terminais da gramática.
- Uma **derivação mais à esquerda** é uma derivação onde todos os passos de derivação substituem sempre o não terminal mais à esquerda da forma sentencial.

Ex: Seja $G=(N,T,P,S)$ onde

$N=\{\text{exp}, \text{dig}\}$

$T=\{+, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$S = \text{exp}$

$P = \{$

- $\text{exp} \rightarrow \text{dig},$
- $\text{exp} \rightarrow \text{exp} + \text{exp},$
- $\text{exp} \rightarrow \text{exp} - \text{exp},$
- $\text{dig} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \mid \}$

Qual a derivação mais à esquerda para $3 - 2 + 8$?

$\text{exp} \rightarrow \text{exp} - \text{exp} \rightarrow \text{dig} - \text{exp} \rightarrow 3 - \text{exp} \rightarrow 3 - \text{exp} + \text{exp} \rightarrow$
 $3 - \text{dig} + \text{exp} \rightarrow 3 - 2 + \text{exp} \rightarrow 3 - 2 + \text{dig} \rightarrow 3 - 2 + 8$

[Voltar para o índice](#)

- Uma derivação mais à direita é uma derivação onde sempre se substitui o não terminal mais à direita . Qual a derivação mais à direita para $3 - 2 + 8$
- $\text{exp} \rightarrow \text{exp} - \text{exp} \rightarrow \text{exp} - \text{exp} + \text{exp} \rightarrow \text{exp} - \text{exp} + \text{dig} \rightarrow \text{exp} - \text{exp} + 8$
- $\rightarrow \text{exp} - \text{dig} + 3 \rightarrow \text{exp} - 2 + 8 \rightarrow \text{dig} - 2 + 8 \rightarrow 3 - 2 + 8$

[Voltar para o índice](#)

Exercícios 4.17 do Linguagens Formais, do Ramos

4. Construa gramáticas livres de contexto que gerem as seguintes linguagens.

a) $L_1 = \{a^i b^j c^* | i \neq j\}$;

b) $L_2 = \{a^i b^j c^k | i = j + k\}$;

c) $L_3 = \{a^i b^j c^k | j = i + k\}$;

d) $L_4 = \{a^i b^j c^k | k = i + j\}$;

e) $L_5 = \{a^{k+1} b c^{2k}, k \geq 1\}$;

f) $L_6 = \{a^k (b|c)(d|e)^k, k \geq 0\}$;

g) $L_7 = \{(a|b)^k c (d|e)^k f, k \geq 0\}$;

h) $L_8 = \{a^i b^j | i \leq j \leq 2i\}$;

Exercícios 4.17 do Linguagens Formais, do Ramos

i) $L_9 = \{a^m b^m c^n d^n \mid m \geq 1, n \geq 1\};$

j) $L_{10} = \{a^m b^n c^{n+1} d^{2m} \mid m \geq 1, n \geq 1\};$

k) $L_{11} = \{a^i b^j c^k d^m e^n \mid i > j, j > 0, k \geq 1, m = 2n, n \geq 0\};$

l) $L_{12} = \{d^{2j+1} a^i b c^{i+2} e^j \mid i \geq 0, j \geq 1\};$

m) $L_{13} = \{d^{2j} a^i b c^{i+1} e^j \mid i \geq 0, j \geq 2\};$

n) $L_{14} = \{a^i b^j c^k a^{k+2} b^{j*3} c^{i-1} \mid i \geq 1, j \geq 0, k \geq 0\};$

o) $L_{15} = \{a^m b^n (c(d|\epsilon)e)^{m+n}, m \geq 0, n \geq 0\};$

[Voltar para o índice](#)

Relação entre Autômato Finito e Gramática Regular

Se $G=(N,T,P,S)$ é uma gramática do tipo 3 então existe um autômato finito $M = (K, T, \delta, S, F)$ tal que $L(M)=L(G)$

Construção do Autômato

- M é um AFND
- Os estados de M são as variáveis de G , mais um estado adicional A , não pertencendo a N
- $K = N \cup \{A\}$
- O estado inicial de M é S
- Se P tem a produção $S \rightarrow \epsilon$ então $F=\{S,A\}$ caso contrário $F=\{A\}$
- Transições de M
 - $\delta(B, a) = A$ para cada $B \rightarrow a \in P$
 - $\delta(B, a) = C$ para cada $B \rightarrow aC \in P$

ex:

$$G = \{\{S, B\}, \{0, 1\}, P, S\}$$

$$P: S \rightarrow 0B$$

$$B \rightarrow 0B \mid 1S \mid 0$$

Construir um autômato que aceite $L(G)$

[Voltar para o índice](#)

3)(3 pontos - 96/1) Seja a gramática regular G definida por $G = (\{S, B, C\}, \{a, b, c\}, P, S)$

$$P = \{ \begin{array}{l} S \rightarrow aS \\ S \rightarrow bB \\ B \rightarrow cB \\ B \rightarrow cC \\ C \rightarrow cS \\ C \rightarrow c \end{array} \}$$

Construa um AFD equivalente:

[Voltar para o índice](#)

Conversão de um AF $M = (K, T, \delta, S, F)$ para uma gramática regular $G = (N, T, P, S)$

- Cada estado de M gera um símbolo não-terminal em G .
- O cada símbolo do vocabulário de M é um terminal de G .
- O estado inicial S de M é o símbolo inicial S de G .
- Cada transição $\delta(X, y) = Z$ gera a produção $X \rightarrow yZ$
- Cada estado final X gera a produção $X \rightarrow \epsilon$
- Eventualmente as produções no formato $X \rightarrow \epsilon$ podem ser eliminadas substituindo X por ϵ nas produções em que X aparece, se X não é o símbolo inicial (se X é o símbolo inicial a produção deve ser mantida, porque a linguagem contém a sentença vazia)

Ambigüidade

- Uma gramática é dita ambígua se podemos construir mais de uma árvore de derivação para a mesma sentença.

Ex: $\text{exp} \rightarrow \text{exp} + \text{exp}$

$\text{exp} \rightarrow \text{exp} - \text{exp}$

$\text{exp} \rightarrow \text{num}$

$\text{num} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid \dots \mid 9$

- Mostre duas árvores de derivação para $3 + 2 * 5$
- De modo geral ambigüidade é indesejável em gramáticas, porque sugere a existência de diferentes interpretações para a mesma sentença.
- A gramática a seguir para gerar o comando if é ambígua?

- $\text{Com} \rightarrow \text{if } (E) \text{ Com}$
- $\text{Com} \rightarrow \text{if } (E) \text{ Com else Com}$
- $\text{Com} \rightarrow \text{id} = E;$

Gramática equivalente de precedência maior para o produto (gramática não ambígua):

$\text{exp} \rightarrow \text{exp} + \text{termo} \mid \text{termo}$

$\text{termo} \rightarrow \text{termo} * \text{dig} \mid \text{dig}$

$\text{dig} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid \dots \mid 9$

Árvore de derivação da expressão $3 + 2 * 6$

[Voltar para o índice](#)

Associatividade de Operadores

- Define a ordem em que operadores iguais ou de mesma precedência são avaliados em uma expressão.
- Associativos à esquerda:
 - A grande maioria dos operadores aritméticos (+,-,*,/...)
- Associativos à direita:
 - Operadores de atribuição ($a=b=0$ em C) e potenciação ($^$ no Python, $**$ em php).
 - Obs: No Excel, o operador de potenciação também é o $^$, mas é associativo à esquerda.
 - Assim, 2^3^2 resulta em 512 no Python e 64 no Excel.
- Não Associativos:
 - Operadores relacionais em linguagens tipadas que não permitem expressões como: $a > b > c$
 - Obs: Em ADA o operador $**$ de potenciação não é associativo, o que faz com que " $a ** b ** c$ " resulte em um erro de sintaxe.

Algoritmo para obter uma gramática não ambígua para expressões

- 1. Criar um não terminal para cada nível de precedência e um para as unidades básicas, usa-se para os operadores de maior precedência índices maiores
- 2. Para cada operador **op associativo à esquerda** acrescenta-se na gramática uma produção da forma
$$E_i \rightarrow E_i \text{ op } E_{i+1}$$
 - **Associativo à direita** (ex: potenciação):
$$E_i \rightarrow E_{i+1} \text{ op } E_i$$
 - **Não associativo**
$$E_i \rightarrow E_{i+1} \text{ op } E_{i+1}$$
 - **Unários** que podem ser aplicados repetidamente (ex: Not Not A)
$$E_i \rightarrow \text{op } E_i$$

- **Unários** que não podem ser aplicados repetidamente (ex: @Proc)
 $E_i \rightarrow \text{op } E_{i+1}$
 - se não existir E_{i+1} utiliza-se B
- 3. Acrescenta-se na gramática produções na forma $E_i \rightarrow E_{i+1}$ e $E_i \rightarrow B$ para o último nível.
- 4. Acrescenta-se na gramática uma produção que define B como as formas básicas de expressões como constantes, variáveis e expressões entre parênteses.

Remova a ambigüidade da gramática a seguir:

$$\begin{aligned} E \rightarrow & E + E \mid E - E \mid E * E \mid E / E \mid E \text{ div } E \mid E \text{ mod } E \mid \\ & E \text{ and } E \mid E \text{ or } E \mid E \text{ xor } E \mid E \text{ shl } E \mid E \text{ shr } E \mid \\ & E = E \mid E <> E \mid E < E \mid \\ & \text{not } E \mid @ E \\ E \rightarrow & 0 \mid 1 \mid 2 \mid 3 \mid \dots \mid 9 \end{aligned}$$

Precedência Operadores

- | | |
|---|-------------------------------|
| 1 | @, not |
| 2 | *, /, div, mod, and, shl, shr |
| 3 | +, -, or, xor |
| 4 | =, <>, <, >, <=, >=, in |

Operadores relacionais : não associativos

$$E_i \rightarrow E_{i+1} \text{ op } E_{i+1}$$

$$E_0 \rightarrow E_1 \text{ op } E_1$$

$$E_0 \rightarrow E_1 = E_1 | E_1 < > E_1 | E_1 < E_1 | E_1 > E_1 | \\ E_1 < = E_1 | E_1 > = E_1 | E_1 \text{ in } E_1 | E_1$$

Operadores associativos à esquerda

$$E_i \rightarrow E_i \text{ op } E_{i+1}$$

$$E_1 \rightarrow E_1 \text{ op } E_2$$

$$E_1 \rightarrow E_1 + E_2 | E_1 - E_2 | E_1 \text{ or } E_2 | E_1 \text{ xor } E_2 | E_2$$

[Voltar para o índice](#)

Operadores associativos à esquerda

$$E_i \rightarrow E_i \text{ op } E_{i+1}$$

$$E_2 \rightarrow E_2 \text{ op } E_3$$

$$E_2 \rightarrow E_2 * E_3 \mid E_2 / E_3 \mid E_2 \bmod E_3 \mid E_2 \text{ and } E_3 \mid E_2 \text{ shl } E_3 \mid E_2 \text{ shr } E_3 \mid E_3$$

Operadores unários

$$E_i \rightarrow \text{op } E_{i+1}$$

$E_3 \rightarrow @ \mid \text{not } B \mid B$ (como não há operador de maior prioridade utiliza-se B)

$$B \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \mid (E_0)$$

Ex: Árvore de derivação da expressão $3 + 2 > 2 * 5 - 3 / 2$

Ex: Árvore de derivação da expressão $(3+2) * 5$

- Ex: Árvore de derivação da expressão $3 \text{ and } 4 > 2$.
 - Esta expressão é semanticamente incorreta mas sintaticamente correta.
 - A análise sintática não se preocupa com os tipos dos operandos.
 - Simplesmente analisa se estão de forma sintática corretos.

Recursão à esquerda

Ocorre se algum não terminal A deriva uma forma sentencial $A\alpha$, ou seja $A \rightarrow A\alpha$.

Recursão imediata

$$S \rightarrow \begin{array}{l} S0 \mid (\text{parte recursiva}) \\ 1 \mid (\text{parte não recursiva}) \end{array}$$

Ex: 100

Recursão não imediata

$$\begin{array}{l} S \rightarrow \begin{array}{l} Aa \mid (\text{parte recursiva}) \\ b \mid (\text{parte não recursiva}) \end{array} \\ A \rightarrow Ac \mid Sd \mid \epsilon \end{array}$$

[Voltar para o índice](#)

Algoritmo para remoção de Recursão à esquerda

a) Agrupar as produções recursivas na forma:

$$A \rightarrow A\alpha_1 | A\alpha_2 | A\alpha_3 | \dots | A\alpha_n \text{ (parte recursiva)} \\ \beta_1 | \beta_2 | \beta_m \text{ (parte não recursiva)}$$

b) Substituir esta produção pelas produções :

$$A \rightarrow \beta_1 B | \beta_2 B | \beta_m B$$

$$B \rightarrow \alpha_1 B | \alpha_2 B | \alpha_3 B | \dots | \alpha_n B | \epsilon$$

[Voltar para o índice](#)

ex: $S \rightarrow Sa|Sb|c|d$

resulta em

$$S \rightarrow cS'|dS'$$
$$S' \rightarrow aS'|bS'|\epsilon$$

[Voltar para o índice](#)

Para remover recursão não imediata substituem-se não terminais pelo lado direito das produções até que a recursão seja imediata

$$S \rightarrow Aa|b$$

$$A \rightarrow Ac|Sd|\epsilon$$

removendo a recursão da segunda produção

$$S \rightarrow Aa|b$$

$$A \rightarrow B|SdB$$

$$B \rightarrow cB|\epsilon$$

substituindo a segunda na terceira

$$S \rightarrow Ba|SdBa|b$$

$$B \rightarrow cB|\epsilon$$

[Voltar para o índice](#)

- Remova a recursão à esquerda das gramáticas a seguir:

- a)

$$A \rightarrow Abc \mid Acd \mid cdB \mid c$$

$$B \rightarrow Bc \mid dd$$

- As duas produções da gramática tem recursão imediata. A recursão de cada produção é removida separadamente:

$$A \rightarrow cdBA' \mid cA'$$

$$A' \rightarrow bcA' \mid cdA' \mid \epsilon$$

$$B \rightarrow ddB'$$

$$B' \rightarrow cB' \mid \epsilon$$

■ b)

$$S \rightarrow Aab \mid Bc \mid ScAb$$

$$A \rightarrow Sac \mid BaA \mid ab$$

$$B \rightarrow Ac \mid aBb \mid ab$$

- Há recursão imediata em $S \rightarrow ScAb$ e há diversas recursões não imediatas. Algumas delas são:

■ $S \rightarrow A \rightarrow S$

■ $S \rightarrow B \rightarrow A \rightarrow S$

■ $A \rightarrow B \rightarrow A$

- Para eliminar as recursões não imediatas através de B substituímos B em A e S

$$S \rightarrow Aab \mid Bc \mid ScAb$$

$$A \rightarrow Sac \mid BaA \mid ab$$

$$B \rightarrow Ac \mid aBb \mid ab$$

- resultando

$$S \rightarrow Aab \mid Acc \mid aBbc \mid abc \mid ScAb$$

$$A \rightarrow Sac \mid AcaA \mid aBbaA \mid abaA \mid ab$$

$$B \rightarrow Ac \mid aBb \mid ab$$

- Restaram recursões imediatas em S e A, e a recursão não imediata $S \rightarrow A \rightarrow S$.

- Antes de substituir A em S , remove-se a recursão imediata em A .

$$\begin{aligned} S &\rightarrow Aab \mid Acc \mid aBbc \mid abc \mid ScAb \\ A &\rightarrow Sac \mid AcaA \mid aBbaA \mid abaA \mid ab \\ B &\rightarrow Ac \mid aBb \mid ab \end{aligned}$$

- resultando

$$\begin{aligned} S &\rightarrow Aab \mid Acc \mid aBbc \mid abc \mid ScAb \\ A &\rightarrow SacA' \mid aBbaAA' \mid abaAA' \mid abA' \\ A' &\rightarrow caAA' \mid \epsilon \\ B &\rightarrow Ac \mid aBb \mid ab \end{aligned}$$

- Restaram a recursão imediata em S e a não imediata $S \rightarrow A \rightarrow S$.

- Substituindo A em S para transformar a recursão não imediata em imediata...

$$S \rightarrow Aab \mid Acc \mid aBbc \mid abc \mid ScAb$$

$$A \rightarrow SacA' \mid aBbaAA' \mid abaAA' \mid abA'$$

$$A' \rightarrow caAA' \mid \epsilon$$

$$B \rightarrow Ac \mid aBb \mid ab$$

- resultando

$$S \rightarrow SacA'ab \mid aBbaAA'ab \mid abaAA'ab \mid abA'ab \mid SacA'cc \mid$$

$$aBbaAA'cc \mid abaAA'cc \mid abA'cc \mid aBbc \mid abc \mid ScAb$$

$$A \rightarrow SacA' \mid aBbaAA' \mid abaAA' \mid abA'$$

$$A' \rightarrow caAA' \mid \epsilon$$

$$B \rightarrow Ac \mid aBb \mid ab$$

- Restou somente a recursão direta em S.

- Removendo a recursão imediata em S...

$$\begin{aligned} S &\rightarrow \text{SacA}'ab \mid a\text{BbaAA}'ab \mid aba\text{AA}'ab \mid ab\text{A}'ab \mid \text{SacA}'cc \mid \\ &a\text{BbaAA}'cc \mid aba\text{AA}'cc \mid ab\text{A}'cc \mid a\text{Bbc} \mid abc \mid \text{ScAb} \\ A &\rightarrow \text{SacA}' \mid a\text{BbaAA}' \mid aba\text{AA}' \mid ab\text{A}' \\ A' &\rightarrow ca\text{AA}' \mid \epsilon \\ B &\rightarrow \text{Ac} \mid a\text{Bb} \mid ab \end{aligned}$$

- resultando

$$\begin{aligned} S &\rightarrow aBbaAA'abS' \mid abaAA'abS' \mid abA'abS' \mid aBbaAA'ccS' \mid \\ &abaAA'ccS' \mid abA'ccS' \mid aBbcS' \mid abcS' \\ S' &\rightarrow acA'abS' \mid acA'ccS' \mid cAbS' \mid \epsilon \\ A &\rightarrow SacA' \mid aBbaAA' \mid abaAA' \mid abA' \\ A' &\rightarrow caAA' \mid \epsilon \\ B &\rightarrow Ac \mid aBb \mid ab \end{aligned}$$

■ c)

$$A \rightarrow Ac \mid Bd \mid cd$$

$$B \rightarrow ac \mid Bd \mid Ac$$

■ removendo a recursão imediata em B resulta

$$A \rightarrow Ac \mid Bd \mid cd$$

$$B \rightarrow acB' \mid AcB'$$

$$B' \rightarrow dB' \mid \epsilon$$

■ substituindo B em A resulta

$$A \rightarrow Ac \mid acB'd \mid AcB'd \mid cd$$

$$B \rightarrow acB' \mid AcB'$$

$$B' \rightarrow dB' \mid \epsilon$$

■ B pode ser eliminado, porque não é o símbolo inicial da gramática e não é usado em nenhuma outra produção.

■ Removendo a recursão imediata em A

$$A \rightarrow Ac \mid acB'd \mid AcB'd \mid cd$$

$$B' \rightarrow dB' \mid \epsilon$$

■ resulta

$$A \rightarrow acB'dA' \mid cdA'$$

$$A' \rightarrow cA' \mid cB'dA' \mid \epsilon$$

$$B' \rightarrow dB' \mid \epsilon$$

[Voltar para o índice](#)

■ d)

$$S \rightarrow Aa \mid Bb \mid Sac$$
$$A \rightarrow Sab \mid ab$$
$$B \rightarrow a \mid Ba$$

■ Removendo a recursão imediata de B...

$$S \rightarrow Aa \mid Bb \mid Sac$$
$$A \rightarrow Sab \mid ab$$
$$B \rightarrow aB'$$
$$B' \rightarrow aB' \mid \epsilon$$

- Substituindo A em S

$$S \rightarrow Aa \mid Bb \mid Sac$$
$$A \rightarrow Sab \mid ab$$
$$B \rightarrow aB'$$
$$B' \rightarrow aB' \mid \epsilon$$

- Resulta...

$$S \rightarrow Saba \mid aba \mid Bb \mid Sac$$
$$A \rightarrow Sab \mid ab$$
$$B \rightarrow aB'$$
$$B' \rightarrow aB' \mid \epsilon$$

- e A pode ser eliminado

■ Removendo a recursão imediata em S

$$S \rightarrow Saba \mid aba \mid Bb \mid Sac$$
$$B \rightarrow aB'$$
$$B' \rightarrow aB' \mid \epsilon$$

■ Resulta...

$$S \rightarrow abaS' \mid BbS'$$
$$S' \rightarrow abaS' \mid acS' \mid \epsilon$$
$$B \rightarrow aB'$$
$$B' \rightarrow aB' \mid \epsilon$$

[Voltar para o índice](#)

■ e)

$S \rightarrow A0 \mid 0B1 \mid 11C$

$A \rightarrow 01 \mid 0BC \mid S0$

$B \rightarrow B01 \mid 01A$

$C \rightarrow 01 \mid 10$

■ f)

$S \rightarrow 01S \mid 00A \mid S10 \mid A01$

$A \rightarrow 00D \mid 01S \mid A00 \mid S11 \mid 0B$

$B \rightarrow B00 \mid B11 \mid C01 \mid 001$

$C \rightarrow B10 \mid C00 \mid 01C \mid 01B \mid 0$

$D \rightarrow D01 \mid D00 \mid 01 \mid 00$

■ g)

$$S \rightarrow SaB \mid Sbc \mid Acd$$

$$A \rightarrow ABc \mid Acd \mid Bcd \mid Cd$$

$$B \rightarrow Acd \mid Bcc \mid Cdc$$

$$C \rightarrow Ac \mid Cd \mid dc$$

■ h)

$$S \rightarrow Cab \mid Ab \mid b$$

$$A \rightarrow Bcd \mid Ac \mid ab$$

$$B \rightarrow Aca \mid Bc \mid Cba \mid a$$

$$C \rightarrow DaC \mid Cc \mid c$$

$$D \rightarrow ac \mid CaD$$

■ i)

$$E \rightarrow E+T \mid E-T \mid T$$
$$T \rightarrow T * F \mid T / F \mid F$$
$$F \rightarrow F ** P \mid P$$
$$P \rightarrow (E) \mid \text{id} \mid \text{cte}$$

[Voltar para o índice](#)

■ j)

$$S \rightarrow BaS \mid \epsilon$$

$$B \rightarrow SAa \mid Bb$$

$$A \rightarrow Sa \mid \epsilon$$

■ removendo a recursão direta de B resulta

$$S \rightarrow BaS \mid \epsilon$$

$$B \rightarrow SAaB'$$

$$B' \rightarrow bB' \mid \epsilon$$

$$A \rightarrow Sa \mid \epsilon$$

[Voltar para o índice](#)

■ substituindo B em S

$$S \rightarrow BaS \mid \epsilon$$

$$B \rightarrow SAaB'$$

$$B' \rightarrow bB' \mid \epsilon$$

$$A \rightarrow Sa \mid \epsilon$$

■ resulta

$$S \rightarrow SAaB'aS \mid \epsilon$$

$$B \rightarrow SAaB'$$

$$B' \rightarrow bB' \mid \epsilon$$

$$A \rightarrow Sa \mid \epsilon$$

■ e B pode ser eliminado

- removendo a recursão direta em S

$$S \rightarrow SAaB'aS \mid \epsilon$$

$$B' \rightarrow bB' \mid \epsilon$$

$$A \rightarrow Sa \mid \epsilon$$

- resulta

$$S \rightarrow S'$$

$$S' \rightarrow AaB'aSS' \mid \epsilon$$

$$B' \rightarrow bB' \mid \epsilon$$

$$A \rightarrow Sa \mid \epsilon$$

- resta a recursão não imediata $S \rightarrow S' \rightarrow A \rightarrow S$

- substituindo A em S'

$$S \rightarrow S'$$

$$S' \rightarrow AaB'aSS' \mid \epsilon$$

$$B' \rightarrow bB' \mid \epsilon$$

$$A \rightarrow Sa \mid \epsilon$$

- resulta

$$S \rightarrow S'$$

$$S' \rightarrow SaaB'aSS' \mid aB'aSS' \mid \epsilon$$

$$B' \rightarrow bB' \mid \epsilon$$

$$A \rightarrow Sa \mid \epsilon$$

- e A pode ser eliminado. E substituindo S' em S resulta

$$S \rightarrow SaaB'aSS' \mid aB'aSS' \mid \epsilon$$

$$S' \rightarrow SaaB'aSS' \mid aB'aSS' \mid \epsilon$$

$$B' \rightarrow bB' \mid \epsilon$$

- e eliminando a recursão imediata em S...

$$S \rightarrow SaaB'aSS' \mid aB'aSS' \mid \epsilon$$

$$S' \rightarrow SaaB'aSS' \mid aB'aSS' \mid \epsilon$$

$$B' \rightarrow bB' \mid \epsilon$$

- resulta

$$S \rightarrow aB'aSS'S'' \mid S''$$

$$S'' \rightarrow aaB'aSS'S'' \mid \epsilon$$

$$S' \rightarrow SaaB'aSS' \mid aB'aSS' \mid \epsilon$$

$$B' \rightarrow bB' \mid \epsilon$$

Conversão de uma gramática regular para uma expressão regular

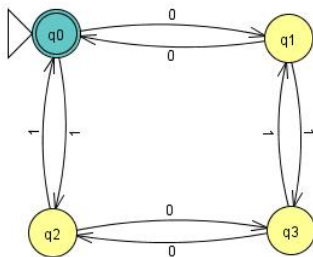
- Cada produção de uma gramática regular pode ser vista como uma expressão regular que descreve as sentenças derivadas a partir dela.
- Assim, a produção $A \rightarrow aA|b$ descreve a linguagem a^*b
- E a produção $S \rightarrow \alpha_1 S | \alpha_2 S | \dots | \beta_1 | \beta_2 | \dots$ descreve a linguagem $(\alpha_1 | \alpha_2 | \dots)^*(\beta_1 | \beta_2 | \dots)$

[Voltar para o índice](#)

Algoritmo para converter uma gramática regular em uma expressão regular

- Para cada produção P recursiva à direita:
 - agrupar as partes recursivas e não recursivas da produção deixando-a na forma: $S \rightarrow \alpha_1 S | \alpha_2 S | \dots | \beta_1 | \beta_2 | \dots$
 - substituir o lado direito da produção pela forma $(\alpha_1 | \alpha_2 | \dots)^*(\beta_1 | \beta_2 | \dots)$
- Para cada produção P , com exceção da produção do símbolo inicial:
 - Substituir todas as ocorrências de P das outras produções, substituindo-as pelo seu lado direito da produção, eliminando assim o símbolo P da gramática.
 - Se nessa substituição alguma produção voltar a ser recursiva, aplica-se novamente o passo anterior.

Encontre a E.R. equivalente ao autômato abaixo:



[Voltar para o índice](#)

- Em primeiro lugar extraí-se a Gramática Regular.
- Para simplificar, vamos associar o símbolo **A** ao estado **q0**, **B** ao estado **q1**, **C** ao estado **q2** e **D** ao estado **q3**.
- A gramática resultante é:

$$A \rightarrow 0B|1C|\epsilon$$

$$B \rightarrow 0A|1D$$

$$C \rightarrow 1A|0D$$

$$D \rightarrow 1B|0C$$

- Substituindo **D** nas produções **B** e **C** resulta:

$$A \rightarrow 0B|1C|\epsilon$$

$$B \rightarrow 0A|11B|10C$$

$$C \rightarrow 1A|01B|00C$$

- $A \rightarrow 0B|1C|\epsilon$
- $B \rightarrow 0A|11B|10C$
- $C \rightarrow 1A|01B|00C$
- Essa substituição torna B e C recursivas.
- Agrupando as partes recursivas e não recursivas...:
 - $A \rightarrow 0B|1C|\epsilon$
 - $B \rightarrow 11B|10C|0A$
 - $C \rightarrow 00C|1A|01B$
- e removendo a recursão:
 - $A \rightarrow 0B|1C|\epsilon$
 - $B \rightarrow (11)^*(10C|0A)$
 - $C \rightarrow (00)^*(1A|01B)$

- $A \rightarrow 0B|1C|\epsilon$
- $B \rightarrow (11)^*(10C|0A)$
- $C \rightarrow (00)^*(1A|01B)$
- Substituindo **C** em **B** e **A**:
 - $A \rightarrow 0B|1(00)^*(1A|01B)|\epsilon$
 - $B \rightarrow (11)^*(10(00)^*(1A|01B)|0A)$
- Separando 1A e 01B:
 - $A \rightarrow 0B|1(00)^*(1A|01B)|\epsilon$
 - $B \rightarrow (11)^*(10(00)^*1A|10(00)^*01B|0A)$

[Voltar para o índice](#)

Exercícios

1) Defina uma CFG que gere sentenças em $\{a, b\}^*$ tal que cada sentença contenha igual número de a's e b's

2) seja a gramática regular G definida por :

$$G = (\{S, B, C\}, \{a, b, c\}, P, S)$$
$$P = \left\{ \begin{array}{l} S \rightarrow aS \\ S \rightarrow bB \\ B \rightarrow cB \\ B \rightarrow cC \\ C \rightarrow cS \\ C \rightarrow c \end{array} \right\}$$

Construa um autômato finito determinístico e a expressão regular equivalente.

3) Defina um autômato finito que reconheça sentenças em $\{a, b\}^*$ que contenham um número par de a's e um número par de b's. Escreva uma gramática regular equivalente.

[Voltar para o índice](#)

4) Considere os seguintes operadores utilizados nas expressões da linguagem de algoritmos utilizada na disciplina de algoritmos

Operador	Precedência	Associatividade
$\wedge$	0	$\leftarrow$
$*$ $\backslash$	1	$\rightarrow$
$+-$	2	$\rightarrow$
Relacionais	3	não associativos
E	4	$\rightarrow$
OU	5	$\rightarrow$

- Escreva uma gramática não ambígua que gere expressões lógicas em linguagem de algoritmos incluindo parênteses e permitindo como elementos da expressão constantes, variáveis ou chamadas de funções como argumentos

Fatoração

- Uma gramática fatorada é uma gramática onde não existem duas produções para um mesmo não terminal que tenham o mesmo prefixo próprio.
- Ex : $S \rightarrow aaS \mid abS \mid c$ - gramática não é fatorada porque aaS e abS têm o mesmo prefixo próprio (a).
- Gramática equivalente fatorada:

$$\begin{aligned} S &\rightarrow aS' \mid c \\ S' &\rightarrow aS \mid bS \end{aligned}$$

- Árvore de derivação de **ababc???**

Fatoração

- Uma GLC está fatorada se ela é determinística, ou seja se ela não possui produções para um mesmo não-terminal no lado esquerdo cujo lado direito inicie com a mesma cadeia de símbolos ou com símbolos que derivam seqüências que iniciem com a mesma cadeia de símbolos.
- De acordo com esta definição, a seguinte gramática é não-determinística:
 - $S \rightarrow aSB \mid aSA$
 - $A \rightarrow a$
 - $B \rightarrow b$

[Voltar para o índice](#)

- Para fatorar uma GLC, deve-se alterar as produções envolvidas no não-determinismo da seguinte forma:

a) As produções com não-determinismo direto da forma

$$A \rightarrow \alpha\beta \mid \alpha\gamma$$

serão substituídas por:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta \mid \gamma$$

[Voltar para o índice](#)

- Na gramática anterior, temos $\alpha = aS$, assim, com a eliminação do não-determinismo, teríamos a seguinte gramática :

$$S \rightarrow aSS'$$

$$S' \rightarrow B \mid A$$

$$A \rightarrow a$$

$$B \rightarrow b$$

[Voltar para o índice](#)

- b) Não-determinismo indireto é retirado através de sua transformação em não-determinismo direto (através de derivações sucessivas) e posterior eliminação segundo o item (a).
- Tomemos por exemplo a seguinte gramática :

$S \rightarrow AC \mid BD$

$A \rightarrow aD \mid cB$

$B \rightarrow aB \mid dD$

$C \rightarrow eC \mid eA$

$D \rightarrow fD \mid AB$

- Onde o não-determinismo indireto se dá pelo fato de que A e B podem iniciar com o terminal a. Assim, transformaremos primeiro o não-determinismo indireto em não-determinismo direto, substituindo os não-terminais A e B pelas suas produções em S :

$$S \rightarrow aDC \mid cBC \mid aBD \mid dDD$$

$$A \rightarrow aD \mid cB$$

$$B \rightarrow aB \mid dD$$

$$C \rightarrow eC \mid eA$$

$$D \rightarrow fD \mid AB$$

- O próximo passo é simples, bastando agora eliminar o não-determinismo direto, segundo a regra do item a, ou seja:

$$S \rightarrow aS' \mid cBC \mid dDD$$

$$S' \rightarrow DC \mid BD$$

$$A \rightarrow aD \mid cB$$

$$B \rightarrow aB \mid dD$$

$$C \rightarrow eC'$$

$$C' \rightarrow C \mid A$$

$$D \rightarrow fD \mid AB$$

- Dica : Fatoração.
 - 1º Eliminar indeterminismo direto.
 - 2º Eliminar indeterminismo indireto.
 - 3º Fazer substituições para descobrir indeterminismos indiretos.
- Obs : fazer 1º os diretos e depois os indiretos.

- Efetue a fatoração das gramáticas a seguir:

a)

$$S \rightarrow V = \text{exp} \mid \text{id} (E)$$

$$V \rightarrow \text{id} [E] \mid \text{id}$$

$$E \rightarrow \text{exp} + E \mid \text{exp}$$

b)

$$S \rightarrow bcD \mid Bcd$$

$$B \rightarrow bB \mid b$$

$$D \rightarrow dD \mid d$$

[Voltar para o índice](#)

c)

$$S \rightarrow aA \mid aB$$

$$A \rightarrow aA \mid a$$

$$B \rightarrow b$$

d)

$$S \rightarrow Ab \mid ab \mid baA$$

$$A \rightarrow aab \mid b$$

[Voltar para o índice](#)

e)

$S \rightarrow Abc \mid bBC \mid bCD$

$A \rightarrow aBC \mid aDC$

$B \rightarrow dCc \mid dc$

$C \rightarrow Acd \mid cd$

$D \rightarrow aBC \mid abc \mid aC$

[Voltar para o índice](#)

f)

$S \rightarrow 1A0 \mid 1B1$

$A \rightarrow 1A0 \mid 100$

$B \rightarrow C10 \mid C01$

$C \rightarrow 11C \mid 11D$

$D \rightarrow 10 \mid 11$

[Voltar para o índice](#)

g)

$S \rightarrow 10D \mid 11C \mid 0B$

$B \rightarrow 1CD \mid 101B \mid 01$

$C \rightarrow 101C \mid 1B0 \mid 00$

$D \rightarrow 00B \mid 011D \mid 110$

h)

$$S \rightarrow aBd \mid acD \mid bC$$
$$B \rightarrow bDc \mid bCd \mid ad$$
$$D \rightarrow cdD \mid caB$$
$$C \rightarrow cbB \mid adD$$

[Voltar para o índice](#)

i)

$$S \rightarrow bcD \mid Bcd$$
$$B \rightarrow bB \mid b$$
$$D \rightarrow dD \mid d$$

j)

$$C \rightarrow V=\text{exp} \mid \text{id}(E)$$
$$V \rightarrow \text{id}[E] \mid \text{id}$$
$$E \rightarrow \text{exp}, E \mid \text{exp}$$

[Voltar para o índice](#)

Exercícios de gramáticas

- 5) Defina uma CFG que gere sentenças em $\{a, b\}^*$ tal que cada sentença seja igual à sua reversa.
- 6) Defina uma CFG que gere sentenças em $\{a, b\}^*$ tal que a's e b's apareçam alternados nas sentenças.
- 7) Escreva uma DFA equivalente à expressão regular $(a|b)^*a(a|b)(a|b)$ e a gramática equivalente.

[Voltar para o índice](#)

8) Mostre por indução que a gramática

$$S \rightarrow aSbS \mid bSaS \mid \epsilon$$

gera a linguagem formada por todas as sentenças sobre $\{a, b\}$ que tenham o mesmo número de a's e b's.

Deve-se mostrar que:

a) A gramática gera somente sentenças com o mesmo número de a's e b's.

b) A primeira produção é utilizada para gerar qualquer sentença iniciada pela letra a. Toda sentença equilibrada (que tenha o mesmo número de a's e b's) e que comece pela letra a pode ser representada da forma aw_b onde w_b é uma sentença com um b a mais. Para mostrar que a produção $S \rightarrow aSbS$ gera todas as sentenças equilibradas iniciadas pela letra a, basta mostrar que toda sentença w_b pode ser representada na forma SbS . Considerando...

Lista de Tópicos da Primeira Área

- Defina uma ER, AFD, AF, Gramática...
- Conversão entre ER, AF e Gramáticas regulares
- Conversão AF-e $\rightarrow$ AFN $\rightarrow$ AFD
- Minimizar AF
- Remoção de ambiguidade de gramática
- Remoção de recursão à esquerda
- Fatoração de gramática

[Voltar para o índice](#)

Análise Sintática (parsing)

- É o processo de determinar se uma seqüência de tokens pode ser gerada por uma gramática.
- Os analisadores sintáticos (também chamados de parsers) podem ser classificados em:
 - Top Down : a árvore sintática é construída da raiz para as folhas
 - Bottom Up : a árvore sintática é construída das folhas em direção à raiz
- O primeiro permite que se construa parsers mais facilmente e o segundo cobre uma faixa mais ampla de gramáticas.

- A análise descendente (top-down) pode ser vista como uma tentativa de construir uma árvore de derivação em pré-ordem (raiz, esquerda, direita)
- Existem três tipos de analisadores sintáticos descendentes :
 - 1 Descendente recursivo com retrocesso (back-tracking)
 - 2 Descendente recursivo preditivo
 - 3 Tabular preditivo

[Voltar para o índice](#)

- Nos dois primeiros, para cada não terminal é implementada uma rotina que efetua o reconhecimento do lado direito da produção que define aquele símbolo.
- O terceiro tipo, tabular preditivo, é implementado através de um autômato de pilha controlado por uma tabela de análise a qual indica qual a produção a ser aplicada, relativa ao símbolo no topo da pilha.

[Voltar para o índice](#)

Exemplo de descendente recursivo

- Um reconhecedor para a gramática $S \rightarrow aSb|c$ pode ser implementado pela função a seguir:

```
int S(){  
    if (tkAtual==TK_a)  
        { le_token();  
          if (S())  
              if (tkAtual==TK_b)  
                  { le_token(); return TRUE;}  
                  else return FALSE;  
          else return FALSE;}  
    else if (tk_Atual==TK_c)  
        { le_token(); return TRUE;}  
        else return FALSE;  
}
```

Observações

- As funções que implementam cada produção da gramática retornam um valor lógico:
 - **verdadeiro**: caso a sentença tenha sido reconhecida até o fim da produção
 - **falso**: caso tenha havido algum erro no reconhecimento antes de chegar até o final do lado direito da produção
- Na função é utilizada a figura do token atual, que é o último token reconhecido na sentença até o momento.
 - O token atual é atualizado pela função `le_token()`, que faz uma chamada ao analisador léxico e busca o próximo token na sentença sendo reconhecida (ou, se for o caso, no arquivo fonte do programa sendo reconhecido)

- No reconhecimento do lado direito de uma produção, símbolos **não-terminais** são representados por uma chamada da função correspondente e símbolos **terminais** são representados pela comparação do token atual com o terminal correspondente na produção;
 - Ao chamar a função correspondente a um não-terminal, assume-se que o token corrente é o primeiro da parte da sentença que será derivada daquele não terminal.
 - Ao retornar da função correspondente a um não-terminal , assume-se que o token corrente é o primeiro imediatamente à parte da sentença que será derivada daquele não terminal

- No reconhecimento do lado direito de uma produção, símbolos **terminais** são representados pela comparação do token atual com o terminal correspondente na produção;
 - Se o token atual é o terminal esperado naquele ponto da produção, deve-se avançar o token no reconhecimento da sentença (através de uma chamada do `le_token`)
 - Se a sentença estiver sendo lida de um string, a `le_token` pode ser simplesmente incrementar o índice do string

[Voltar para o índice](#)

Análise recursiva com retrocesso

- Considere a gramática abaixo:

$$S \rightarrow aB \mid aC$$
$$B \rightarrow ab$$
$$C \rightarrow ac$$

- A não ser que a gramática seja modificada, não é possível, dentro da função **S**, verificar se após o reconhecimento de **a** deverá ser chamada a função **B** ou **C**
- Uma alternativa é, após o reconhecimento de **a**, chamar a função **B** e, caso ela falhe, chamar a função **C**
- A função **B** pode avançar no reconhecimento da sentença e, caso ela falhe, a situação anterior à chamada (representada pelo token atual) deverá ser restaurada antes de se chamar a função **C**.

- Essa gramática pode ser implementada pela função a seguir:

```
int S(){
if (tk_Atual==tk_A)
{le_token(); marca_pos(pos);
 if (B()) return TRUE;
 else
   {restaura(pos);
    if (C()) return TRUE;
    else return FALSE;}
}
else return FALSE;
```

[Voltar para o índice](#)

- A função `marca_pos(pos)` armazena em `pos` a posição corrente da sentença sendo reconhecida.
- Essa posição corrente pode ser:
 - a posição atual do string, caso a sentença esteja sendo reconhecida de um string
 - a posição corrente do arquivo, caso a sentença esteja sendo lida de um arquivo
 - pode ser necessário também salvar o valor do token atual
- A função `restaura(pos)` restaura a posição atual do reconhecimento da sentença e, possivelmente, o token corrente

- Considere a seguinte gramática para expressões regulares (considere que nessa gramática o $+$ representa o operador de união e o ϵ representa a sentença vazia):

$\text{exp} \rightarrow \text{exp} + \text{exp1} \mid \text{exp1}$

$\text{exp1} \rightarrow \text{exp1} \text{ exp2} \mid \text{exp2}$

$\text{exp2} \rightarrow \text{exp2} * \mid \text{exp3}$

$\text{exp3} \rightarrow a \mid b \mid c \mid \epsilon \mid (\text{exp})$

- Escreva um parser descendente recursivo.
- O programa deve ler um string com uma expressão regular e verificar se o string é uma expressão regular válida ou não.
 - Como cada token possui exatamente um caractere, não há análise léxica, o token corrente é a posição corrente do string e a `le_token` é o incremento da posição do string.

Análise Recursiva Preditiva

- É análoga à análise recursiva com retrocesso, mas o próximo símbolo de entrada identifica com certeza qual a produção a ser utilizada.
- A gramática não pode ter recursividade à esquerda e deve ser fatorada.

Ex: $\langle \text{comando} \rangle \rightarrow$ if $\langle \text{expr} \rangle$ then $\langle \text{comando} \rangle$ |
while $\langle \text{expr} \rangle$ do $\langle \text{comando} \rangle$ |
repeat $\langle \text{lista} \rangle$ until $\langle \text{expr} \rangle$ |
id := $\langle \text{expr} \rangle$

[Voltar para o índice](#)

Ex2: <comando> → <condicional> |
 <iterativo> |
 <atribuição>
 <condicional> → if <expr> then <comando>
 <iterativo> → repeat <lista> until <expr> |
 while <expr> do <comando>
 <atribuição> → id := <expr>

[Voltar para o índice](#)

Dada a gramática:

$\text{Decl} \rightarrow \text{Decl ListaDecl} \mid \text{ListaDecl}$

FALTA

Construir um parser descendente recursivo preditivo

[Voltar para o índice](#)

$\text{Prog} \rightarrow \text{Decl Comandos}$
 $\text{Decl} \rightarrow \text{Lista_Decl Decl1}$
 $\text{Decl1} \rightarrow \text{ListaDecl Decl1} \mid \epsilon$
 $\text{ListaDecl} \rightarrow \text{ListaVar : Tipo;}$
 $\text{ListaVar} \rightarrow \text{id ListaVar1}$
 $\text{ListaVar1} \rightarrow \text{,id ListaVar1} \mid \epsilon$
 $\text{Tipo} \rightarrow \text{Simplex} \mid \text{id} \mid \text{array [num] of Tipo}$
 $\text{Simplex} \rightarrow \text{integer} \mid \text{char}$

Voltar para o índice

```
char Decl()  
{  
  if (token==id)  
    if ListaDecl()==TRUE
```

[Voltar para o índice](#)

Faça um parser descendente recursivo para reconhecimento de expressões relacionais em pascal.

[Voltar para o índice](#)

Parser Tabular Preditivo

- Neste tipo de parser a pilha de execução é montada explicitamente, ao invés de implicitamente nas chamadas recursivas.
- O problema chave é determinar qual produção aplicar para um não terminal.
- O parser trabalha analisando o símbolo no topo da pilha (X) e o símbolo corrente na entrada (a).
 - 1 Se $X = a = \$ \Rightarrow$ sucesso (significa que consumiu todos os símbolos da pilha)
 - 2 Se $X = a \neq \$ \Rightarrow$ tira X da pilha e avança entrada
 - 3 Se X é não terminal $\Rightarrow$ o programa consulta a entrada $M(X,a)$ da tabela M , esta entrada pode ser uma produção de X ou um erro. Se for uma produção de X , X é substituído na pilha pelos símbolos do lado direito da produção (em ordem inversa). Caso a entrada da tabela seja um erro, a respectiva rotina de tratamento de erro é chamada.

Considere a gramática abaixo que gera expressões aritméticas

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

que equivale a:

$$E \rightarrow T E'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid \text{id}$$

[Voltar para o índice](#)

Uma tabela de análise preditiva para esta gramática é mostrada a seguir:

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

[Voltar para o índice](#)

Sentença : id + id * id \$

Pilha:

\$E

\$E'T

\$E'T'F

\$E'T'

\$E'T+T'

\$E'T+

\$E'T'F+

\$E'T'

\$E'T'F*

\$E'T'

\$

[Voltar para o índice](#)

A construção de um analisador tabular é auxiliada por duas funções associadas à gramática G , as funções FIRST e FOLLOW.

Se α é uma seqüência de símbolos da gramática, então $\text{FIRST}(\alpha)$ é o conjunto de terminais que iniciam sentenças derivadas de α . Se $\alpha \rightarrow \epsilon$ então ϵ está em $\text{FIRST}(\alpha)$.

1. Se X é terminal, então $\text{FIRST}(X) = X$

2. Se $X \rightarrow \epsilon$ é uma produção, então adicione ϵ ao $\text{FIRST}(X)$

3. Se $X \rightarrow Y_1 Y_2 Y_3 \dots Y_k$ é uma produção, então a é um elemento de $\text{FIRST}(X)$ se para algum i , a está em $\text{FIRST}(Y_i)$ e $Y_1 Y_2 \dots Y_{i-1} \rightarrow \epsilon$.

Se ϵ está em $\text{FIRST}(Y_j)$ para todo $j = 1, 2, \dots, k$, então adicione ϵ ao $\text{FIRST}(X)$.

$E \rightarrow T E'$	$\text{FIRST}(E) = \{ (, \text{id} \}$
$E' \rightarrow +TE' \mid \epsilon$	$\text{FIRST}(E') = \{ +, \epsilon \}$
$T \rightarrow FT'$	$\text{FIRST}(T) = \{ (, \text{id} \}$
$T' \rightarrow *FT' \mid \epsilon$	$\text{FIRST}(T') = \{ *, \epsilon \}$
$F \rightarrow (E) \mid \text{id}$	$\text{FIRST}(F) = \{ (, \text{id} \}$

[Voltar para o índice](#)

Ex: A produção $F \rightarrow (E)$, entra na regra 3 onde Y_1 é (, Y_2 é E e Y_3 é).

$E \rightarrow BCD$ $\text{FIRST}(E) = \{ a, b, c, \epsilon \}$

$D \rightarrow aT \mid \epsilon$ $\text{FIRST}(D) = \{ a, \epsilon \}$

$B \rightarrow bX \mid \epsilon$ $\text{FIRST}(B) = \{ b, \epsilon \}$

$C \rightarrow cD \mid \epsilon$ $\text{FIRST}(C) = \{ c, \epsilon \}$

[Voltar para o índice](#)

- Para computar FOLLOW(A), aplicam-se as regras abaixo até que não se possa adicionar mais símbolos ao conjunto.
 - 1 \$ está em FOLLOW(S), sendo S o símbolo inicial e \$ o marcador de fim de cadeia de entrada
 - 2 Se existe uma produção $A \rightarrow \alpha B \beta$ então todos os terminais de FIRST(β) farão parte de FOLLOW(B)
 - 3 Se existe uma produção $A \rightarrow \alpha B$ ou $A \rightarrow \alpha B \beta$, sendo que $\beta \rightarrow \epsilon$, então todos os terminais que estiverem em FOLLOW(A) farão parte de FOLLOW(B)

[Voltar para o índice](#)

Ex:

$E \rightarrow T E'$	$\text{FOLLOW}(E) = \{ \$,) \}$
$E' \rightarrow +TE' \mid \epsilon$	$\text{FOLLOW}(E') = \{ \$,) \}$
$T \rightarrow FT'$	$\text{FOLLOW}(T) = \{ + , \$,) \}$
$T' \rightarrow *FT' \mid \epsilon$	$\text{FOLLOW}(T') = \{ + , \$,) \}$
$F \rightarrow (E) \mid \text{id}$	$\text{FOLLOW}(F) = \{ * , + , \$,) \}$

[Voltar para o índice](#)

Algoritmo para construção da tabela de Análise Tabular Preditiva

Entrada : Gramática G

Resultado : Tabela de Análise M

Método :

- 1 Para cada produção $A \rightarrow \alpha$ de G execute os passos 2 e 3
- 2 Para cada terminal a de $\text{FIRST}(\alpha)$ adicione a produção $A \rightarrow \alpha$ a $M[A, a]$
- 3 Se $\text{FIRST}(\alpha)$ inclui a palavra vazia, então adicione $A \rightarrow \alpha$ a $M[A, b]$ para cada terminal b em $\text{FOLLOW}(A)$. Se a palavra vazia pertence a $\text{FIRST}(\alpha)$ e $\$$ pertence a $\text{FOLLOW}(A)$, então adicione $A \rightarrow \alpha$ a $M[A, \$]$.

[Voltar para o índice](#)

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

[Voltar para o índice](#)

Escreva uma tabela de análise preditiva para a gramática

$S \rightarrow \text{Case } E \text{ of } L \text{ end}$

$S \rightarrow a$

$E \rightarrow n$

$L \rightarrow L ; C$

$L \rightarrow C$

$C \rightarrow k : S$

[Voltar para o índice](#)

Removendo a recursão à esquerda resulta:

$S \rightarrow \text{Case } E \text{ of } L \text{ end}$

$S \rightarrow a$

$E \rightarrow n$

$L \rightarrow CB$

$B \rightarrow ; C B \mid \epsilon$

$C \rightarrow k : S$

[Voltar para o índice](#)

	Case	of	end	a	n	k	;	\$	:
S	$S \rightarrow \text{Case } E \text{ of } L \text{ end}$			$S \rightarrow a$					
E					$E \rightarrow n$				
L						$L \rightarrow CB$			
B			$B \rightarrow \epsilon$				$B \rightarrow ; CB$		
C						$C \rightarrow k : S$			

$\text{FIRST}(S) = \{\text{Case}, a\}$ $\text{FOLLOW}(S) = \{\$, :, \text{end}\}$
 $\text{FIRST}(E) = \{n\}$ $\text{FOLLOW}(E) = \{\text{of}\}$
 $\text{FIRST}(L) = \{k\}$ $\text{FOLLOW}(L) = \{\text{end}\}$
 $\text{FIRST}(B) = \{;, \epsilon\}$ $\text{FOLLOW}(B) = \{\text{end}\}$
 $\text{FIRST}(C) = \{k\}$ $\text{FOLLOW}(C) = \{:, \text{end}\}$

Voltar para o índice

$S \rightarrow \text{if } E \text{ then } S \ S'$

$S \rightarrow a$

$E \rightarrow n$

$S' \rightarrow \text{else } S \mid \epsilon$

$\text{FIRST}(S) = \{\text{if}, a\}$

$\text{FIRST}(E) = \{n\}$

$\text{FIRST}(S') = \{\text{else}, \epsilon\}$

$\text{FOLLOW}(S) = \{\$, \text{else}\}$

$\text{FOLLOW}(E) = \{\text{then}\}$

$\text{FOLLOW}(S') = \{\$, \text{else}\}$

[Voltar para o índice](#)

	if	then	a	n	else	\$
S	$S \rightarrow \text{if } E \text{ then } S \ S'$		$S \rightarrow a$			
E				$E \rightarrow n$		
S'					$S' \rightarrow \text{else } S$ $S' \rightarrow \epsilon$	$S' \rightarrow \epsilon$

[Voltar para o índice](#)

if n then if n then a else a

S

S'S then E if

S'S then n

S'S then

S'S'S then E if

S'S'S then n

S'S'a

S'S else

S'a

€

[Voltar para o índice](#)

Gramáticas LL(1)

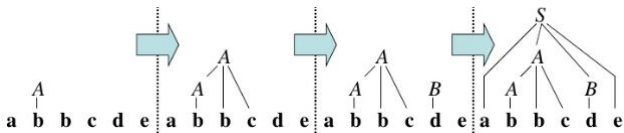
- Quando é possível implementar, para uma gramática, um analisador sintático descendente preditivo analisando, da esquerda para a direita, apenas um (1) único símbolo da entrada (lookahead), produzindo uma derivação mais à esquerda, diz-se que a gramática é LL(1).
- O primeiro 'L' diz respeito à leitura da esquerda para a direita, e o segundo 'L' diz respeito à derivação mais à esquerda.
- As gramáticas LL(1) são as mais simples de serem analisadas.
- Para ser LL(1) uma gramática não pode ser ambígua ou recursiva à esquerda e deve ser fatorada, e sua tabela de análise deve conter apenas uma produção por entrada.
- Se para decidir qual produção a utilizar na análise é necessário verificar X tokens à frente, diz-se que a gramática é LL(X).

Análise Redutiva (Bottom-up)

- A análise redutiva pode ser vista como uma tentativa de construir uma árvore de derivação **a partir das folhas para a raiz**, produzindo uma derivação mais à direita ao reverso.
- A denominação **redutiva** se refere ao processo de redução que sofre a sentença de entrada para o símbolo inicial da gramática.
- Dá-se o nome de **redução** à operação de substituição do lado direito da produção pelo não terminal correspondente (lado esquerdo).

[Voltar para o índice](#)

- Consider a



- Os analisadores redutivos em geral são implementados por autômatos de pilha cujos controles são dirigidos por tabelas de análise.
- No início da análise a fita de entrada contém a sentença de entrada seguida por um marcador \$ e a pilha contém apenas o marcador de base \$ (configuração inicial do analisador).

[Voltar para o índice](#)

- O processo de reconhecimento consiste em transferir símbolos da fita de entrada para a pilha até que se tenha no topo da pilha uma seqüência que represente um lado direito da produção.
- Quando isto ocorre, esta forma sentencial deve ser reduzida para o não terminal correspondente. O processo segue adiante com estes movimentos, empilhamento e redução, até que a sentença de entrada seja completamente lida e a pilha fique reduzida ao símbolo inicial na base da pilha.

Considere a gramática que gera listas:

$$S \rightarrow [L] \mid a$$
$$L \rightarrow L;S \mid S$$

[Voltar para o índice](#)

No reconhecimento da sentença $[a;a]$ um analisador redutivo faria os seguintes movimentos

Pilha	Entrada	Ação
\$	$[a;a] \$$	empilha $[$
$[$$	$a;a] \$$	empilha a
$[$a$	$;a] \$$	$S \rightarrow a$
$[$S$	$;a] \$$	$L \rightarrow S$
$[$L$	$;a] \$$	empilha $;$
$[$L;$	$a] \$$	empilha a
$[$L;a$	$] \$$	$S \rightarrow a$
$[$L;S$	$] \$$	$L \rightarrow L;S$
$[$L$	$] \$$	empilha $]$
$[$L]$	$\$$	$S \rightarrow [L]$
$$$$	$\$$	reconheceu!!! (pois S é o símbolo inicial)

- A sequência de reduções acima pode ser visualizada como:
 - $\rightarrow[a;a] \rightarrow[S;a] \rightarrow[L;a] \rightarrow[L;S] \rightarrow[L] \rightarrow S$
 - Que caracteriza uma derivação mais à direita ao reverso, daí o nome LR. A sentença é analisada da esquerda para a direita (Left-to-Right) produzindo uma derivação mais à direita (Right)

A seqüência de símbolos que é lado direito da produção tal que sua redução leva, eventualmente, à redução para o símbolo inicial através do reverso de uma derivação mais à direita, dá-se o nome de **handle**.

Analísadores do tipo empilha - reduz defrontam-se com dois problemas:

- Como localizar o handle na pilha
- Que redução aplicar, se existem duas ou mais produções com lados direitos iguais

Ações que podem ser tomadas por um reconhecedor empilha - reduz.

- **empilha** : coloca no topo da pilha o símbolo que está sendo lido e avança o cabeçote de leitura
- **reduz** : substitui o handle no topo da pilha pelo não terminal correspondente
- **aceita** : reconhece que a sentença de entrada foi gerada pela gramática
- **erro** : ocorre erro de sintaxe, uma subrotina de atendimento a erros deve ser chamada

Analísadores LR

Os analisadores LR são analisadores redutores eficientes que lêem a sentença em análise da esquerda para a direita (left to right) e produzem uma derivação mais à direita (rightmost derivation) ao reverso. Suas vantagens são:

- 1 São capazes de reconhecer praticamente todas as estruturas sintáticas definidas por gramáticas livres de contexto
- 2 O método de reconhecimento LR é mais geral do que o de precedência de operadores ou quaisquer outro tipo de empilha-reduz, e pode ser implementado com o mesmo grau de eficiência
- 3 Analisadores LR são capazes de descobrir erros sintáticos o mais cedo possível, na leitura da sentença analisada

A desvantagem dos métodos é que a construção da tabela de análise é complexa, exigindo ferramentas para isso.

Há basicamente três tipos de analisadores LR:

- 1 SLR (Simple LR) - mais fácil de implementar, mas abrangendo uma classe menor de gramáticas
- 2 LR Canônico - Mais poderoso, pode ser aplicado a um grande número de linguagens
- 3 LALR (Look Ahead LR) - Nível intermediário, funciona para a maioria das linguagens e pode ser implementado eficientemente.

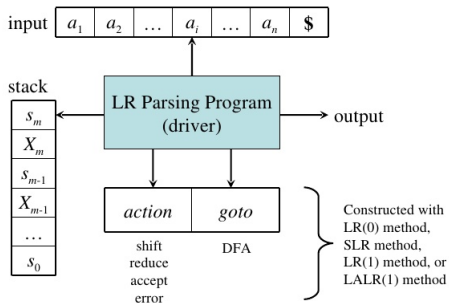
Um parser LR consiste de uma entrada, uma saída, uma pilha e uma tabela de parsing formada por duas partes, action e goto. O parser utiliza a pilha para armazenar alternadamente símbolos da gramática e estados que ajudam a definir o comportamento do parser.

A cada instante da análise o parser verifica o topo da pilha e o símbolo de entrada e isto define a ação a tomar.

- 1 Empilhar o símbolo de entrada e um estado S
- 2 Reduzir pela produção $A \rightarrow B$
- 3 Aceitar
- 4 Erro

[Voltar para o índice](#)

Model of an LR Parser



Funcionamento do Analisador

Seja E_m o estado no topo da pilha, e a , o símbolo sob o cabeçote de leitura. O analisador consulta a tabela AÇÃO $[E_m, a]$, que pode assumir um dos valores:

- empilha E_n , que causa o empilhamento de a e E_n ;
- reduz n , sendo n o número da produção $A \rightarrow \beta$, substituindo o handle β por A , desempilhando $2r$ símbolos (onde $r = |\beta|$), e empilhando E_t obtido a partir de TRANSIÇÃO $[E_{m-r}, A]$;
- aceita : o analisador suspende a execução reconhecendo a sentença;
- erro : o analisador suspende a execução identificando o erro sintático;

Exemplo : Para a gramática da expressão aritmética tem-se, abaixo, a tabela de análise LR e os passos que o analisador faz ao reconhecer a sentença $\text{id} + \text{id} * \text{id}\$$

$$1) E \rightarrow E + T$$

$$2) E \rightarrow T$$

$$3) T \rightarrow T * F$$

$$4) T \rightarrow F$$

$$5) F \rightarrow (E)$$

$$6) F \rightarrow \text{id}$$

[Voltar para o índice](#)

	id	+	*	(	)	\$	E	T	F
0	e5			e4			1	2	3
1		e6				AC			
2		r2	e7		r2	r2			
3		r4	r4		r4	r4			
4	e5			e4			8	2	3
5		r6	r6		r6	r6			
6	e5			e4				9	3
7	e5			e4					10
8		e6			e11				
9		r1	e7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

[Voltar para o índice](#)

Pilha	Entrada	Ação
0	id * id + id \$	empilhar
0 id 5	* id + id \$	$F \rightarrow id$
0 F 3	* id + id \$	$T \rightarrow F$
0 T 2	* id + id \$	empilhar
0 T 2 * 7	id + id \$	empilhar
0 T 2 * 7 id 5	+ id \$	$F \rightarrow id$
0 T 2 * 7 F 10	+ id \$	$T \rightarrow T * F$
0 T 2	+ id \$	$E \rightarrow T$
0 E 1	+ id \$	empilhar
0 E 1 + 6	id \$	empilhar
0 E 1 + 6 id 5	\$	$F \rightarrow id$
0 E 1 + 6 F 3	\$	$T \rightarrow F$
0 E 1 + 6 T 9	\$	$E \rightarrow E + T$
0 E 1	\$	Aceitar

■ E a derivação mais à direita ao reverso gerada é:

■ $E \rightarrow E + T \rightarrow E + F \rightarrow E + id \rightarrow T + id \rightarrow T * F + id \rightarrow$
 $T * id + id \rightarrow F * id + id \rightarrow id * id + id$

[Voltar para o índice](#)

Configuração inicial de um analisador LR:

$$(E_0 , \quad a_1 \ a_2 \ a_3 \ \dots \ a_n \ \$)$$

< Pilha > < —entrada— >

onde E_n refere-se ao estado n .

[Voltar para o índice](#)

Configuração antes de um movimento de empilhamento ou redução:

$(E_0 X_1 E_1 X_2 E_2 \dots X_m E_m , a_i a_{i+1} \dots a_n \$)$

Configuração após um empilhamento (AÇÃO $[E_m , a_i] = \text{empilha } E$)

$(E_0 X_1 E_1 X_2 E_2 \dots X_m E_m a_i E , a_{i+1} \dots a_n \$)$

Configuração após uma redução (AÇÃO $[E_m , a_i] = \text{reduz } A \rightarrow \beta$)

$(E_0 X_1 E_1 X_2 E_2 \dots X_{m-r} E_{m-r} A E , a_i a_{i+1} \dots a_n \$)$

sendo $\beta = X_{m-r+1} \dots X_m$, $|\beta|=r$; são desempilhados $2r$ símbolos, e é empilhado E , resultante da consulta TRANSIÇÃO $[E_{m-r}, A]$

Construção de Analisadores SLR

A Construção da Tabela de Análise para analisadores SLR baseia-se na Coleção Canônica de Conjuntos de Itens LR(0). Item LR(0), para uma gramática G , é uma produção com um ponto em alguma posição do lado direito. O ponto indica quanto da produção foi analisada até um determinado momento no processo de reconhecimento.

Exemplo: $A \rightarrow XYZ$ gera quatro itens

$A \rightarrow \bullet XYZ$

$A \rightarrow X \bullet YZ$

$A \rightarrow XY \bullet Z$

$A \rightarrow XYZ \bullet$

e

$A \rightarrow \epsilon$ gera $A \rightarrow \bullet$

A construção da coleção canônica de conjuntos de itens requer:

- 1) acrescentar à gramática a produção $S' \rightarrow S$ para indicar ao analisador quando parar e aceitar a sentença; e
- 2) a computação das funções **closure** e **goto**.

[Voltar para o índice](#)

Closure (I)

Se I é um conjunto de itens para G , então, o conjunto de itens $\text{Closure}(I)$ é construído a partir de I pelas regras:

- 1) todo item em I pertence a $\text{Closure}(I)$
- 2) Se $A \rightarrow \alpha \bullet B\beta$ está em $\text{Closure}(I)$ e $B \rightarrow \gamma$ é uma produção, então adicione $B \rightarrow \bullet\gamma$ a $\text{Closure}(I)$.

Exemplo: $E' \rightarrow E$ $I = \{E' \rightarrow \bullet E\}$

$E \rightarrow E + T$	$\text{Closure}(I) = \{$	$E' \rightarrow \bullet E$
$E \rightarrow T$		$E \rightarrow \bullet E + T$
$T \rightarrow T * F$		$E \rightarrow \bullet T$
$T \rightarrow F$		$T \rightarrow \bullet T * F$
$F \rightarrow (E)$		$T \rightarrow \bullet F$
$F \rightarrow id$		$F \rightarrow \bullet (E)$
		$F \rightarrow \bullet id \}$

Goto(I,X)

Para X , símbolo terminal ou não terminal da gramática, $\text{Goto}(I, X)$ é definido como a Closure do conjunto de todos os itens $[A \rightarrow \alpha X \bullet \beta]$, tal que $[A \rightarrow \alpha \bullet X \beta]$, pertence a I .

Exemplo: $I = \{ E' \rightarrow E \bullet, E \rightarrow E \bullet + T \}$

$$\begin{aligned} \text{Goto}(I, +) = \{ & E \rightarrow E + \bullet T \\ & T \rightarrow \bullet T * F \\ & T \rightarrow \bullet F \\ & F \rightarrow \bullet (E) \\ & F \rightarrow \bullet id \} \end{aligned}$$

Determinação da Coleção Canônica de Conjuntos de Itens LR(0)

Seja C , a coleção Canônica de conjuntos de Itens LR(0) inicialmente vazia.

Algoritmo

$C := \{ \text{Closure}(\{S' \rightarrow \bullet S\}) \}$

repita para cada I em C e X em G , tal que $\text{Goto}(I, X) \neq \emptyset$

 adicione $\text{Goto}(I, X)$ a C

até que todos os conjuntos tenham sido adicionados a C .

Exemplo :

$E' \rightarrow E$
 $E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow id$

$I_0 = \text{Closure}(\{ E' \rightarrow \bullet E \})$
 $I_0 = \{$
 $E' \rightarrow \bullet E$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet (E)$
 $F \rightarrow \bullet id \}$

[Voltar para o índice](#)

$\text{Goto}(0, E) = \text{Closure} (E' \rightarrow E \bullet , E \rightarrow E \bullet + T) = \{ E' \rightarrow E \bullet , E \rightarrow E \bullet + T \}$

Como este item ainda não está em C, faz-se

$$I_1 = \{ E' \rightarrow E \bullet , E \rightarrow E \bullet + T \}$$

$\text{Goto}(0, T) = \text{Closure} (E \rightarrow T \bullet , T \rightarrow T \bullet * F) = \{ E \rightarrow T \bullet , T \rightarrow T \bullet * F \}$

Como este item ainda não está em C, faz-se $I_2 =$

$$\{ E \rightarrow T \bullet , T \rightarrow T \bullet * F \}$$

$\text{Goto}(0, F) = \text{Closure} (T \rightarrow F \bullet) = \{ T \rightarrow F \bullet \}$

Como este item ainda não está em C, faz-se $I_3 = \{ T \rightarrow F \bullet \}$

$\text{Goto}(0, () = \text{Closure} (F \rightarrow (\bullet E)) = \{ F \rightarrow (\bullet E)$

$E \rightarrow \bullet E + T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T * F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet id \}$

Como este item ainda não está em C , faz-se $I_4 = \{ F \rightarrow (\bullet E)$

$E \rightarrow \bullet E + T$

$E \rightarrow \bullet T$

$T \rightarrow \bullet T * F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet id \}$

$\text{Goto}(0, id) = \text{Closure} (F \rightarrow id \bullet) = \{ F \rightarrow id \bullet \}$

Como este item ainda não está em C, faz-se $I_5 = \{ F \rightarrow id \bullet \}$

$\text{Goto} (1, +) = \text{Closure} (E \rightarrow E + \bullet T) = \{ E \rightarrow E + \bullet T$

$T \rightarrow \bullet T * F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet id \}$

Como este item ainda não está em C, faz-se $I_6 = \{ E \rightarrow E + \bullet T$

$T \rightarrow \bullet T * F$

$T \rightarrow \bullet F$

$F \rightarrow \bullet (E)$

$F \rightarrow \bullet id \}$

Goto (2,*) = Closure ($T \rightarrow T * \bullet F$) = $\{ T \rightarrow T * \bullet F$
 $F \rightarrow \bullet(E)$
 $F \rightarrow \bullet id \}$

Como este item ainda não está em C, faz-se $I_7 = \{ T \rightarrow T * \bullet F$
 $F \rightarrow \bullet(E)$
 $F \rightarrow \bullet id \}$

Goto (4,E) = Closure ($F \rightarrow (E\bullet)$, $E \rightarrow E \bullet + T$) = $\{ F \rightarrow (E\bullet)$,
 $E \rightarrow E \bullet + T \}$

Como este item ainda não está em C, faz-se $I_8 = \{ F \rightarrow (E\bullet)$,
 $E \rightarrow E \bullet + T \}$

[Voltar para o índice](#)

$\text{Goto}(4, T) = \text{Closure}(E \rightarrow T \bullet, T \rightarrow T \bullet * F) = \{ E \rightarrow T \bullet, T \rightarrow T \bullet * F \}$

Como este item já existe em C (é igual a l_2), $\text{Goto}(4, T) = l_2$

$\text{Goto}(4, F) = \text{Closure}(T \rightarrow F \bullet) = \{ T \rightarrow F \bullet \}$

Como este item já existe em C (é igual a l_3), $\text{Goto}(4, F) = l_3$

Da mesma forma, $\text{Goto}(4, ()) = l_4$ e $\text{Goto}(4, \text{id}) = l_5$

$\text{Goto}(6, T) = \text{Closure}(E \rightarrow E + T \bullet, T \rightarrow T \bullet * F) = \{ E \rightarrow E + T \bullet, T \rightarrow T \bullet * F \}$

Como este item ainda não está em C , faz-se $l_9 = \{ E \rightarrow E + T \bullet, T \rightarrow T \bullet * F \}$

$\text{Goto}(6, F) = \text{Closure}(T \rightarrow F\bullet) = \{ T \rightarrow F\bullet \}$

Como este item já existe em C (é igual a l_3), $\text{Goto}(6, F) = l_3$

Da mesma forma, $\text{Goto}(6, ()) = l_4$ e $\text{Goto}(6, \text{id}) = l_5$

$\text{Goto}(7, F) = \text{Closure}(T \rightarrow T * F\bullet) = \{ T \rightarrow T * F\bullet \}$

Como este item ainda não está em C , faz-se $l_{10} = \{ T \rightarrow T * F\bullet \}$

[Voltar para o índice](#)

$\text{Goto}(7, ()) = I_4$ e $\text{Goto}(7, \text{id}) = I_5$

$\text{Goto}(8, ()) = \text{Closure}(F \rightarrow (E)\bullet) = \{ F \rightarrow (E)\bullet \}$

Como este item ainda não está em C , faz-se $I_{11} = \{ F \rightarrow (E)\bullet \}$

$\text{Goto}(8, +) = I_6$ e $\text{Goto}(9, *) = I_7$

[Voltar para o índice](#)

Coleção Canônica de conjuntos de itens resultante

$l_0 = \{$
 $E' \rightarrow \bullet E$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet (E)$
 $F \rightarrow \bullet id \}$

$l_1 = \{$
 $E' \rightarrow E \bullet$
 $E \rightarrow E \bullet + T \}$

$l_2 = \{$
 $E \rightarrow T \bullet$
 $T \rightarrow T \bullet * F \}$

$l_3 = \{$
 $T \rightarrow F \bullet \}$

$l_4 = \{$
 $F \rightarrow (\bullet E)$
 $E \rightarrow \bullet E + T$
 $E \rightarrow \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet (E)$
 $F \rightarrow \bullet id \}$

$l_5 = \{$
 $F \rightarrow id \bullet \}$

$l_6 = \{$
 $E \rightarrow E + \bullet T$
 $T \rightarrow \bullet T * F$
 $T \rightarrow \bullet F$
 $F \rightarrow \bullet (E)$
 $F \rightarrow \bullet id \}$

$l_7 = \{$
 $T \rightarrow T * \bullet F$
 $F \rightarrow \bullet (E)$
 $F \rightarrow \bullet id \}$

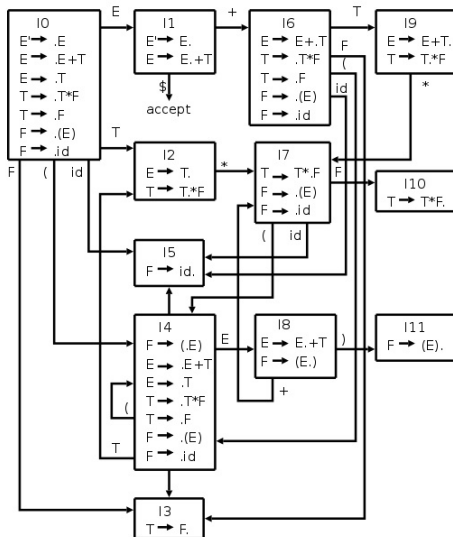
$l_8 = \{$
 $F \rightarrow (E \bullet)$
 $E \rightarrow E \bullet + T \}$

$l_9 = \{$
 $E \rightarrow E + T \bullet$
 $T \rightarrow T \bullet * F \}$

$l_{10} = \{$
 $T \rightarrow T * F \bullet \}$

$l_{11} = \{$
 $F \rightarrow (E) \bullet \}$

Autômato correspondente



Exercício resolvido : Construa o conjunto canônico de itens LR(0) da gramática a seguir:

$$S' \rightarrow S$$

$$S \rightarrow a \mid (L)$$

$$L \rightarrow S, L \mid S$$

[Voltar para o índice](#)

$$I_0 = \text{Closure}(\{S' \rightarrow \bullet S\}) = \{ \begin{array}{l} S' \rightarrow \bullet S \\ S \rightarrow \bullet a \\ S \rightarrow \bullet(L) \end{array} \}$$

$$\text{Goto}(I_0, S) = \text{Closure}(\{S' \rightarrow S\bullet\}) = \{S' \rightarrow S\bullet\}$$

Como este item ainda não está em C, faz-se $I_1 = \{S' \rightarrow S\bullet\}$

$$\text{Goto}(I_0, a) = \text{Closure}(\{S \rightarrow a\bullet\}) = \{S \rightarrow a\bullet\}$$

Como este item ainda não está em C, faz-se $I_2 = \{S \rightarrow a\bullet\}$

[Voltar para o índice](#)

$$\begin{aligned} \text{Goto}(l_0, "(") &= \text{Closure}(\{S \rightarrow (\bullet L)\}) = \\ &\{S \rightarrow (\bullet L) \\ &L \rightarrow \bullet S, L \\ &L \rightarrow \bullet S \\ &S \rightarrow \bullet a \\ &S \rightarrow \bullet(L)\} \end{aligned}$$

Como este item ainda não está em C, faz-se

$$\begin{aligned} l_3 = \{ &S \rightarrow (\bullet L) \\ &L \rightarrow \bullet S, L \\ &L \rightarrow \bullet S \\ &S \rightarrow \bullet a \\ &S \rightarrow \bullet(L)\} \end{aligned}$$

Como l_1 e l_2 só tem itens com o ponto no final, seus Goto's são vazios.

$\text{Goto}(I_3, L) = \text{Closure}(\{S \rightarrow (L\bullet)\}) = \{S \rightarrow (L\bullet)\}$

Como este item ainda não está em C, faz-se $I_4 = \{S \rightarrow (L\bullet)\}$

$\text{Goto}(I_3, S) = \text{Closure}(\{L \rightarrow S\bullet, L,$

$L \rightarrow S\bullet\}) =$

$\{L \rightarrow S\bullet, L,$

$L \rightarrow S\bullet\}$

Como este item ainda não está em C, faz-se

$I_5 = \{L \rightarrow S\bullet, L$

$L \rightarrow S\bullet\}$

$\text{Goto}(I_3, a) = \text{Closure}(\{S \rightarrow a\bullet\}) = I_2$

[Voltar para o índice](#)

$\text{Goto}(I_3, "(") = \text{Closure}(\{S \rightarrow (\bullet L)\}) = I_3$

$\text{Goto}(I_4, ")") = \text{Closure}(\{S \rightarrow (L)\bullet\}) = \{S \rightarrow (L)\bullet\}$

Como este item ainda não está em C, faz-se $I_6 = \{S \rightarrow (L)\bullet\}$

$\text{Goto}(I_5, ", ") = \text{Closure}(\{L \rightarrow S, \bullet L\}) =$

$\{L \rightarrow S, \bullet L\}$

$L \rightarrow \bullet S, L$

$L \rightarrow \bullet S$

$S \rightarrow \bullet a$

$S \rightarrow \bullet(L)\}$

Como este item ainda não está em C, faz-se

$I_7 = \{L \rightarrow S, \bullet L\}$

$L \rightarrow \bullet S, L$

$L \rightarrow \bullet S$

$S \rightarrow \bullet a$

$S \rightarrow \bullet(L)\}$

Como I_5 só tem itens com o ponto no final, seus Goto's são vazios.

$$\text{Goto}(I_6, L) = \text{Closure}(\{L \rightarrow S, L\bullet\}) = \{L \rightarrow S, L\bullet\}$$

Como este item ainda não está em C, faz-se $I_8 = \{L \rightarrow S, L\bullet\}$

$$\text{Goto}(I_6, S) = \text{Closure}(\{L \rightarrow S\bullet, L, L \rightarrow S\bullet\}) = I_4$$

$$\text{Goto}(I_6, a) = \text{Closure}(\{S \rightarrow a\bullet\}) = I_6$$

$$\text{Goto}(I_6, "(") = \text{Closure}(\{S \rightarrow (\bullet L)\}) = I_3$$

[Voltar para o índice](#)

Coleção Canônica de conjuntos de itens resultante

$$I_0 = \{ \begin{array}{l} S' \rightarrow \bullet S \\ S \rightarrow \bullet a \\ S \rightarrow \bullet(L) \end{array} \}$$

$$I_1 = \{ S' \rightarrow S\bullet \}$$

$$I_2 = \{ S \rightarrow a\bullet \}$$

$$I_3 = \{ \begin{array}{l} S \rightarrow (\bullet L) \\ L \rightarrow \bullet S, L \\ L \rightarrow \bullet S \\ S \rightarrow \bullet a \\ S \rightarrow \bullet(L) \end{array} \}$$

$$I_4 = \{ S \rightarrow (L\bullet) \}$$

$$I_5 = \{ \begin{array}{l} L \rightarrow S\bullet, L \\ L \rightarrow S\bullet \end{array} \}$$

$$I_6 = \{ S \rightarrow (L)\bullet \}$$

$$I_7 = \{ \begin{array}{l} L \rightarrow S, \bullet L \\ L \rightarrow \bullet S, L \\ L \rightarrow \bullet S \\ S \rightarrow \bullet a \\ S \rightarrow \bullet(L) \end{array} \}$$

$$I_8 = \{ L \rightarrow S, L\bullet \}$$

[Voltar para o índice](#)

Construção da Tabela de Análise SLR

- Dada uma gramática G , aumenta-se G com a produção $S' \rightarrow S$ obtendo-se G' . A partir de G' determina-se C , coleção canônica de conjuntos de itens LR(0). Constrói-se as tabelas Ação e Transição conforme o algoritmo abaixo.
 - Algoritmo: Construção da tabela SLR para G
 - Entrada : C para G' .
 - Resultado : Se possível, a tabela de análise SLR para G' .
 - Método: Seja $C = I_0, I_1, \dots, I_n$. Os estados do analisador serão $0, 1, \dots, n$. As ações do analisador para o estado i são determinadas como se segue:
 - 1) Se $A \rightarrow \alpha \bullet a\beta$ está em I_i e $\text{Goto}(I_i, a) = I_j$ então faça $\text{Ação}[i, a] = \text{empilha } j$;
 - 2) Se $A \rightarrow \alpha \bullet$ está em I_i então $\text{Ação}[i, a] = \text{reduz } n$, sendo n o número da produção $A \rightarrow \alpha$ para todo a em $\text{FOLLOW}(A)$.
 - 3) Se $S' \rightarrow S \bullet$ está em I_i , então faça $\text{Ação}[i, \$] = \text{aceita}$;

Se ações conflitantes são geradas pelas regras acima, então a gramática não é SLR(1). As transições para o estado i são construídas usando a regra:

4) Se $\text{Goto}(I_i, A) = I_j$, então $\text{TRANSIÇÃO}(I_i, A) = j$;

As entradas não definidas pelas regras acima constituem erro;

O estado inicial do analisador é aquele construído a partir do conjunto de itens que contem $S' \rightarrow \bullet S$.

[Voltar para o índice](#)

- Gramática SLR(1) é aquela a partir da qual, pode-se construir uma tabela de análise SLR.
- Toda gramática SLR(1) é não ambígua.
- Existem gramáticas não ambíguas que não são SLR(1).
- Ex: Construa a tabela SLR para a gramática a seguir:
 $S \rightarrow L = R$
 $S \rightarrow R$
 $L \rightarrow *R$
 $L \rightarrow \mathbf{id}$
 $R \rightarrow L$
- Quais as ações para o estado 2 da tabela de análise?

Coleção Canônica de conjuntos de itens resultante

$$\begin{aligned}I_0 : \quad & S' \rightarrow \bullet S \\ & S \rightarrow \bullet L = R \\ & S \rightarrow \bullet R \\ & L \rightarrow \bullet * R \\ & L \rightarrow \bullet \mathbf{id} \\ & R \rightarrow \bullet L\end{aligned}$$

$$I_1 : \quad S' \rightarrow S \bullet$$

$$\begin{aligned}I_2 : \quad & S \rightarrow L \bullet = R \\ & R \rightarrow L \bullet\end{aligned}$$

$$I_3 : \quad S \rightarrow R \bullet$$

$$\begin{aligned}I_4 : \quad & L \rightarrow * \bullet R \\ & R \rightarrow \bullet L \\ & L \rightarrow \bullet * R \\ & L \rightarrow \bullet \mathbf{id}\end{aligned}$$

$$I_5 : \quad L \rightarrow \mathbf{id} \bullet$$

$$\begin{aligned}I_6 : \quad & S \rightarrow L = \bullet R \\ & R \rightarrow \bullet L \\ & L \rightarrow \bullet * R \\ & L \rightarrow \bullet \mathbf{id}\end{aligned}$$

$$I_7 : \quad L \rightarrow * R \bullet$$

$$I_8 : \quad R \rightarrow L \bullet$$

$$I_9 : \quad S \rightarrow L = R \bullet$$

Parsers LR(0)

- Um algoritmo ainda mais simplificado de construção da tabela de parsing é o chamado parsing LR(0)
- A única diferença em relação ao parser SLR é que nos estados em que ocorrerá uma redução (que possuem itens no formato $A \rightarrow \alpha \bullet$) a redução ocorrerá para qualquer próximo símbolo e não apenas para aqueles que pertencem a $\text{Follow}(A)$.
- Isso simplifica a construção da tabela, mas aumenta o potencial de ocorrência de conflitos.

[Voltar para o índice](#)

- Todo estado com um item de redução e algum outro item causa conflito LR(0)
- A técnica LR(0) é bem fraca, mas ainda assim existem gramáticas que ela consegue analisar mas que as técnicas de análise descendente não:
 - $S \rightarrow E$
 - $E \rightarrow E + T$
 - $E \rightarrow T$
 - $T \rightarrow - T$
 - $T \rightarrow \text{num}$
 - $T \rightarrow (E)$
- Vamos construir o autômato de itens LR(0) dessa gramática e usá-lo para analisar $- (\text{num} + \text{num}) + \text{num}$

Gramáticas de expressões ambíguas

- Vamos agora construir a coleção canônica de conjuntos de itens e a tabela de análise SLR para a gramática ambígua abaixo, em que a ambiguidade vem da falta de definição da associatividade da soma:
 - 0) $E' \rightarrow E$
 - 1) $E \rightarrow E + E$
 - 2) $E \rightarrow id$
- e analisar a sentença $id+id+id$

$$I_0 : \begin{array}{l} E' \rightarrow \bullet E \\ E \rightarrow \bullet E + E \\ E \rightarrow \bullet id \end{array}$$

$$I_1 : \begin{array}{l} E' \rightarrow E \bullet \\ E \rightarrow E \bullet + E \end{array}$$

$$I_2 : \begin{array}{l} E \rightarrow id \bullet \end{array}$$

$$I_3 : \begin{array}{l} E \rightarrow E + \bullet E \\ E \rightarrow \bullet E + E \\ E \rightarrow \bullet id \end{array}$$

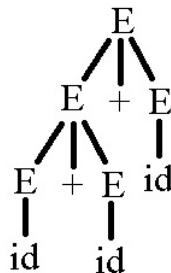
$$I_4 : \begin{array}{l} E \rightarrow E + E \bullet \\ E \rightarrow E \bullet + E \end{array}$$

	+	id	\$	E
0		s2		1
1	s3		ac.	
2	r2		r2	
3		s2		4
4	r1 s3		r1	

[Voltar para o índice](#)

■ Reconhecimento de $\text{id} + \text{id} + \text{id}$ escolhendo $r1$

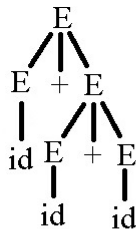
\$0	$\text{id} + \text{id} + \text{id} \$$	$s2$
\$0 $\text{id} 2$	$+ \text{id} + \text{id} \$$	$r2$
\$0 $E 1$	$+ \text{id} + \text{id} \$$	$s3$
\$0 $E 1 + 3$	$\text{id} + \text{id} \$$	$s2$
\$0 $E 1 + 3 \text{id} 2$	$+ \text{id} \$$	$r2$
\$0 $E 1 + 3 E 4$	$+ \text{id} \$$	$r1??$
\$0 $E 1$	$+ \text{id} \$$	$s3$
\$0 $E 1 + 3$	$\text{id} \$$	$s2$
\$0 $E 1 + 3 \text{id} 2$	$\$$	$r2$
\$0 $E 1 + 3 E 4$	$\$$	$r1$
\$0 $E 1$	$\$$	$ac.$



■ Reconhecimento de $id + id + id$ escolhendo s_3

$\$0$
 $\$0 \ id \ 2$
 $\$0 \ E \ 1$
 $\$0 \ E \ 1 \ + \ 3$
 $\$0 \ E \ 1 \ + \ 3 \ id \ 2$
 $\$0 \ E \ 1 \ + \ 3 \ E \ 4$
 $\$0 \ E \ 1 \ + \ 3 \ E \ 4 \ + \ 3$
 $\$0 \ E \ 1 \ + \ 3 \ E \ 4 \ + \ 3 \ id \ 2$
 $\$0 \ E \ 1 \ + \ 3 \ E \ 4 \ + \ 3 \ E \ 4$
 $\$0 \ E \ 1 \ + \ 3 \ E \ 4$
 $\$0 \ E \ 1$

$id \ + \ id \ + \ id$
 $\ + \ id \ + \ id$
 $\ + \ id \ + \ id$
 $\ \ \ id \ + \ id$
 $\ \ \ \ + \ id$
 $\ \ \ \ + \ id$
 $\ \ \ \ id$



- O exemplo anterior mostra que ao escolher a ação r2 o operador de soma fica com associatividade à esquerda, e ao escolher s3 ele fica com associatividade à direita.
- E a gramática a seguir?
 - 0) $E' \rightarrow E$
 - 1) $E \rightarrow E + E$
 - 2) $E \rightarrow E * E$
 - 3) $E \rightarrow id$
- Quais os conflitos gerados na construção da tabela?
- Como devem ser resolvidos para que os operadores de multiplicação e soma sejam associativos à esquerda, e a multiplicação tenha prioridade maior que a soma?

Parsers LR(1)

- Algumas gramáticas como a anterior, mesmo não sendo ambíguas, não permitem a geração de um analisador SLR.
- Isso ocorre por que não é possível identificar, sem olhar o próximo símbolo, qual a ação a ser tomada.
- Analisadores LR(1) aumentam a classe de gramáticas passíveis de serem analisadas acrescentando, a cada item, um terminal que auxilia na identificação da ação a ser tomada.
 - Esses itens são chamados itens **LR(1)**
 - Esse terminal é chamado de **lookahead**
- Nesses parsers, faz-se a redução somente se o próximo símbolo da sentença é igual ao lookahead do item.
- A construção da Coleção Canônica de Conjuntos de Itens LR(1) utiliza versões ligeiramente diferentes de Closure e Goto

Closure (I)

repetir

para

cada item $[A \rightarrow \alpha \bullet B\beta, a]$ em I
cada produção $B \rightarrow \gamma$ em G' ,
e cada terminal b em $\text{FIRST}(\beta a)$
tal que $[B \rightarrow \bullet \gamma, b]$ não está em I faça
incluir $[B \rightarrow \bullet \gamma, b]$ em I

até que não possam ser adicionados mais itens a I ;

retornar I

[Voltar para o índice](#)

Goto(I,X)

- seja J o conjunto de itens $[A \rightarrow \alpha X \bullet \beta, a]$ tais que $[A \rightarrow \alpha \bullet X \beta, a]$ esteja em I;
- retornar Closure (J)

[Voltar para o índice](#)

Construção da Coleção Canônica de Itens LR(1)

$C := \{\text{closure}(\{[S' \rightarrow \bullet S, \$]\})\}$

Repetir

para cada conjunto de itens I em C e cada símbolo gramatical X tal que $\text{goto}(I, X)$ não é vazio e não está em C **faça**
 incluir $\text{goto}(I, X)$ a C

até que não possam ser adicionados itens a C

[Voltar para o índice](#)

- Ex: Construa o conjunto de itens LR(1) para a gramática a seguir:

$$S' \rightarrow S$$

$$S \rightarrow CC$$

$$C \rightarrow cC \mid d$$

[Voltar para o índice](#)

Coleção Canônica de conjuntos de itens resultante

$l_0 : S' \rightarrow \bullet S, \$$

$S \rightarrow \bullet CC, \$$

$C \rightarrow \bullet cC, c$

$C \rightarrow \bullet cC, d$

$C \rightarrow \bullet d, c$

$C \rightarrow \bullet d, d$

$l_1 : S' \rightarrow S \bullet, \$$

$l_2 : S \rightarrow C \bullet C, \$$

$C \rightarrow \bullet cC, \$$

$C \rightarrow \bullet d, \$$

$l_3 : C \rightarrow c \bullet C, c$

$C \rightarrow c \bullet C, d$

$C \rightarrow \bullet cC, c$

$C \rightarrow \bullet cC, d$

$C \rightarrow \bullet d, c$

$C \rightarrow \bullet d, d$

$l_4 : C \rightarrow d \bullet, c$

$C \rightarrow d \bullet, d$

$l_5 : S \rightarrow CC \bullet, \$$

$l_6 : C \rightarrow c \bullet C, \$$

$C \rightarrow \bullet cC, \$$

$C \rightarrow \bullet d, \$$

$l_7 : C \rightarrow d \bullet, \$$

$l_8 : C \rightarrow cC \bullet, c$

$C \rightarrow cC \bullet, d$

$l_9 : C \rightarrow cC \bullet, \$$

Construção da Tabela Sintática LR Canônica

- Dada uma gramática G , aumenta-se G com a produção $S' \rightarrow S$ obtendo-se G' . A partir de G' determina-se C , conjunto canônico de itens LR(1). Constrói-se as tabelas Ação e Transição conforme o algoritmo abaixo.
- Algoritmo: Construção da tabela LR para G
- Entrada : C para G' .
- Resultado : Se possível, a tabela de análise LR para G' .
- Método: Seja $C = I_0, I_1, \dots, I_n$. Os estados do analisador serão $0, 1, \dots, n$. As ações do analisador para o estado i são determinadas como se segue:

- 1 Se $[A \rightarrow \alpha \bullet a\beta, b]$ está em I_i e $\text{Goto}(I_i, a) = I_j$ então faça $\text{Ação}[i, a] = \text{empilha } j$;
 - 2 Se $[A \rightarrow \alpha \bullet, a]$ está em I_i , $A \neq S'$, então $\text{Ação}[i, a] = \text{reduz } n$, sendo n o número da produção $A \rightarrow \alpha$.
 - 3 Se $[S' \rightarrow S \bullet, \$]$ está em I_i , então faça $\text{Ação}[i, \$] = \text{aceita}$;
- Se ações conflitantes são geradas pelas regras acima, então a gramática não é LR(1). As transições para o estado i são construídas usando a regra:
 - 4) Se $\text{Goto}(I_i, A) = I_j$, então $\text{TRANSIÇÃO}(I_i, A) = j$;
 - As entradas não definidas pelas regras acima constituem erro;
 - O estado inicial do analisador é aquele construído a partir do conjunto de itens que contem $[S' \rightarrow \bullet S, \$]$.

1 $S' \rightarrow S$

2 $S \rightarrow CC$

3 $C \rightarrow cC$

4 $C \rightarrow d$

Como seria o reconhecimento
da sentença "ccdcd\$"?

	c	d	\$	S'	S	C
0	e3	e4			1	2
1			ac.			
2	e6	e7				5
3	e3	e4				8
4	r4	r4				
5			r2			
6	e6	e7				9
7			r4			
8	r3	r3				
9			r3			

Construção das tabelas de parsing LALR

- Para o conjunto de itens do LALR vários conjuntos de itens LR(1) são agrupados, obtendo aproximadamente o conjunto de itens LR(0) usado no SLR.
- Itens LR(1) tem dois componentes, o primeiro, chamado o núcleo, que é uma produção com um ponto e um terminal.
- Para o LALR agrupam-se todos os conjuntos de itens que tem os mesmos núcleos combinando os seus segundos componentes (assim permitindo reduções quando qualquer um dos terminais é o próximo símbolo).
- Assim obtemos o mesmo número de estados (conjuntos de itens) que no SLR, uma vez que apenas os núcleos distinguirão os conjuntos de itens.

- Ao contrário do SLR, reduções ocorrem somente para certos símbolos de entrada. LR(1) fornece um refinamento maior; é possível que o LALR apresente conflitos reduce-reduce mesmo quando o conjunto de itens LR(1) no qual ele é baseado seja livre de conflitos.
- Apesar de esses conflitos serem possíveis, são raros e a redução de tamanho de LR(1) para LALR é muito grande. LALR é o método mais usado para analisadores empilha-reduz.
- Para o conjunto de itens LR(1) do exemplo anterior, quais conjuntos de itens podem ser combinados?

Coleção Canônica de conjuntos de itens LR(1) (antes de agrupar itens)

$$\begin{aligned}I_0 : \quad & S' \rightarrow \bullet S, \$ \\ & S \rightarrow \bullet CC, \$ \\ & C \rightarrow \bullet cC, cd \\ & C \rightarrow \bullet d, cd\end{aligned}$$

$$I_1 : \quad S' \rightarrow S \bullet, \$$$

$$\begin{aligned}I_2 : \quad & S \rightarrow C \bullet C, \$ \\ & C \rightarrow \bullet cC, \$ \\ & C \rightarrow \bullet d, \$\end{aligned}$$

$$\begin{aligned}I_3 : \quad & C \rightarrow c \bullet C, cd \\ & C \rightarrow \bullet cC, cd \\ & C \rightarrow \bullet d, cd\end{aligned}$$

$$I_4 : \quad C \rightarrow d \bullet, cd$$

$$I_5 : \quad S \rightarrow CC \bullet, \$$$

$$\begin{aligned}I_6 : \quad & C \rightarrow c \bullet C, \$ \\ & C \rightarrow \bullet cC, \$ \\ & C \rightarrow \bullet d, \$\end{aligned}$$

$$I_7 : \quad C \rightarrow d \bullet, \$$$

$$I_8 : \quad C \rightarrow cC \bullet, cd$$

$$I_9 : \quad C \rightarrow cC \bullet, \$$$

Coleção Canônica de conjuntos de itens LALR(1) resultante

$I_0 :$ $S' \rightarrow \bullet S, \$$
 $S \rightarrow \bullet CC, \$$
 $C \rightarrow \bullet cC, cd$
 $C \rightarrow \bullet d, cd$

$I_1 :$ $S' \rightarrow S\bullet, \$$

$I_2 :$ $S \rightarrow C\bullet C, \$$
 $C \rightarrow \bullet cC, \$$
 $C \rightarrow \bullet d, \$$

$I_3 :$ $C \rightarrow c\bullet C, cd\$$
 $C \rightarrow \bullet cC, cd\$$
 $C \rightarrow \bullet d, cd\$$

$I_4 :$ $C \rightarrow d\bullet, cd\$$

$I_5 :$ $S \rightarrow CC\bullet, \$$

$I_6 :$ $C \rightarrow cC\bullet, cd\$$

[Voltar para o índice](#)

Construção da Tabela Sintática LALR

- 1 Construir $C = \{I_0, I_1, \dots, I_n\}$, a coleção de conjuntos de itens LR(1).
- 2 Para cada núcleo presente entre os conjuntos de itens LR(1), encontrar todos os conjuntos que tenham o mesmo núcleo e substituí-los por sua união.
- 3 Seja $C' = \{J_0, J_1, \dots, J_n\}$, a coleção de conjuntos de itens LR(1) resultante. As ações sintáticas para o estado i são construídas a partir de J_i da mesma maneira que no algoritmo da construção da tabela LR canônica. Se houver algum conflito de ação sintática, o algoritmo falha em produzir um analisador sintático e a gramática não é considerada LALR(1).

- 4) A tabela de transição é construída como se segue. Se J for a união de um ou mais conjuntos de itens LR(1), isto é, $J = I_1 \cup I_2 \cup \dots \cup I_k$, então os núcleos de $\text{Goto}(I_1, X)$, $\text{Goto}(I_2, X)$, ..., $\text{Goto}(I_k, X)$ são os mesmos, uma vez que $I_1, I_2, \dots, I_k$ possuem todos os mesmos núcleos. Seja K a união de todos os conjuntos de itens que tenham o mesmo núcleo que $\text{desvio}(I_1, X)$. Então $\text{desvio}(J, X) = K$.
- A tabela produzida por esse algoritmo é chamada de tabela sintática LALR para G . Se não existirem conflitos, a gramática é dita LALR(1)

$$1 \quad S' \rightarrow S$$

$$2 \quad S \rightarrow CC$$

$$3 \quad C \rightarrow cC$$

$$4 \quad C \rightarrow d$$

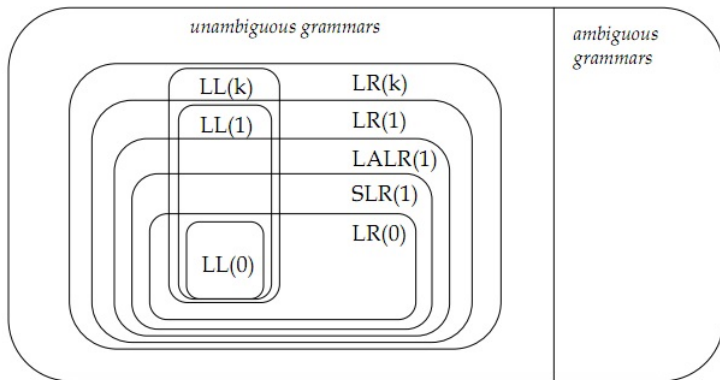
Como seria o reconhecimento da sentença "ccdc d\$"?

	c	d	\$	S'	S	C
0	e3	e4			1	2
1			ac.			
2	e3	e4				5
3	e3	e4				6
4	r4	r4	r4			
5			r2			
6	r3	r3	r3			

Voltar para o índice

LL(1) versus LR(k)

A picture is worth a thousand words:



Parsing Simulator

- Parsing Simulator é um software que gera a tabela de parsing a partir de uma gramática.
- Ele gera tabelas de parsing LL1, SLR, LR e LALR.

[Voltar para o índice](#)



ParsingEmu



Grammar:

```

I->E
E->E+T
E->T
T->T*F
T->F
F->(E)
F->id
          
```

Follow:

```

Follow(I): $
Follow(E): $+
Follow(T): $+)*
Follow(F): $+)*
          
```

Note: 'ε' = epsilon
For use SLR, LR and LALR must be inserted the extend grammar

Example:
Grammar2

Word:
i+(i*i)*i

First

LL1 Table

SLR Table

LR Table

LALR Table

Follow

LL1 Evaluation

SLR Evaluation

LR Evaluation

LALR Evaluation

Exit

Info

LR0 Items

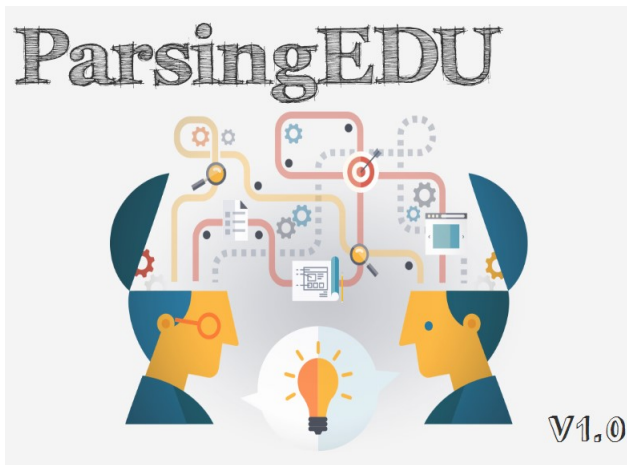
LR1 Items

LALR Items

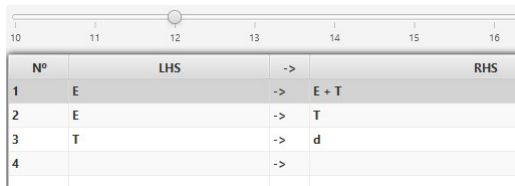
	(	)	*	+	d	i	\$	I	E	T	F
0	s4					s5			1	2	3
1				s6			a				
2		r2	s7	r2			r2				
3		r4	r4	r4			r4				
4	s4					s5			8	2	3
5					s9						

- O Parsing Edu é um programa desenvolvido por Diogo Manica como TCC em 2018.
- Ele é levemente baseado no ParsingEmu, mas com algumas melhoras na interface e funcionalidades a mais.
- Ele oferece diversos recursos como:
 - Auxílio na geração de conjuntos de itens LR(0) e LR(1) e tabelas de parsing LR.
 - Remoção de recursão à esquerda e fatoração (ainda não 100% confiável) de gramáticas)
 - Geração de tabelas de análise tabular preditiva
 - Geração de código de analisadores descendentes recursivos preditivos e com retrocesso

- No meu computador deve ser chamado de um prompt de comando "java -jar ParsingEdu.jar"(deve ser alguma incompatibilidade da versão do Java com o Windows 10)



- Tudo começa definindo a gramática.
- Os símbolos da gramática (terminais e não-terminais) podem ter vários símbolos (ao contrário do ParsingEmu). Terminais começam por minúsculas, não-terminais começam por maiúsculas.
- Para cada lado direito e lado esquerdo de produção deve ser teclado enter para efetuar a alteração



The screenshot shows a software interface for defining a grammar. At the top, there is a horizontal slider with a knob positioned at 12, and a scale from 10 to 16. Below the slider is a table with four columns: 'Nº', 'LHS', '->', and 'RHS'. The table contains four rows of data, representing grammar productions.

Nº	LHS	->	RHS
1	E	->	E + T
2	E	->	T
3	T	->	d
4		->	

- Para remover a recursão à esquerda:
- A gramática resultante aparecerá na janela à direita

o \o.

 **Gramática Modificada** ⓘ

☒ Remove Recursão ⓘ
☐ Fatorar ⓘ

18 10 11 12 13 14 15 16 17 18

Nº	LHS	->	RHS
1	E	->	T E#
2	E#	->	+ T E#
3	E#	->	λ
4	T	->	d

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
// DEFINE TOKENS
#define TK1 1 // +
#define TK2 2 // d
// Variáveis Globais <o>
int tk;
int tklinha = 0;
int tkcoluna = 0;
// <***** INICIO DO ANALISADOR LÉXICO *****>
// Implemente aqui seu analisador Léxico
// <***** FIM DO ANALISADOR LÉXICO *****>
// Protótipo das Funções do Analisador Sintático
int E();
int E1Hash();
int T();
// <***** INICIO DO ANALISADOR SINTÁTICO
// DESCENDENTE RECURSIVO SEM RETROCESSO *****>

```

- A geração de código pode definir os tokens automaticamente, atribuindo uma numeração sequencial e o nome TKn para cada terminal encontrado, ou a lista de tokens pode ser importada e editada.
- O arquivo de tokens é um arquivo texto simples, em que cada linha contém o nome do token e o terminal correspondente na gramática. Para a gramática acima:

TKd d

TKSoma +

- Ao gerar o código, é atribuída uma numeração sequencial aos tokens

- Para editar a lista de tokens, selecionar "Definir meus tokens" no menu superior



- Para cada token inserir o seu nome e o terminal correspondente na gramática

ParsingEDU - Definição de Tokens

Definição Lista de Tokens

Importar Lista para Edição **Salvar Lista**

Nome do Token

Token

Inserir **Deletar**

NOME TOKEN	TOKEN
TKSoma	+

Uma micro-gramática de C

- $\text{Prog} \rightarrow \text{ListaDec}$
- $\text{ListaDec} \rightarrow \text{Dec RListaDec}$
- $\text{RListaDec} \rightarrow \text{ListaDec} \mid \text{vazio}$
- $\text{Dec} \rightarrow \text{int id} ;$

Lista de Tópicos da Segunda Área

- Analisadores descendentes (gramática não recursiva à esquerda e fatorada, logo...):
 - Descendentes recursivos preditivos
 - Descendentes recursivos com retrocesso
 - Analisador Tabular preditivo
 - Cálculo de First e Follow
 - Montagem da Tabela
 - Uso da Tabela

[Voltar para o índice](#)

- Analisadores Redutivos (ascendentes, bottom up)
 - Analisadores SLR e LR(0):
 - Calcular coleção de conjuntos de itens LR(0)
 - Montar e usar tabela de analisador SLR
 - Montar e usar tabela de analisador LR(0)
 - Analisadores LR(1) e LALR:
 - Calcular coleção de conjuntos de itens LR(1)
 - Montar e usar tabela de analisador LR(1)
 - Montar e usar tabela de analisador LALR
 - classificação da ordem de abrangência entre LR(0), SLR, LALR(1), LR(1)

8)(2003 - Tec - 47) Considere as seguintes afirmativas sobre as linguagens usadas para análise sintática:

- I A classe LL(1) não aceita linguagens com produções que apresentem recursões **diretas** à esquerda (ex. $L \rightarrow La$) mas aceita linguagens com recursões indiretas (ex. $L \rightarrow Ra$, $R \rightarrow Lb$)
- II A linguagem LR(1) reconhece a mesma classe de linguagens que LALR(1)
- III A linguagem SLR(1) reconhece uma classe de linguagens maior que LR(0)

Selecione a afirmativa correta:

- (a) As afirmativas I e II são verdadeiras
- (b) As afirmativas I e III são verdadeiras
- (c) Apenas a afirmativa III é verdadeira
- (d) As afirmativas II e III são verdadeiras
- (e) As afirmativas I e III são falsas

12) (2004 - Tec - 61) Considere as seguintes afirmações sobre as linguagens usadas para análise sintática:

- I Os analisadores LL(1) aceitam linguagens com produções que apresentem recursões a esquerda
- II Os analisadores LR(1) reconhecem uma classe de linguagens maior que os analisadores LALR(1)
- III Os analisadores SLR(1) reconhecem uma classe de linguagens menor que os analisadores LR(0)

Selecione a afirmativa correta:

- a) As afirmações I e II são verdadeiras
- b) As afirmações I e III são verdadeiras
- c) Apenas a afirmativa III é verdadeira
- d) As afirmações II e III são verdadeiras
- e) As afirmações I e III são falsas

16)(2006 - Fundamentos - 46) A respeito da gramática G abaixo,

$$S \rightarrow aAa$$

$$S \rightarrow bAb$$

$$A \rightarrow b$$

$$A \rightarrow \epsilon$$

considere as afirmativas:

I G é SLR(1).

II G é LL(1).

III G é sensível ao contexto.

É correto afirmar que:

(a) Somente I é verdadeira

(b) Somente II é verdadeira

(c) Somente III é verdadeira

(d) Somente I e III são verdadeiras

(e) Todas as 3 afirmativas são verdadeiras

(2012 - Tecnologia 70) Considere a gramática a seguir.

$$S \rightarrow E\$$$

$$E \rightarrow T + E$$

$$E \rightarrow T$$

$$T \rightarrow x$$

Com relação a essa gramática, atribua V (verdadeiro) ou F (falso) às afirmativas a seguir.

☐ A gramática é LR(0).

☐ Em uma tabela de análise SLR, a produção $T \rightarrow x$ terá reduções somente nos terminais $+$ e $\$$.

☐ A gramática é SLR.

☐ Em uma tabela de análise LR(0), a produção $E \rightarrow T$ terá reduções somente nos terminais x e $+$.

☐ A gramática é LR(1).

Assinale a alternativa que contém, de cima para baixo, a sequência correta.

- a) V, V, F, F, V.
- b) V, F, V, F, F.
- c) V, F, F, V, F.
- d) F, V, V, F, V.
- e) F, V, F, V, F.

[Voltar para o índice](#)

(2013 - Tecnologia 68) Considere a gramática a seguir.

$$E \rightarrow \text{num}$$

$$E \rightarrow E + E$$

$$E \rightarrow E - E$$

Sobre essa gramática, atribua V (verdadeiro) ou F (falso) às afirmativas a seguir.

☐ É ambígua.

☐ É LL(1).

☐ É LR(1).

☐ É SLR.

☐ Possui recursão à esquerda.

Assinale a alternativa que contém, de cima para baixo, a sequência correta.

- a) V, V, F, F, V.
- b) V, F, V, V, F.
- c) V, F, F, F, V.
- d) F, V, V, F, F.
- e) F, V, F, V, F.

[Voltar para o índice](#)

O gerador de analisadores léxicos Lex

- O Lex é um gerador de analisadores léxicos que recebe como entrada uma especificação léxica utilizando expressões regulares e gera um DFA.
- Normalmente ele é compilado e executado juntamente com uma implementação do Yacc, gerador de parsers, mas se a linguagem for uma linguagem regular, pode ser executado independentemente do Yacc.
- Há diversas implementações dele, aqui usaremos o flex (fast lex), e como implementação do Yacc usaremos o Bison.

- Um arquivo de especificação lex é dividido em 3 seções, separadas por linhas que contem apenas dois sinais de percentual ("%").
 - A seção de **definições**
%%
 - A seção de **regras**
%%
 - A seção de **código em C do usuário**

- A seção de definições contém definições de elementos que serão utilizados nas regras, bem como arquivos .h que serão incorporados no programa gerado.
- Essas definições podem ser expressas usando a mesma notação de expressões regulares descrita mais adiante, nas regras.
- Ex: numero $[0-9]^+$
- Também podem ser colocados trechos de código em C que serão copiados literalmente para o programa gerado, como protótipos de funções.

- A seção de regras contém uma série de expressões regulares estendidas que definem cada token que será reconhecido, bem como o que será retornado para cada token
- As expressões utilizam os mesmos operadores de expressões regulares simples (barra vertical para união, * para fecho, parênteses...) e mais alguns.
- "[e]" são utilizados para designar qualquer caracter de um determinado conjunto
 - Ex: '[xyz]' designa qualquer caracter do conjunto definido

- Referências a variáveis definidas na primeira seção devem ser feitas entre chaves (p.ex. {numero})
- 'r*' - zero ou mais ocorrências da expressão regular r
- O símbolo de interrogação é utilizado para designar um elemento opcional:
 - 'r?' - zero ou um r (um r opcional)

[Voltar para o índice](#)

- 'r+' - um ou mais r's
- 'r{2,5}' - de 2 a 5 ocorrências de r
- 'r{2,}' - 2 ou mais r's
- 'r{4}' - exatamente 4 r's
- '[a-d]' - qualquer caractere entre **a** e **d**
- '.' (caractere ponto) - qualquer caractere, exceto quebra de linha
- '[^abc]' - qualquer caractere exceto a,b,c

[Voltar para o índice](#)

```
/* Mini Calculator */  
/* calc.lex */
```

```
%{  
#include "heading.h"  
#include "tok.h"  
int yyerror(char *s);  
int yylineno = 1;  
%}
```

```
digit [0-9]  
int_const {digit}+
```


- A "compilação" bem sucedida de uma especificação lex gera uma função chamada `yylex()` que a cada chamada retorna o próximo token do arquivo de entrada. O token é o código numérico correspondente ao caracter do token, quando mesmo tem um único símbolo, ou um código sequencial numérico definido na especificação sintática ou na léxica por `defines` ou `enum`.
- Ao reconhecer um token e retorná-lo na função `yylex`, a variável global `yyltext` é atualizada com o lexema onde o token foi reconhecido.

- O arquivo de entrada por default é lido da entrada padrão (stdin, console). A entrada pode ser feita de um arquivo abrindo o arquivo e atribuindo seu descritor à variável global yyin.

```
FILE *fd=fopen("fonte.c","r");  
yyin=fd;
```

- Ao terminar de ler um arquivo (chegar no EOF) o lex consulta a função yywrap(). Se ela retornar 0 o scanner volta ao início do arquivo e recomeça, caso contrário ela encerra a varredura e a função yylex() retorna 0.
- O que permite fazer o main a seguir na especificação léxica:

```

int main()
{
    FILE *fd=fopen("fonte.c","r");
    yyin=fd;
    while (1)
    {
        int tk=yylex();
        if (tk==0) break;
        printf("\n%s\ttoken=%d\n",yytext,tk);
    }
    return 0;
}

```

- A lista completa de pré-definições globais do Lex é a seguinte:

Nome	Função
int yylex(void)	call to invoke lexer, returns token
char *yytext	pointer to matched string
yylen	length of matched string
yyval	value associated with token
int yywrap(void)	wrapup, return 1 if done, 0 if not done
FILE *yyout	output file
FILE *yyin	input file
INITIAL	initial start condition
BEGIN	condition switch start condition
ECHO	write matched string

```
%%
```

```
{int_const} { yy|val.int_val = atoi(yytext); return INTEGER_LITERAL; }  
"+" { yy|val.op_val = new std::string(yytext); return PLUS; }  
"*" { yy|val.op_val = new std::string(yytext); return MULT; }
```

```
[ \t]* {}  
[\n] { yylineno++; }
```

```
. { std::cerr << "SCANNER_"; yyerror(""); exit(1); }
```

[Voltar para o índice](#)

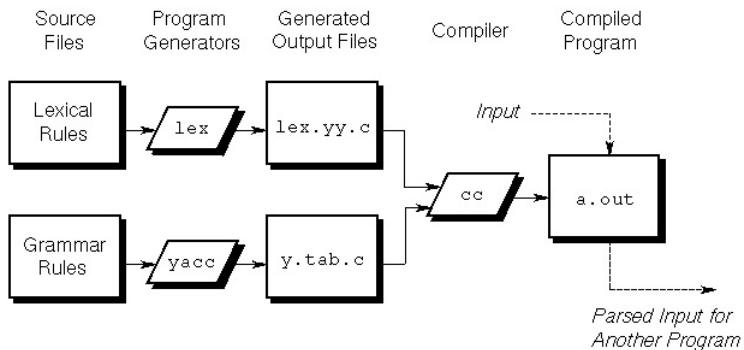
- O código na seção de código do usuário é copiado direto para a saída.
 - Pode ser usada para colocar funções que chamam ou são chamadas pelo léxico.
 - Se não há código de usuário, o segundo %% pode ser omitido
- A especificação completa do formato da entrada do flex pode ser encontrada em
<http://flex.sourceforge.net/manual/Format.html#Format>

[Voltar para o índice](#)

YACC - Yet Another Compiler Compiler

- Produz um parser bottom-up para uma dada gramática
- Usado para produzir compiladores para Pascal, C, C++ entre outras
- Bison : Substituto do yacc da GNU
 - Bison para Windows - Um gerador de parsers compatível com YACC
 - O Bison é um gerador de parsers de uso geral que converte uma gramática LALR(1) em um código em C que efetua a análise sintática para aquela gramática.

[Voltar para o índice](#)



ZK-0455U-R

Instalação do Flex e do Bison

- Baixar a versão atual do Flex em <http://gnuwin32.sourceforge.net/packages/flex.htm>
- Baixar a versão atual do Bison em <http://gnuwin32.sourceforge.net/packages/bison.htm>
- Efetuar a instalação completa de ambos no diretório C:\GnuWin32
 - Não instalar no diretório default (C:\Program Files (x86)\GnuWin32) porque o Bison não gosta de espaços e parênteses
- Instalar o Dev-Cpp em C:\Dev-Cpp (verificar antes se já não está instalado)
- Na janela do prompt acrescentar ao path C:\GnuWin32\bin;C:\Dev-Cpp\bin
 - algo como
PATH= %PATH%;C:\GnuWin32\bin;C:\Dev-Cpp\bin

- Carregar do acervo da disciplina o arquivo Hello.l com a especificação léxica para um diretório conhecido (p.ex. raiz do drive C)
- Idem para o arquivo com a especificação sintática Hello.y
- Rodar o flex sobre o arquivo Hello.l gerando o arquivo lex.yy.c
- Rodar o bison sobre o arquivo Hello.y gerando os arquivos y.tab.c e y.tab.h
 - Para que o y.tab.h seja gerado é necessário que o bison seja chamado com as opções -dy, conforme mostrado no exemplo a seguir
- Compilá-los com o gcc

C:\ Administrador: Prompt de Comando

```
C:\>set PATH=%PATH%;C:\GnuWin32\bin;C:\Dev-Cpp\bin
C:\>flex hello.l
C:\>bison -dy hello.y
C:\>gcc lex.yy.c y.tab.c -o hello.exe
C:\>hello
hi
Hello World
bye
Bye World
C:\>
```

[Voltar para o índice](#)

Hello.l

```
%{  
    #include <stdlib.h>  
    #include "y.tab.h"  
%}  
  
%%  
  
("hi"|"oi")"\n"      { return HI; }  
("tchau"|"bye")"\n"  { return BYE; }  
.                     { yyerror(); }  
  
%%  
  
int main( void )  
{  
    yyparse();  
    return 0;  
}  
  
int yywrap( void )  
{  
    return 0;  
}  
  
int yyerror( void )  
{  
    printf("Error\n");  
    exit(1);  
}
```

Hello.y

```
%token HI BYE
```

```
%%
```

```
program :
```

```
    hi bye
```

```
    ;
```

```
hi      :
```

```
    HI
```

```
        { printf("Hello_World\n"); }
```

```
    ;
```

```
bye     :
```

```
    BYE
```

```
        { printf("Bye_World\n"); exit(0); }
```

```
    ;
```

Formato de uma especificação do YACC

Definições

%%

Regras

%%

Código Suplementar

[Voltar para o índice](#)

- A seção de definições contem definições de elementos que serão utilizados nas regras, bem como arquivos .h que serão incorporados no programa gerado.
- Nessa seção são definidos os nomes dos tokens que serão usados na gramática. Tokens que tem apenas um símbolo são identificados pelo seu próprio caracter (p.ex. '+'). Às palavras reservadas ou tokens compostos por mais de um símbolo (ex. "+=") é associado um nome e listados nessa seção:
 - p.ex: % token IDENTIFIER CONSTANT
 - O YACC atribui a cada token uma numeração sequencial a partir de 280. Os números até 255 são para os tokens de um símbolo.

- Também podem ser colocados trechos de código em C que serão copiados literalmente para o programa gerado, como protótipos de funções.

- A seção de regras contém regras de produção no formato:

- $LHS : RHS1 \mid RHS2;$

- Uma especificação no YACC para a gramática

- $S \rightarrow 0 S 1$

- $S \rightarrow \epsilon$

- seria o arquivo ex1.y abaixo:

```
%%  
S : '0' S '1'  
  | /* epsilon */  
  ;
```

- E sua compilação por "bison -dy ex1.y" gera dois arquivos de saída chamados y.tab.c e y.tab.h

Expressões Regulares e Autômatos Finitos

- 1) Construa um autômato finito que reconheça o conjunto de strings sobre o alfabeto $\{0,1\}$ com o mesmo número de 0's e 1's tal que todo prefixo do string contenha no máximo um 0 a mais ou a menos que o número de 1's.
- 2) Dada uma expressão regular $(a(ab|cb)^*b(a|c)^*c)$ construa um autômato finito que reconheça a linguagem gerada pela expressão. Dê exemplos de sentenças que podem ser obtidas a partir da expressão.
- 3) Escreva uma expressão regular que gere comentários em Pascal, delimitados por $(*$ e $*)$.
- 4) Defina um autômato finito que aceite valores expressos em reais no formato $R\$ddd.ddd....ddd,dd$

5) Defina um autômato finito que aceite comentários em PL/1. Estes são delimitados por `/*` e `*/` não contendo a seqüência `*/`.

6) Considere o autômato $A(\{a,b,c,d,e,f,g,h\}, \{0,1\}, M, a, \{d\})$

com	$M(a,0)=b$	$M(a,1)=a$	$M(b,0)=a$	$M(b,1)=c$
	$M(c,0)=d$	$M(c,1)=b$	$M(d,0)=d$	$M(d,1)=a$
	$M(e,0)=d$	$M(e,1)=f$	$M(f,0)=g$	$M(f,1)=e$
	$M(g,0)=f$	$M(g,1)=g$	$M(h,0)=g$	$M(h,1)=d$

Encontre o autômato mínimo equivalente.

7) Construa NFA's para as expressões regulares a seguir. Mostre a seqüência de movimentos para processar o string ababbab

a) $(a|b)^*$

b) $(a^*|b^*)^*$

c) $((\epsilon|a)b^*)^*$

d) $(a|b)^*abb(a|b)^*$

8) Converta os NFA's do exercício anterior em DFA's. Minimize os autômatos e mostre os movimentos para ABABBAB.

[Voltar para o índice](#)

9) Desenvolva autômatos finitos determinísticos que reconheçam as seguintes linguagens sobre $\Sigma = \{a, b\}$

a) $\{w \mid w \text{ possui } aaa \text{ como subpalavra}\}$

b) $\{w \mid \text{o sufixo de } w \text{ é } aa\}$

c) $\{w \mid w \text{ possui número ímpar de } a \text{ e } b\}$

d) $\{w \mid w \text{ possui número par de } a\text{'s e ímpar de } b\text{'s ou } w \text{ possui número par de } b\text{'s e ímpar de } a\text{'s}\}.$

e) $\{w \mid \text{o quinto símbolo da direita para a esquerda de } w \text{ é } a\}$

[Voltar para o índice](#)

10) Desenvolver AFNs com ou sem transições- ϵ , que reconheçam as seguintes linguagens:

a) sobre o alfabeto $\Sigma = \{a, b, c\}$

a.1) $\{w \mid aa \text{ ou } bb \text{ é subpalavra e } cccc \text{ é sufixo de } w\};$

b) sobre o alfabeto $\Sigma = \{a, b\}$

b.1) $\{w_1 w_2 w_1 \mid w_2 \text{ é qualquer e } |w_1|=3\}$

b.2) $\{w \mid \text{o décimo símbolo da direita para a esquerda de } w \text{ é } a\}$

b.3) $\{w \mid w \text{ possui igual número de } a\text{'s e } b\text{'s e qualquer prefixo de } w \text{ possui no máximo dois } a\text{'s a mais que } b\text{'s ou dois } b\text{'s a mais que } a\text{'s.}\}$

11) Descreva em palavras as linguagens geradas pelas seguintes expressões regulares:

a) $(aa + b)^*(a + bb)$

b) $(b + ab)^*(\epsilon + a)$

c) $(aa + bb + (aa + bb)(ab + ba)(aa + bb))^*$

12) Aplique o algoritmo de tradução de formalismo de expressão regular para autômato finito:

a) $(ab + ba)^*(aa + bb)^*$

b) $ab(abb^* + baa^*)^*ba$

[Voltar para o índice](#)

1) Descreva a linguagem gerada por cada uma das seguintes expressões regulares, e escreva um autômato finito determinístico e uma gramática regular equivalente a cada um deles.

a) $(0|1)^*011$

b) $(a|b)(a|b|0|1)^*$

c) $(00|11)^*((01|10)(00|11)^*(01|10)(00|11)^*)^*$

2) Construa uma gramática regular para a linguagem gerada pela seguinte gramática:

$A \rightarrow B|C$

$B \rightarrow 0B|1B|011$

$C \rightarrow 0D|1C|e$

$D \rightarrow 0C|1D$

3) Construa uma expressão regular, um autômato finito e uma gramática regular que gere os strings em $\{0, 1\}^*$ cujo tamanho é múltiplo de 3.

4) Construa uma expressão regular e uma gramática equivalentes ao autômato $M = (\{p, q, r\}, \{0, 1\}, d, p, \{r\})$ com d especificado abaixo. Descreva a linguagem gerada por este autômato.

	Entrada	
δ	0	1
p	{q}	{p}
q	{r}	{p}
r	{r}	{r}

[Voltar para o índice](#)

5) Encontre o autômato finito com menor número de estados para a linguagem especificada pelo autômato finito

$M = (\{A, B, C, D, E, F\}, \{0, 1\}, d, A, \{E, F\})$ onde d é dada por:

	Entrada	
d	0	1
A	B	C
B	E	F
C	A	A
D	F	E
E	D	F
F	D	E

[Voltar para o índice](#)

6) Construa autômatos finitos que aceitem as seguintes linguagens sobre o alfabeto $\{0,1\}$

a) O conjunto de todos strings que terminam em 00.

b) O conjunto de todos os strings com 3 0's consecutivos.

c) O conjunto de todos strings tal que a cada 3 caracteres consecutivos pelo menos dois são 0.

d) O conjunto de todos os strings tal que o penúltimo símbolo é 0.

7) Escreva uma expressão regular e uma gramática equivalente para cada autômato.

[Voltar para o índice](#)

Considere a gramática

$$S \rightarrow (L) \mid a$$
$$L \rightarrow L,S \mid S$$

- a) Quais são os terminais, não terminais e símbolo inicial?
- b) Encontre árvores de derivação para as seguintes sentenças:
 - i) (a,a)
 - ii) $(a,(a,a))$
 - iii) $(a,((a,a),(a,a)))$
- c) Construa uma derivação mais a esquerda para cada uma das sentenças em b)
- d) Construa uma derivação mais à direita para cada uma das sentenças em b)
- e) Que linguagem esta gramática gera?

8) Considere a gramática

$$S \rightarrow aSbS \mid bSaS \mid \epsilon$$

- a) Mostre que esta gramática é ambígua construindo duas derivações mais à esquerda para a sentença abab
- b) Construa as derivações mais à direita correspondentes para abab
- c) Construa as árvores de derivação correspondentes para abab
- d) Qual a linguagem gerada por esta gramática?

[Voltar para o índice](#)

9) Tente projetar uma gramática para cada uma das seguintes linguagens. Quais linguagens são regulares?

a) O conjunto de todos os strings de 0's e 1's tal que cada 0 é imediatamente seguido por ao menos um 1.

b) Strings de 0's e 1's com o mesmo número de 0's e 1's.

c) Strings de 0's e 1's com número diferente de 0's e 1's

d) Strings de 0's e 1's em que 011 não aparece como substring.

e) Strings de 0's e 1's na forma XY onde $X < > Y$

f) Strings de 0's e 1's na forma XX .

[Voltar para o índice](#)

10) Escreva um autômato finito determinístico equivalente à expressão regular

$$(a|b)^*a(a|b)(a|b)$$

e a gramática equivalente

11) Remova a recursão da gramática de expressões booleanas a seguir:

$$E \rightarrow E \text{ or } T \mid T$$

$$T \rightarrow T \text{ and } F \mid F$$

$$F \rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false}$$

[Voltar para o índice](#)

12) Especifique uma gramática livre de contexto que gere sentenças da forma

$$a^n b^m c^m d^n$$

onde $n \geq 1$ e $m \geq 1$, ou seja, seqüências de a, b, c, d onde o número de a's e d's é igual, assim como o número de b's e c's (Resposta : $S \rightarrow aSd \mid aAd$, $A \rightarrow bc \mid bAc$)

13) Especifique uma CFG que gere sentenças sobre $\{a, b\}^*$ onde o número de a's é duas vezes o número de b's. (Resposta : $S \rightarrow SaSaSbS \mid S \rightarrow SaSbSaS \mid S \rightarrow SbSaSaS \mid S \rightarrow \epsilon$)

14) Especifique uma CFG que gere sentenças sobre $\{a, b\}$ no formato $a^n b a^m$, $n, m > 0$.

15) Especifique uma AFD, uma gramática regular e a expressão regular equivalente que gerem todas as sentenças de $\{0,1\}^*$ formadas por 0's e 1's alternados.

16) Especifique um autômato finito, uma gramática regular e uma expressão regular equivalente que gerem todas as sentenças em $\{0,1\}$ com no mínimo três símbolos e que o terceiro da direita para a esquerda (antepenúltimo) seja 0.

17) Defina uma gramática cujas sentenças sejam expressões regulares sobre o alfabeto $\{a,b\}$

18) Seja a gramática G abaixo, que gera expressões do cálculo de predicados:

$$G = (\{S\}, \{p, q, \sim, [,], \supset\}, P, S)$$

$$P = \{S \rightarrow \sim S \mid [S] \mid S \supset S \mid p \mid q\}$$

Pergunta-se : G é ambígua? Justifique. Caso afirmativo defina G' não ambígua, tal que $L(G') = L(G)$

19) Elimine a recursividade à esquerda da gramática anterior.

[Voltar para o índice](#)

20) Dada a gramática G abaixo, resolva:

$G=(N,T,P,S)$

$N=\{S,L,E\}$

$T=\{\text{begin, end, id, ;, :=, v, }\}$

P: $S \rightarrow \text{begin } L \text{ end} \mid \text{id} := E$

$L \rightarrow L;S \mid S$

$E \rightarrow E \vee E \mid \sim E \mid \text{id}$

a) A gramática G é ambígua? Justifique. Caso positivo, defina uma G'equivalente

b) Elimine a recursividade à esquerda das produções de G.

[Voltar para o índice](#)

20) Considere a gramática $G = (N, T, P, S)$, onde

$N = \{S, A, B\}$

$T = \{a, b\}$

$P = \{$
 $S \rightarrow aB$
 $S \rightarrow bA$
 $A \rightarrow bAA$
 $A \rightarrow a$
 $A \rightarrow aS$
 $B \rightarrow b$
 $B \rightarrow bS$
 $B \rightarrow aBB \}$

Verifique se as palavras abaixo pertencem a $L(G)$. Caso afirmativo, desenhe a árvore de derivação correspondente.

a) ababab

b) bbbbaaab

c) aabbaabb

21) Defina uma CFG que gere expressões aritméticas em Pascal. Considere variáveis, constantes e nomes de funções como símbolos terminais da gramática.

22) Defina uma CFG que gere sentenças em $\{a, b\}^*$ tal que cada sentença contenha igual número de a's e b's.

[Voltar para o índice](#)

A Sentença Vazia

Para que uma gramática (do tipo 1, 2 ou 3) gere a sentença vazia ela deve conter a produção $S \rightarrow \epsilon$, sendo S o símbolo inicial.

Entretanto, isto só é possível se S não aparecer do lado direito de qualquer outra produção. Deste modo a regra $S \rightarrow \epsilon$ só será utilizada para dar origem à sentença vazia. A gramática que possuir o símbolo inicial aparecendo do lado direito de regras de produção deverá ser transformada em outra equivalente que obedeça a esta restrição.

$$\begin{aligned}\text{Ex: } P: \quad S &\rightarrow aA \\ A &\rightarrow aS \mid a\end{aligned}$$

Solução:

$$G' = (\{S', S, A\}, \{a\}, P', S')$$

$$P': \quad S' \rightarrow aA \mid \epsilon$$

$$S \rightarrow aA$$

23) Seja a gramática regular G definida por

$G = (\{S, B, C\}, \{a, b, c\}, P, S)$

$P = \{$
 $S \rightarrow aS$
 $S \rightarrow bB$
 $B \rightarrow cB$
 $B \rightarrow cC$
 $C \rightarrow cS$
 $C \rightarrow c \}$

a) Construa G' regular, tal que $L(G') = L(G) \cup \{\epsilon\}$

b) Construa um AFD e a expressão regular equivalente.

[Voltar para o índice](#)

24) Defina uma gramática regular que gere sentenças em $\{0, 1\}^*$, as quais não contêm a seqüência 101

25) Construa uma gramática que gere identificadores segundo o formato

$$L(L|D)^*(_ (L|D)(L|D)^*)^*$$

26) Construa um autômato finito e a gramática regular equivalente à seguinte expressão regular

$$a(a|bc)^*b(a|b)^*$$

[Voltar para o índice](#)

- 27) Defina uma gramática regular que gera seqüências de 0's e 1's tal que todo 0 seja imediatamente seguido por 1.
- 28) Defina uma gramática livre de contexto que gere sentenças em $\{a, b\}^+$ tal que cada sentença seja igual à sua reversa.
- 29) Defina uma gramática livre de contexto que gere sentenças em $\{a, b\}^+$ tal que a's e b's apareçam alternados em cada sentença.
- 30) Defina um autômato finito que reconheça sentenças em $\{a|b\}^+$ que contem um número par de a's e um número par de b's.
Escreva uma gramática regular equivalente.

31) Considere os seguintes operadores utilizados nas expressões da linguagem utilizada na disciplina de algoritmos:

Operador	Precedência	Associatividade
$\wedge$ (potenciação)	0	$\leftarrow$
$*, /$	1	$\rightarrow$
$+, -$	2	$\rightarrow$
relacionais	3	não associativo
E lógico	4	$\rightarrow$
OU lógico	5	$\rightarrow$

[Voltar para o índice](#)

32) Escreva uma gramática não ambígua que gere expressões relacionais na linguagem de algoritmos incluindo parênteses e permitindo como elementos das expressões constantes, variáveis ou chamadas de funções com argumentos.

33) Desenvolva expressões regulares e gramáticas regulares que gerem as seguintes linguagens sobre $\Sigma = \{a, b\}$

a) $\{w \mid w \text{ tem no máximo um par de } a\text{'s como subpalavra e no máximo um par de } b\text{'s como subpalavra}\}$

b) $\{w \mid \text{qualquer par de } a\text{'s antecede qualquer par de } b\text{'s}\}$

c) $\{w \mid w \text{ não possui aba como subpalavra}\}$

[Voltar para o índice](#)

34) Representa a seguinte linguagem baseada em unidades léxicas da linguagem de programação pascal, usando os formalismos AFD, ER e gramática regular:

$$\{w \mid w \text{ é número inteiro ou } w \text{ é número real ou } w \text{ é identificador da linguagem pascal}\}$$

[Voltar para o índice](#)

35) Escreva um AFD equivalente à ER $(a|b)^*a(a|b)(a|b)$ e a gramática equivalente.

36) Aplique os algoritmos de tradução de formalismo apresentados e, a partir da expressão regular $(b + \epsilon)(a + bb)^*$, realize as diversas etapas até gerar a gramática regular correspondente
($ER \Rightarrow AF_{\epsilon} \Rightarrow AFN \Rightarrow AFD \Rightarrow GR$)

[Voltar para o índice](#)

Parsers descendentes recursivos

1) Escreva um parser descendente recursivo com retrocesso para a seguinte gramática:

$$S \rightarrow aC \mid aD$$
$$C \rightarrow cC \mid c$$
$$D \rightarrow cE$$
$$E \rightarrow dE \mid d$$

[Voltar para o índice](#)

- 2) Redefina a gramática da questão 1 e escreva um parser descendente recursivo preditivo para a gramática da questão 1.
- 3) É possível construir um parser descendente recursivo preditivo para a gramática $S \rightarrow aSbS \mid bSaS \mid \epsilon$? E com retrocesso? Construa este parser.
- 4) A gramática $S \rightarrow aSa \mid aa$ gera todas as cadeias de a's de comprimento par, exceto a cadeia vazia.
- a) Construa um parser descendente recursivo com retrocesso para esta gramática que teste a alternativa aSa antes de aa . Mostre que o procedimento S tem sucesso para 2, 4 e 8 a's, mas falha para 6 a's.
- b) Que linguagem o parser reconhece?.

Análise de Precedência de Operadores

5) Defina a tabela de precedência de operadores para a gramática a seguir

$S \rightarrow \text{if } c \text{ then } S \text{ else } S$

$S \rightarrow \text{if } c \text{ then } S$

$S \rightarrow s$

[Voltar para o índice](#)

6) Defina as funções de precedência de operadores $f()$ e $g()$ para a tabela de precedência a seguir:

	or	b	\$
or	=	b	>
>	\$	<	<

[Voltar para o índice](#)

Resposta :

	or	b	\$
fa	1	2	0
ga	1	1	0

[Voltar para o índice](#)

7) Considere a gramática

$$S \rightarrow (L) \mid a$$
$$L \rightarrow L,S \mid S$$

7.1) Construa a tabela de precedência de operadores e faça o reconhecimento de:

i) (a,a)

ii) $(a,(a,a))$

iii) $(a,((a,a),(a,)))$

7.2) Encontre funções de precedência para a tabela de precedência.

8) (Aho 4.33) Considere a gramática ambígua

$S \rightarrow AS \mid b$

$A \rightarrow AS \mid b$

a) Construa a coleção de conjuntos LR(0) para esta gramática.

b) Construa a tabela sintática SLR.

c) Mostre todos os movimentos permitidos pela tabela a partir de
(b) à entrada abab

[Voltar para o índice](#)

9) (Aho 4.34) Construa a tabela sintática SLR para a gramática do exercício 4.3, mostrada a seguir:

$\text{bexpr} \rightarrow \text{bexpr or btermo} \mid \text{btermo}$

$\text{btermo} \rightarrow \text{btermo and bfator} \mid \text{bfator}$

$\text{bfator} \rightarrow \text{not bfator} \mid (\text{bexpr}) \mid \text{true} \mid \text{false}$

10) (Aho 4.35) Considere a seguinte gramática

$E \rightarrow E + T \mid T$

$T \rightarrow TF \mid F$

$F \rightarrow F^* \mid a \mid b$

a) Construir a tabela sintática SLR para esta gramática

[Voltar para o índice](#)

11) Construa um parser descendente recursivo para a gramática $S \rightarrow aSa|aa$. É possível construir um parser preditivo para esta gramática?

12) Para a gramática G de expressões booleanas a seguir:

$E \rightarrow E \text{ or } T \mid T$

$T \rightarrow T \text{ and } F \mid F$

$F \rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false}$

a) remova a recursão da gramática

b) Calcule First e Follow dos não terminais da gramática G' resultante

c) Escreva um parser descendente recursivo preditivo

d) Escreva uma tabela de precedência de operadores para G

13) Escreva uma tabela de precedência de operadores para a gramática a seguir.

```
Comando → while exp do Comando |  
          begin lista end |  
          atrib  
lista →   lista ; comando |  
          comando
```

[Voltar para o índice](#)

14) Considere a gramática G abaixo:

$S \rightarrow \text{case } E \text{ of } L \text{ end}$

$S \rightarrow a$

$E \rightarrow n$

$L \rightarrow L ; C \mid C$

$C \rightarrow k : S$

a) Escreva um analisador descendente recursivo que reconheça $L(G)$;

b) Determine a matriz de precedência e complete a tabela da função de precedência para G;

	case	of	n	end	a	;	k	:	\$
f		0		2		2		2	
g	3		1		3		3		0

[Voltar para o índice](#)

- c) Construa uma tabela LL (a tabela do analisador tabular preditivo) para reconhecer $L(G)$ e mostre os movimentos que um analisador LL (tabular preditivo) faria no reconhecimento da sentença "case n of k:a;a end";
- d) G é uma gramática SLR(1)? Justifique.

[Voltar para o índice](#)

15) Seja a sintaxe simplificada (abaixo) que gera cláusulas Prolog. Especifique um analisador top-down e um bottom-up que reconheçam cláusulas em Prolog.

$$C \rightarrow P. \mid P \leftarrow S.$$
$$P \rightarrow p (L)$$
$$S \rightarrow S \& P \mid P$$
$$L \rightarrow L , T \mid T$$
$$T \rightarrow P \mid v$$

[Voltar para o índice](#)

16) Dada a gramática $G = (N, T, P, S)$ onde

$N = \{ S, L, E \}$

$T = \{ \text{repeat}, \text{until}, \text{id}, ;, :=, v, \}$

$P = \{ S \rightarrow \text{repeat } L \text{ until } E \mid \text{id} := E$

$L \rightarrow L ; S \mid S$

$E \rightarrow E \vee E \mid E \mid \text{id} \}$

a) Escreva um analisador descendente recursivo preditivo que reconheça $L(G)$

b) Escreva um analisador tabular preditivo que reconheça $L(G)$.

Mostre o First e Follow de todos os não terminais

c) Gere uma gramática equivalente não ambígua. Diga quais suposições foram feitas em relação à precedência e associatividade dos operadores

d) G é uma gramática de precedência de operadores? Justifique. Construa uma tabela de precedência de operadores para reconhecer $L(G)$. Diga quais suposições foram feitas com relação à precedência e associatividade dos operadores.

e) Construa uma tabela SLR(1) para G' , gramática resultante da seguinte simplificação de G :

$E \rightarrow E \vee id \mid E \mid id$

[Voltar para o índice](#)

17) Considere a gramática

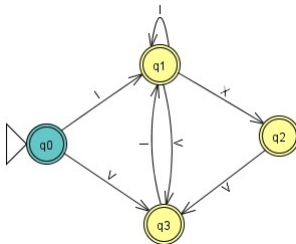
$$S \rightarrow (L) \mid a$$
$$L \rightarrow L , S \mid S$$

Elimine a recursividade à esquerda e construa um analisador preditivo para a gramática resultante.

[Voltar para o índice](#)

Questões do ENADE

(2005 - Ciência da Computação - Questão 70 - DISCURSIVA) A máquina finita de estados (MFE) cujo diagrama é visto abaixo reconhece seqüências de caracteres compostos pelas letras I, V e X, que representam, em notação romana, números correspondentes ao intervalo de 1 a 9, na notação arábica. Considere que todas as sentenças de entrada da MFE representam números romanos válidos, ou seja, a MFE não tem mecanismo de tratamento de erros.



Observe a correspondência da representação dos alfabetos romano e árabe fornecida pela tabela abaixo.

Romano	Árabe
I	1
V	5
X	10
L	50
C	100
D	500

Considerando essas informações, estenda a MFE apresentada acima para:

- reconhecer números no alfabeto romano correspondentes aos números de 1 a 20 no alfabeto árabe, com no máximo oito estados (valor: 5,0 pontos).
- reconhecer números no alfabeto romano correspondentes aos números de 1 a 500 no alfabeto árabe, com no máximo oito estados. (valor: 5,0 pontos)

Questões do POSCOMP

1)(2002 - Fund - 26) Sobre a hierarquia de Chomsky podemos afirmar que:

- (a) Uma linguagem que é recursivamente enumerável não pode ser uma linguagem regular
- (b) As linguagens livres de contexto e as linguagens sensíveis ao contexto se excluem
- (c) Uma linguagem que não é regular é livre de contexto
- (d) As linguagens reconhecidas por autômatos a pilha são as linguagens regulares
- (e) Há linguagens que não são nem livres de contexto e nem sensíveis ao contexto

2)(2002 - Tec - 67) Para a gramática a seguir, qual o conjunto de terminais que pode aparecer como primeiro terminal após o não-terminal A, em qualquer forma sentencial gerada pela gramática abaixo (isto é, não necessariamente imediatamente após A), onde ϵ representa a sentença vazia?

$$S \rightarrow ABCDd$$

$$A \rightarrow aA | \epsilon$$

$$B \rightarrow bC | \epsilon$$

$$C \rightarrow cD | \epsilon$$

$$D \rightarrow e$$

a) $\{d\}$

b) $\{b\}$

c) $\{b, c, e\}$

d) $\{b, c, d, e\}$

e) $\{e\}$

3)(2002 - Tec - 68) Qual das afirmações a seguir, relativas à análise sintática, está INCORRETA?

- a) As gramáticas LL podem descrever mais linguagens do que as gramáticas LR.
- b) Analisadores sintáticos descendentes recursivos são mais simples de implementar do que analisadores sintáticos redutivos.
- c) Uma das diferenças entre os diversos algoritmos de análise redutiva é a forma de identificar o *handle* na pilha.
- d) Algoritmos de análise redutiva podem ser utilizados mesmo para gramáticas ambíguas.
- e) Algoritmos descendentes recursivos podem ser utilizados para algumas gramáticas ambíguas.

4)(2003 - Fund - 27) Uma gramática G é definida por:

$$G = (\{x, y, z\}, \{S, W, X, Y, Z\}, P, S)$$

na qual os membros de P são:

$$S \rightarrow WZ$$

$$W \rightarrow X|Y$$

$$X \rightarrow x|xX$$

$$Y \rightarrow y|yY$$

$$Z \rightarrow z|zZ$$

Qual das expressões regulares abaixo corresponde a esta gramática?

(a) $(xx^*|yy^*)zz^*$

(b) $xx^*|yy^*|zz^*$

(c) $xx^*(yy^*|zz^*)$

(d) $(xx|yy)^*zz^*$

(e) $xx^*yy^*zz^*$

5)(2003 - Fund - 28) Considere o seguinte trecho de programa:

```
i:=1;    {linha 1}  
while i<=n do {linha 2}  
begin  
    sum:= sum + a[i];    {linha 3}  
    i:=i+1;              {linha 4}  
end;
```

Considere que:

- *I* representa a inicialização da variável $i:=1$ na linha 1;
- *T* representa o teste da linha 2;
- *A* representa os comandos da linha 3;
- *P* representa o incremento na linha 4.

Qual é a expressão regular que representa todas as seqüências de passos possíveis de serem executados por este trecho de programa?

(a) $I(TAP)^+$

(b) $I(TAP)^*$

(c) $IT^+A^*P^*$

(d) $IT(APT)^*$

(e) $IT(APT)^+$

[Voltar para o índice](#)

6)(2003 - Fund - 29) Um compilador detecta:

- (a) erros que podem ocorrer durante a execução do programa
- (b) erros nos resultados gerados pelo programa
- (c) erros de sintaxe do programa
- (d) erros aritméticos
- (e) todos os erros citados acima

[Voltar para o índice](#)

7) (2003 - Tec - 46) Considere as seguintes afirmações sobre autômatos finitos e expressões regulares:

- I A classe de linguagens aceita por um Autômato Finito Determinístico (AFD) não é a mesma que um Autômato Finito Não Determinístico (AFND).
- II Para algumas expressões regulares não é possível construir um AFD.
- III A expressão regular $(b|ba)^+$ aceita os *strings* de b's e a's começando com b e não tendo dois a's consecutivos.

Selecione a afirmativa correta:

- (a) As afirmativas I e II são verdadeiras
- (b) As afirmativas I e III são falsas
- (c) Apenas a afirmativa III é verdadeira
- (d) As afirmativas II e III são falsas
- (e) As afirmativas I e III são verdadeiras

8)(2003 - Tec - 47) Considere as seguintes afirmativas sobre as linguagens usadas para análise sintática:

- I A classe $LL(1)$ não aceita linguagens com produções que apresentem recursões **diretas** à esquerda (ex. $L \rightarrow La$) mas aceita linguagens com recursões indiretas (ex. $L \rightarrow Ra$, $R \rightarrow Lb$)
- II A linguagem $LR(1)$ reconhece a mesma classe de linguagens que $LALR(1)$
- III A linguagem $SLR(1)$ reconhece uma classe de linguagens maior que $LR(0)$

Selecione a afirmativa correta:

- (a) As afirmativas I e II são verdadeiras
- (b) As afirmativas I e III são verdadeiras
- (c) Apenas a afirmativa III é verdadeira
- (d) As afirmativas II e III são verdadeiras
- (e) As afirmativas I e III são falsas

9)(2003 - Tec - 48) Seja a seguinte linguagem, onde ϵ representa o string vazio e $\$$ representa um marcador de fim de entrada:

$$S \rightarrow ABCD$$

$$A \rightarrow a|\epsilon$$

$$B \rightarrow a|\epsilon$$

$$C \rightarrow c|\epsilon$$

$$D \rightarrow S|c|\epsilon$$

É incorreto afirmar que:

- (a) O conjunto $\text{FIRST}(A) = a, \epsilon$
- (b) O conjunto $\text{FIRST}(D)$ é igual ao conjunto $\text{FIRST}(S)$
- (c) O conjunto $\text{FOLLOW}(A) = a, c, \$$
- (d) O conjunto $\text{FOLLOW}(B) = c, \$$
- (e) O conjunto $\text{FOLLOW}(D)$ é igual ao $\text{FOLLOW}(S)$

10) (2004 - Fund - 21) Seja $\Sigma = \{a, b\}$. Uma expressão regular denotando a linguagem $L = \{w \in \Sigma^* \text{ tal que toda ocorrência de "a" em } w \text{ é imediatamente seguida de "b"}\}$ é:

(a) $(a^*b)^*$

(b) $(b + ab)^*$

(c) a^*b

(d) $b + (ab)^*$

(e) $(ab)^*$

[Voltar para o índice](#)

11) (2004 - Fund - 36) As seguintes expressões regulares denotam as linguagens P, Q, L e R, respectivamente: $(1 + 10)^*$, $(0 + 01)^*$, $(0 + 1)^*$, $0(11)^* + 1(00)^*$. Não se pode afirmar que: (obs: o operador " $\setminus$ " representa a operação de diferença de conjuntos)

- (a) $P \cap Q \neq \emptyset$
- (b) $P \cup Q \neq L$
- (c) $P \cap Q = \{\epsilon\}$
- (d) $(1 + 0)^* \setminus P = Q$
- (e) $R \subset L \setminus (P \cup Q)$

[Voltar para o índice](#)

12) (2004 - Tec - 61) Considere as seguintes afirmações sobre as linguagens usadas para análise sintática:

- I Os analisadores LL(1) aceitam linguagens com produções que apresentem recursões a esquerda
- II Os analisadores LR(1) reconhecem uma classe de linguagens maior que os analisadores LALR(1)
- III Os analisadores SLR(1) reconhecem uma classe de linguagens menor que os analisadores LR(0)

Selecione a afirmativa correta:

- a) As afirmações I e II são verdadeiras
- b) As afirmações I e III são verdadeiras
- c) Apenas a afirmativa III é verdadeira
- d) As afirmações II e III são verdadeiras
- e) As afirmações I e III são falsas

13) (2004 - Tec - 62) Seja a seguinte linguagem onde ϵ representa a sentença vazia:

$$S \rightarrow AB|CD$$

$$A \rightarrow a|\epsilon$$

$$B \rightarrow b|f$$

$$C \rightarrow c|g$$

$$D \rightarrow h|i$$

Qual o conjunto de terminais que podem começar sentenças derivadas de S?

- a) $\{a, c, g\}$
- b) $\{a, b, f, c, g\}$
- c) $\{a, b, f, c, g, h, i\}$
- d) $\{a, c, g, h, i\}$
- e) $\{a, b, f\}$

14)(2005 - Fund - 28) Qual das seguintes afirmações é **falsa**?

- (a) Dada uma máquina de Turing M com alfabeto de entrada Σ e uma string $w \in \Sigma$, não se sabe se a computação de M com entrada w vai ou não parar.
- (b) O problema da parada é indecidível.
- (c) Não existe algoritmo que determina quando uma gramática livre de contexto arbitrária é ambígua.
- (d) Não existe autômato finito determinístico que reconheça alguma linguagem livre de contexto.
- (e) Um autômato com duas pilhas pode ser simulado por uma máquina de Turing.

15)(2006 - Fund - 27) Considere as afirmações abaixo, onde o alfabeto das linguagens é sempre dado por $\Sigma = \{0, 1\}$.

- (I) A linguagem formada por todas as cadeias $x \in \Sigma^*$ onde após cada dois zeros **consecutivos** sempre ocorrem **pelo menos** dois uns. Note que: os uns não precisam ser consecutivos, nem precisam ocorrer **imediatamente** após os zeros.
- (II) Se L é livre de contexto e R é regular, então a linguagem

[Voltar para o índice](#)

16)(2006 - Fundamentos - 46) A respeito da gramática G abaixo,

$$S \rightarrow aAa$$

$$S \rightarrow bAb$$

$$A \rightarrow b$$

$$A \rightarrow \epsilon$$

considere as afirmativas:

I G é SLR(1).

II G é LL(1).

III G é sensível ao contexto.

É correto afirmar que:

(a) Somente I é verdadeira

(b) Somente II é verdadeira

(c) Somente III é verdadeira

(d) Somente I e III são verdadeiras

(e) Todas as 3 afirmativas são verdadeiras

(2010 - Fundamentos - 42) Considerando as linguagens

$$L_1 = \{a^l c^m b^n; l \geq 0, m \geq 0, n \geq 0\} \text{ e}$$

$$L_2 = \{a^l c^m b^n; l \geq 0, m \geq 0, n = l + m\} \text{ sobre o alfabeto}$$

$P = \{a, b, c\}$, considere as afirmativas a seguir.

- I. L_1 é uma linguagem regular.
- II. L_2 é uma linguagem regular.
- III. Existe um autômato de pilha determinístico que reconhece L_1 .
- IV. A linguagem L_2 pode ser gerada pela $G = (\{X, Y\}, \{a, b, c\}, \{X \rightarrow aXb, X \rightarrow Y, Y \rightarrow cYb, Y \rightarrow \lambda\}, X)$, onde λ é a palavra vazia.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas II e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.
- e) Somente as afirmativas I, III e IV são corretas.

[Voltar para o índice](#)

(2010 - Fundamentos - 44) Em relação a autômatos e linguagens, podemos afirmar:

- a) Existem linguagens reconhecidas por autômatos finitos não determinísticos que não podem ser reconhecidas por autômatos finitos determinísticos.
- b) Existem linguagens reconhecidas por autômatos de pilha não determinísticos que não podem ser reconhecidas por autômatos de pilha determinísticos.
- c) Linguagens infinitas somente são reconhecidas por autômatos de pilha.
- d) Linguagens regulares não podem ser reconhecidas por autômatos de pilha.
- e) Linguagens livres de contexto podem ser reconhecidas por autômatos finitos.

(2010 - Fundamentos - 45) Dado um inteiro fixo $k > 0$ e a linguagem $L = \{a^{2^n}c^n b^n; n \leq k\}$, podemos afirmar que

- a) L é uma linguagem livre de contexto, mas não regular.
- b) L é uma linguagem sensível ao contexto, mas não livre dele.
- c) L é uma linguagem regular apenas.
- d) L é uma linguagem recursiva, mas não sensível ao contexto.
- e) L é uma linguagem recursiva, mas não regular.

[Voltar para o índice](#)

(2010 - Técnicas - 65) Considere o trecho de código C a seguir.

```
int main()  
{  
    int i;  
    float f;  
    int v[10]  
    i = 3;  
    v[f] = 45;  
    while  
        {  
        }  
}
```

Quantos erros semânticos o código possui?

- a) 1
- b) 2
- c) 3
- d) 4
- e) 5

(2010 - Técnicas - 68) Considere os tipos de gramáticas para análise sintática a seguir.

I. Gramática de precedência simples.

II. Gramática LL(1).

III. Gramática de operadores.

Qual alternativa indica o método correto de análise sintática para cada uma das gramáticas apresentadas?

- a) I - Análise Descendente, II - Análise Descendente, III - Análise Ascendente.
- b) I - Análise Ascendente, II - Análise Ascendente, III - Análise Descendente.
- c) I - Análise Descendente, II - Análise Ascendente, III - Análise Descendente.
- d) I - Análise Ascendente, II - Análise Descendente, III - Análise Ascendente.
- e) I - Análise Ascendente, II - Análise Ascendente, III - Análise Ascendente.

(2011 - Fundamentos - 32) Considere a seguinte propriedade sobre uma linguagem formal L : "Existe um número $p \geq 0$, tal que para qualquer palavra $w \in L$, $|w| \geq p$, existem palavras x , y e z , com $y \neq \epsilon$ e $|xy| \leq p$, tais que, para qualquer inteiro $i \geq 0$, a palavra $xy^i z \in L$ ".

Com base no enunciado e nos conhecimentos sobre o tema, atribua V (verdadeiro) ou F (falso) para as afirmativas a seguir.

- () Se L é aceita por AFND, então L satisfaz a propriedade acima.
- () A linguagem formada de 1's e 0's com igual quantidade de ocorrências das palavras 01 e 10 satisfaz a propriedade acima.
- () A propriedade acima é falsa para a linguagem $0^i 1^k 2^j | i, j, k \geq 0$ e se $i = 1$, então $k = j$.
- () A linguagem $\{a^n b^n c^n | n \geq 0\}$ não satisfaz a propriedade acima.
- () A linguagem $\{a^n b^m | n, m \geq 0 \text{ e } n \neq m\}$ satisfaz a propriedade acima.

Assinale a alternativa que contém, de cima para baixo, a sequência correta.

- a) V, V, V, V, F.
- b) V, V, F, V, F.
- c) V, F, V, F, F.
- d) F, V, V, F, V.
- e) F, V, F, V, V.

[Voltar para o índice](#)

(2011 - Fundamentos - 36) Sejam as linguagens

$L_1 = a^i b^n c^m | i, n, m \geq 0$ e $L_2 = a^n b^m c^i d^k | i, n, k, m \geq 0$, com $i = m$ ou $n = m$. Com base nessa informação, é correto afirmar:

- a) $L_1 \cap L_2$ é aceita por autômato finito não determinístico.
- b) $L_1.L_2$, isto é, a concatenação das linguagens L_1 e L_2 não é livre de contexto.
- c) L_2 é aceita por autômato de pilha determinístico.
- d) $L_1 \cup L_2$ é aceita por autômato finito possuindo, no mínimo, 6 estados.
- e) $L_1 \cap L_2$ possui gramática livre de contexto geradora.

(2011 - Fundamentos - 38) Com relação às linguagens e seus aceptadores, considere as afirmativas a seguir.

- I. $\{ww^{rev} \mid w \in \{a, b\}^*\}$ é aceita por autômato de pilha determinístico.
- II. $\{wcw^{rev} \mid w \in \{a, b\}^*\}$ é aceita por autômato finito não determinístico.
- III. $\{a, b\}^* - \{ww \mid w \in \{a, b\}^*\}$ é aceita por autômato de pilha não determinístico.
- IV. $\{M \mid M \text{ é M.T. e } M \text{ pára}\}$ é aceita por Máquina de Turing não determinística.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas II e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.
- e) Somente as afirmativas I, III e IV são corretas.

(2011 - Fundamentos - 52) Considere, a seguir, a gramática livre de contexto:

$$S \rightarrow aS|Sb|c$$

Qual expressão regular gera a mesma linguagem que a gramática definida acima?

- a) a^*cb^*
- b) a^+b^+c
- c) a^+cb^+
- d) ca^*b^*
- e) ca^+b^+

[Voltar para o índice](#)

(2011 - Fundamentos - 52) Considere, a seguir, a gramática livre de contexto:

$$S \rightarrow aS | Sb | c$$

Qual expressão regular gera a mesma linguagem que a gramática definida acima?

- a) a^*cb^*
- b) a^+b^+c
- c) a^+cb^+
- d) ca^*b^*
- e) ca^+b^+

[Voltar para o índice](#)

(2011 - Fundamentos 56) Sobre análise sintática, considere as afirmativas a seguir.

- I. Um analisador sintático descendente recursivo pode apenas ser utilizado para reconhecer gramáticas em que o primeiro símbolo terminal de cada subexpressão fornece informações suficientes para a escolha da produção a ser utilizada.
- II. Não é possível construir um analisador sintático descendente recursivo para reconhecer a gramática: $S \rightarrow Sa|a$.
- III. De forma geral, os analisadores sintáticos descendentes são capazes de reconhecer um número maior de gramáticas do que os analisadores sintáticos ascendentes.
- IV. Os analisadores sintáticos ascendentes fazem uso de pilha e um autômato finito para auxiliar na validação da sintaxe de um programa.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas I e III são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e IV são corretas.
- e) Somente as afirmativas II, III e IV são corretas.

[Voltar para o índice](#)

(2012 - Fundamentos 40) Sobre gramáticas e linguagens, considere as afirmativas a seguir.

- I Uma gramática na Forma Normal de Chomsky pode ser ambígua.
- II Uma gramática ambígua pode gerar uma linguagem inerentemente não ambígua.
- III Uma gramática na Forma Normal de Greibach pode ser convertida para a Forma Normal de Chomsky.
- IV O algoritmo de conversão de Gramática Livre de Contexto para Gramática na Forma Normal de Chomsky pode ser diretamente aplicado a uma gramática que não seja λ -livre.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas I e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.
- e) Somente as afirmativas II, III e IV são corretas.

[Voltar para o índice](#)

(2012 - Fundamentos 41) Seja um Autômato Finito Não Determinístico (AFN) com 6 estados. Aplicando-se o algoritmo de conversão de um AFN para um Autômato Finito Determinístico (AFD), em quantos estados, no máximo, resultaria o AFD considerando-se os estados inúteis?

- a) 12
- b) 36
- c) 64
- d) 1024
- e) 46656

(2012 - Fundamentos 42) Assinale a alternativa que apresenta, corretamente, uma expressão regular que denota todas as strings de a's e b's que têm pelo menos dois b's consecutivos.

a) $(a^* + bb)(a + ba)^*(a + b)^*$

b) $(a + ba)^*bb(ba + a)^*$

c) $(a + b)^*ba^*b(a + b)^*$

d) $(a + bb)^*(bb + a)^*$

e) $(a + ba)^*bb(a + b)^*$

[Voltar para o índice](#)

(2012 - Tecnologia 65) Considere a gramática das expressões a seguir.

$$S \rightarrow E\$$$

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow \text{id}$$

$$F \rightarrow (E)$$

Sobre essa gramática, considere as afirmativas a seguir.

- I. A gramática é LL(1).
- II. O operador + possui uma precedência maior que o operador *.
- III. Não é possível construir um analisador descendente recursivo para a gramática.
- IV. Os terminais + *) \$ pertencem ao conjunto FOLLOW de F.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas I e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.
- e) Somente as afirmativas II, III e IV são corretas.

[Voltar para o índice](#)

(2012 - Tecnologia 68) Considere o autômato a seguir.

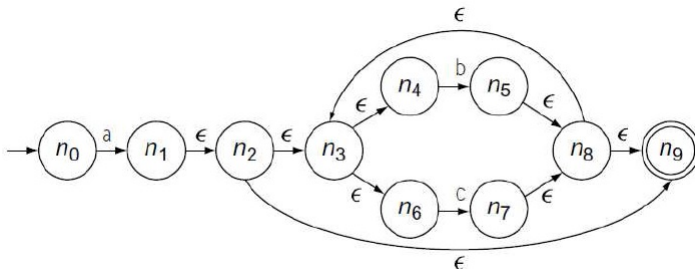


Figura: (COOPER, K.; TORCZON, L. Engineering a Compiler. 2nd Edition. San Francisco: Morgan Kaufmann Publishers, 2012. p.51.)

Assinale a alternativa que apresenta a expressão regular que gera a mesma linguagem reconhecida pelo autômato.

- a. $(ab)c^*$
- b. $(a|b)c^*$
- c. $a(b|c)^*$
- d. $a(bc)^*$
- e. $a(b)^*c$

[Voltar para o índice](#)

(2012 - Tecnologia 70) Considere a gramática a seguir.

$$S \rightarrow E\$$$

$$E \rightarrow T + E$$

$$E \rightarrow T$$

$$T \rightarrow x$$

Com relação a essa gramática, atribua V (verdadeiro) ou F (falso) às afirmativas a seguir.

☐ A gramática é LR(0).

☐ Em uma tabela de análise SLR, a produção $T \rightarrow x$ terá reduções somente nos terminais $+$ e $\$$.

☐ A gramática é SLR.

☐ Em uma tabela de análise LR(0), a produção $E \rightarrow T$ terá reduções somente nos terminais x e $+$.

☐ A gramática é LR(1).

Assinale a alternativa que contém, de cima para baixo, a sequência correta.

- a) V, V, F, F, V.
- b) V, F, V, F, F.
- c) V, F, F, V, F.
- d) F, V, V, F, V.
- e) F, V, F, V, F.

[Voltar para o índice](#)

(2013 - Fundamentos 38) Sobre o Lema do Bombeamento (pumping lemma) para linguagens regulares, considere as afirmativas a seguir.

- I. Se o alfabeto $\Sigma = \{a, b\}$, então pode-se provar por absurdo, por meio do Bombeamento, que a linguagem $L_1 = \{w \in \Sigma^* \mid w \text{ termina com } b\}$ não é regular.
- II. Se o alfabeto $\Sigma = \{a, b\}$, então pode-se provar por absurdo, por meio do Bombeamento, que a linguagem $L_2 = \{(a^n)^2 \mid n \geq 1\}$ não é regular.
- III. Se o alfabeto $\Sigma = \{a, b\}$, então pode-se provar por absurdo, por meio do Bombeamento, que as linguagens $L_3 = \{a^n \mid n \geq 1\}$, $L_4 = \{a^n b a^m b a^{n+m} \mid n, m \geq 1\}$ e $L_5 = \{a^{m+1} b^{n+1} \mid 2 \leq n \leq m \leq 3n\}$ não são regulares.
- IV. Se a linguagem for do tipo 3, então aplica-se o Bombeamento.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas I e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.
- e) Somente as afirmativas II, III e IV são corretas.

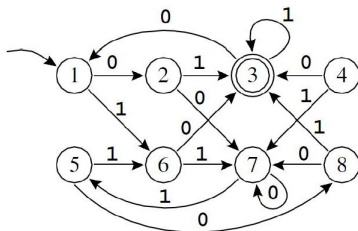
[Voltar para o índice](#)

(2013 - Fundamentos 41) Se o estado inicial for também estado final em um autômato finito, então esse autômato

- a) não aceita a cadeia vazia.
- b) não tem outros estados finais.
- c) é determinístico.
- d) aceita a cadeia vazia.
- e) é não determinístico.

[Voltar para o índice](#)

(2013 - Tecnologia 66) Considere o autômato a seguir.



Sobre esse autômato, considere as afirmativas a seguir.

- I. Os estados 3 e 7 são equivalentes.
- II. Os estados 4 e 6 são equivalentes.
- III. Os estados 1 e 5 são equivalentes.
- IV. Os estados 2 e 8 são equivalentes.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas I e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.
- e) Somente as afirmativas II, III e IV são corretas.

[Voltar para o índice](#)

(2013 - Tecnologia 68) Considere a gramática a seguir.

$$E \rightarrow \text{num}$$

$$E \rightarrow E + E$$

$$E \rightarrow E - E$$

Sobre essa gramática, atribua V (verdadeiro) ou F (falso) às afirmativas a seguir.

☐ É ambígua.

☐ É LL(1).

☐ É LR(1).

☐ É SLR.

☐ Possui recursão à esquerda.

Assinale a alternativa que contém, de cima para baixo, a sequência correta.

- a) V, V, F, F, V.
- b) V, F, V, V, F.
- c) V, F, F, F, V.
- d) F, V, V, F, F.
- e) F, V, F, V, F.

[Voltar para o índice](#)

(2013 - Tecnologia 70) Considere a gramática a seguir.

$$A \rightarrow a$$

$$A \rightarrow C B A$$

$$B \rightarrow \epsilon$$

$$B \rightarrow b$$

$$C \rightarrow B$$

$$C \rightarrow c$$

Sobre essa gramática, assinale a alternativa correta.

- a) O conjunto FIRST de B é $\{b, c\}$.
- b) O conjunto FIRST de C é $\{a, b\}$.
- c) O conjunto FOLLOW de A é vazio.
- d) O conjunto FOLLOW de C é vazio.
- e) O conjunto FOLLOW de B é $\{a, b\}$.

(2014 - Fundamentos 39) Observe a gramática a seguir.

$$S \rightarrow aAbba$$

$$aAb \rightarrow aabbbA \mid ab$$

$$bAb \rightarrow bbA$$

$$bAa \rightarrow Bbaa$$

$$bB \rightarrow Bb$$

$$aB \rightarrow aA$$

Sobre essa gramática, assinale a alternativa correta.

- a) É irrestrita e aceita a linguagem $\{a^n b^{2n+1} a^n \mid n \geq 1\}$.
- b) É irrestrita e aceita a linguagem $\{a^n b^{2n} a^n \mid n \geq 1\}$.
- c) É sensível ao contexto e aceita a linguagem $\{a^n b^{2n+1} a^n \mid n \geq 1\}$.
- d) É sensível ao contexto e aceita a linguagem $\{a^n b^{2n} a^n \mid n \geq 1\}$.
- e) É livre de contexto e aceita a linguagem $\{a^n b^{2n+1} a^n \mid n \geq 1\}$.

(2014 - Fundamentos 40) Sobre o lema do bombeamento (pumping lemma) para linguagens regulares, considere as afirmativas a seguir.

- I. Seja o alfabeto $P = \{a, b\}$. Pode-se provar por absurdo, através do bombeamento, que a linguagem $L_1 = \{w \in \Sigma^* \mid w \text{ termina com } b\}$ não é regular.
- II. Seja o alfabeto $P = \{a, b\}$. Pode-se provar por absurdo, através do bombeamento, que a linguagem $L_2 = \{(a^n)^2 \mid n \geq 1\}$ não é regular.
- III. Seja o alfabeto $P = \{a, b\}$. Pode-se provar por absurdo, através do bombeamento, que as linguagens
 $L_3 = \{a^n \mid n \geq 1\}$,
 $L_4 = \{a^n b a^m b a^{n+m} \mid n, m \geq 1\}$ e
 $L_5 = \{a^{m+1} b^{n+1} \mid 2 \leq n \leq m \leq 3n\}$
não são regulares.

IV. Se a linguagem for do tipo 3, pode-se aplicar o bombeamento.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas I e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.
- e) Somente as afirmativas II, III e IV são corretas.

[Voltar para o índice](#)

(2014 - Fundamentos 41) Sobre as linguagens regulares, considere as afirmativas a seguir.

- I. As linguagens regulares podem ser expressas por máquinas de Moore e de Mealy.
- II. As linguagens regulares podem ser expressas por um autômato finito.
- III. Se A e B são linguagens regulares, então $A \cap B$ também é.
- IV. Seja $B = \{ba, na\}$. Pode-se dizer que $B^* = \{\lambda, ba, na, ab, an, baba, bana, naba, anab, nana, aban, bababa, babana, banaba, banana, nababa, nabana, nanaba, nanana, abanba, babababa, \dots\}$.

Assinale a alternativa correta.

- a) Somente as afirmativas I e II são corretas.
- b) Somente as afirmativas I e IV são corretas.
- c) Somente as afirmativas III e IV são corretas.
- d) Somente as afirmativas I, II e III são corretas.
- e) Somente as afirmativas II, III e IV são corretas.

[Voltar para o índice](#)

(2014 - Técnicas 68) Considere a expressão regular a seguir.
 $(c^*a[abc]^*b[abc]^*)|c^*$ Assinale a alternativa que descreve, corretamente, todas as cadeias geradas por essa expressão regular.

- a) Cadeias sobre o alfabeto $\{a, b, c\}$ onde o primeiro a precede o primeiro b.
- b) Cadeias sobre o alfabeto $\{a, b, c\}$ com um número par de a's.
- c) Cadeias sobre o alfabeto $\{a, b, c\}$ contendo a substring baa.
- d) Cadeias sobre o alfabeto $\{a, b, c\}$ contendo um número ímpar de c's.
- e) Cadeias sobre o alfabeto $\{a, b, c\}$ terminadas por c.

(2015 - Fundamentos 39) A gramática $G=(\{S,A,B\},\{0,1\},P,S)$, onde P é dado pelas regras de produção.

$$S \rightarrow 0AB \mid 1BA$$

$$A \rightarrow 0AS \mid 1A \mid \epsilon$$

$$B \rightarrow 0B \mid 1BS \mid \epsilon$$

gera uma linguagem que

- (A) pertence à classe regular.
- (B) contém a cadeia vazia ϵ .
- (C) pode ser aceita por um autômato com pilha.
- (D) pode ser denotada por uma expressão regular.
- (E) é igual ao conjunto de cadeias $\{x \in \{0,1\}^* \mid x \text{ tem quantidade igual de zero (0) e de um (1)}\}$

(2015 - Fundamentos 40) Considerando as linguagens $L = \{0^n 1^n 2^i \mid n \geq 0 \text{ e } i \geq 0\}$ e $M = \{0^i 1^n 2^n \mid n \geq 0 \text{ e } i \geq 0\}$, pode-se afirmar que

- (A) a linguagem $L \cup M$ pode ser gerada por uma gramática livre de contexto.
- (B) a linguagem M pode ser gerada por uma gramática regular.
- (C) a linguagem L pode ser aceita por um autômato finito determinístico.
- (D) a linguagem $L \cap M$ pertence à classe das linguagens livres de contexto.
- (E) a linguagem M pode ser denotada por uma expressão regular.

(2015 - Técnicas 68) Qual é a classe de método de análise sintática determinístico, ascendente, que processa a sequência de símbolos da esquerda para a direita?

- (A) LL
- (B) LR
- (C) Árvore de derivação anotada
- (D) GAD
- (E) Árvore associativa

[Voltar para o índice](#)

(2016 - Fundamentos 39) O grafo rotulado $G(r)$, exposto abaixo, representa qual expressão regular?

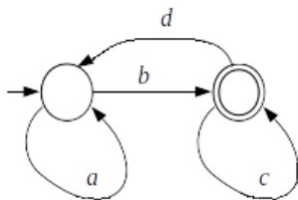
A) $r = ab^*(da^* + cb)^*$

B) $r = a^*b(d^* + cb)$

C) $r = (bb + d)^*(aa + c)^*$

D) $r = a^*b(c + da^*b)^*$

E) $r = a^*c^*(b + d)^*$



(2016 - Fundamentos 40) A linguagem $L = \{a^n b^m \mid n \leq m + 3\}$ para $n \geq 0$ e $m \geq 0$, é:

- A) Regular e gerada pela gramática $S \rightarrow aA, A \rightarrow baA \mid \epsilon$.
- B) Sensível ao contexto e gerada pela gramática $S \rightarrow aSBC, S \rightarrow aBC, CB \rightarrow BC, aB \rightarrow ab, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc$.
- C) Recursivamente enumerável e gerada por uma gramática sem restrições nas regras de produção.
- D) Estrutura de frase e gerada por uma gramática sem restrições nas regras de produção.
- E) Livre de contexto e gerada pela gramática $S \rightarrow aaaA, A \rightarrow aAb \mid B, B \rightarrow Bb \mid \epsilon$.

(2017 - Fundamentos 39) Analise as seguintes assertivas sobre autômatos e linguagens:

- I) Autômatos finitos determinísticos e autômatos finitos não determinísticos aceitam o mesmo conjunto de linguagens.
- II) Seja L uma linguagem livre de contexto, existe um autômato com duas pilhas determinístico que reconhece L.
- III) Toda linguagem enumerável recursivamente é também uma linguagem recursiva.

Quais estão corretas?

- A) Apenas I.
- B) Apenas II.
- C) Apenas I e II.
- D) Apenas I e III.
- E) Apenas II e III.

(2017 - Fundamentos 68) A tarefa principal de um analisador léxico consiste em ler os caracteres da entrada do programa-fonte, agrupá-los em lexemas e gerar uma sequência de tokens que será enviada ao analisador sintático. Sobre o analisador léxico, analise as assertivas abaixo:

- I) Além da identificação de lexemas, outras tarefas podem ser realizadas por esse analisador, tais como: remoção de comentários e espaços em branco e a associação de mensagens de erros às linhas do programa-fonte.
- II) Token é a unidade básica do texto-fonte. Pode ser representado por três informações: a classe do token, que representa o tipo do token reconhecido, o valor do token, que é o texto do lexema reconhecido e a posição que indica o local do texto-fonte (linha e coluna) onde ocorreu o token.
- III) Expressões regulares e geradores de analisadores léxicos são notações utilizadas para especificar os padrões de lexemas.
- IV) Na análise léxica, uma representação intermediária do tipo árvore é criada. Esta apresenta a estrutura gramatical da sequência de tokens.

Quais estão corretas?

- A) Apenas I.
- B) Apenas II.
- C) Apenas IV.
- D) Apenas I e II.
- E) Apenas III e IV

(2018 - Fundamentos 39) Considere os seguintes formalismos:

- I) Autômatos finitos.
- II) Autômatos finitos com uma pilha.
- III) Autômatos finitos com duas pilhas.

Quais contêm SOMENTE os formalismos nos quais a variante não determinística reconhece o mesmo conjunto de linguagens que a respectiva versão determinística?

- A) Apenas I.
- B) Apenas II.
- C) Apenas I e III.
- D) Apenas II e III.
- E) I, II e III.

(2018 - Fundamentos 40) Considere a gramática G descrita a seguir: conjunto de terminais $\{a, c\}$, conjunto de não terminais $\{S, A\}$, símbolo inicial S e contendo as produções abaixo:

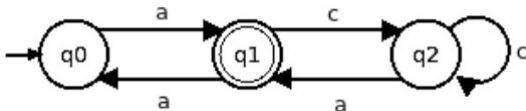
$$S \rightarrow AcS$$

$$S \rightarrow A$$

$$A \rightarrow aAa$$

$$A \rightarrow a$$

Considere também o autômato finito A sobre o alfabeto $\{a, c\}$, com conjunto de estados $\{q_0, q_1, q_2\}$ - dos quais q_0 é inicial e q_1 é final - e com função de transição de estados determinada pelo seguinte grafo:



Seja $L(G)$ a linguagem gerada pela gramática G e $L(A)$ a linguagem reconhecida pelo autômato A , assinale a alternativa correta.

- A) $L(G)$ é regular e $L(A)$ é subconjunto próprio de $L(G)$.
- B) $L(G)$ não é regular e $L(A)$ é subconjunto próprio de $L(G)$.
- C) $L(A) = L(G)$.
- D) $L(G)$ é regular e $L(G)$ é subconjunto próprio de $L(A)$.
- E) $L(G)$ não é regular e $L(G)$ é subconjunto próprio de $L(A)$.

(2018 - Fundamentos 41) Abaixo são apresentadas duas linguagens formais sobre o alfabeto $\{a,b,c\}$:

■ $L1 = \{ a^n b^n \mid n > 1 \}$

■ $L2 = \{ b^n a^p c^q \mid n \geq 0, p \geq 0, q \geq 0 \}$

Considere as seguintes afirmações:

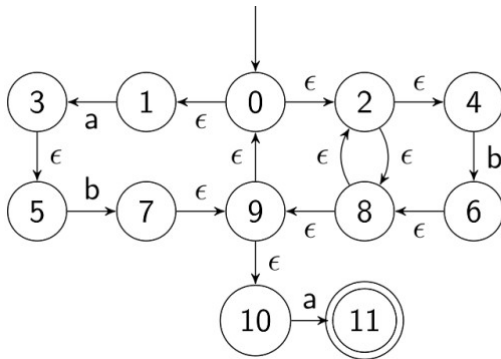
- I) $L1$ é uma linguagem regular.
- II) $L2$ é uma linguagem regular.
- III) $L2$ é uma linguagem livre de contexto.

Quais estão corretas?

- A) Apenas I.
- B) Apenas II.
- C) Apenas I e III.
- D) Apenas II e III.
- E) I, II e III.

(2018 - Fundamentos 63) O Autômato Finito Não Determinista (AFND) abaixo foi construído utilizando o algoritmo de Thompson tomando-se como base uma determinada Expressão Regular (ER). Esse AFND deve ser transformado para um Autômato Finito Determinístico (AFD), utilizando o algoritmo de subconjuntos. Em relação à ER e à conversão AFND para AFD, considere as assertivas abaixo, assinalando V, se verdadeiras, ou F, se falsas.

- ☐ () A ER de origem é " $(ab|b^+)+a$ ".
- ☐ () A ER de origem é " $(ab|b^*)+a$ ".
- ☐ () A ER de origem é " $(ab|b^*)^*a$ ".
- ☐ () O AFD resultante tem 4 estados.
- ☐ () O AFD resultante tem 5 estados.



A ordem correta de preenchimento dos parênteses, de cima para baixo, é:

- A) V ? F ? F ? F ? V.
- B) F ? V ? F ? V ? F.
- C) F ? V ? F ? F ? V.
- D) F ? F ? V ? F ? V.
- E) V ? F ? V ? V ? F.

(2018 - Fundamentos 68) - A fase de análise sintática de um compilador pode ser implementada através de reconhecedores (parsers) de gramáticas livres de contexto, com estratégias ascendentes (bottomup) ou descendentes (top-down). Considere a gramática com cinco produções abaixo, sendo que são não terminais os símbolos S e A , sendo o primeiro o símbolo não terminal inicial da gramática, e, os demais, símbolos terminais:

- $S \rightarrow Aa$
- $S \rightarrow bAc$
- $S \rightarrow bc$
- $S \rightarrow bda$
- $A \rightarrow d$

Analise as seguintes assertivas:

- I) A gramática é reconhecida por um analisador preditivo LL(1), pois as características da gramática não inibem a construção da tabela de reconhecimento.
- II) Esta gramática não é reconhecida por um analisador LR(0), pois existe um conflito do tipo empilha-reduz no estado que contém os seguintes itens LR(0) " $S \rightarrow bd \cdot a$ ", e " $A \rightarrow d$ ".
- III) A gramática é reconhecida por um analisador SLR(1), pois ele resolve o conflito do tipo empilha-reduz LR(0).
- IV) A gramática é LR(1)

Quais estão corretas?

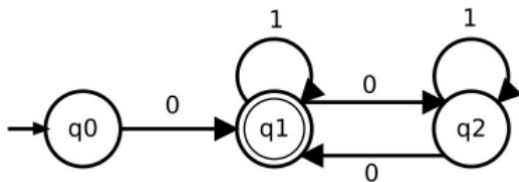
- A) Apenas I.
- B) Apenas II.
- C) Apenas II e III.
- D) Apenas II e IV.
- E) I, II, III e IV.

(2019 - Fundamentos 41) Considere L1 e L2 duas linguagens formais sobre o alfabeto $\Sigma = \{0,1\}$, descritas como segue:

$$L1 = \{ ww \mid w \text{ in } \Sigma^* \}$$

$$L2 = \{ 0^a 1^b \mid a > 0, b > 0, b \text{ ímpar} \}$$

Na descrição acima, justaposição significa concatenação de palavras e Σ^* denota o conjunto de todas as palavras sobre o alfabeto Σ . Seja A1 o autômato finito sobre alfabeto $\Sigma = \{0,1\}$ descrito pelo seguinte diagrama de transição de estados:



Denotemos por $ACEITA(A1)$ o conjunto de palavras aceitas por $A1$.

Nesse sentido, considere as seguintes afirmações:

- I) L1 é uma linguagem regular.
- II) L2 é uma linguagem livre de contexto.
- III) $ACEITA(A1) = \{ w \mid w \in \Sigma^* \text{ e } w \text{ possui um número ímpar de zeros} \}$.

Quais estão corretas?

- A) Apenas I.
- B) Apenas II.
- C) Apenas I e III.
- D) Apenas II e III.
- E) I, II e III.

Provas Antigas - Área 1

Prova de Recuperação de Linguagens Formais - Área 1 - 11/12/13

1)(2 pontos) Remova toda a recursão à esquerda direta e indireta da gramática a seguir:

$$A \rightarrow AaB|B$$

$$B \rightarrow CbC|b$$

$$C \rightarrow Aac|c$$

resposta: substituindo C em B:

$$A \rightarrow AaB|B$$

$$B \rightarrow AacbAac|Aacbc|cbAac|cbc|b$$

substituindo B em A:

$$A \rightarrow AaAacbAac|AaAacbc|AacbAac|Aacbc|Aab|AacbAac|$$

$$A \rightarrow Aacbc|cbAac|cbc|b$$

removendo a recursão à esquerda:

$$A \rightarrow cbAacA' | cbcA' | bA'$$

$$A' \rightarrow AaAacbAacA' | AaAacbcA' | AacbAacA' | AacbcA'$$

$$A' \rightarrow AabA' | AacbAacA' | AacbcA' | \epsilon$$

2) (2.0 pontos) Escreva o AFD mínimo equivalente à gramática a seguir

$$S \rightarrow aA | bB$$

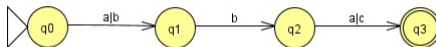
$$A \rightarrow bC$$

$$B \rightarrow bD$$

$$C \rightarrow a | c$$

$$D \rightarrow a | c$$

resposta:



3) (1.5 pontos) Escreva uma GLC que gere a linguagem
 $\{a^n b^{2n} | n \geq 1\} \cup \{a^{2^n} b^n | n \geq 1\}$

■ resposta:

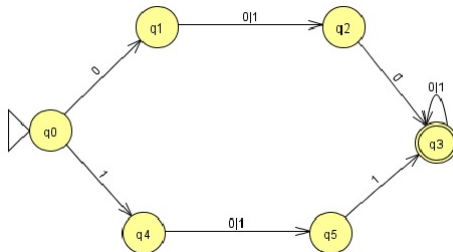
$$S \rightarrow A|B$$

$$A \rightarrow aAbb|abb$$

$$B \rightarrow aaBb|aab$$

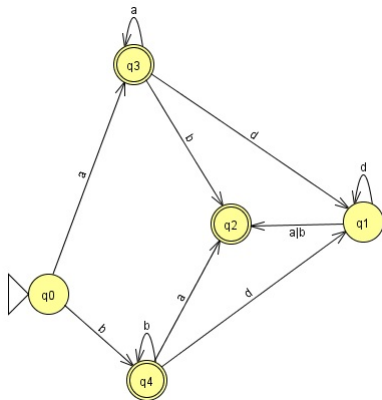
[Voltar para o índice](#)

4) (2.0 pontos) Escreva um AFD que reconheça sentenças em $\{0,1\}^*$ em que o primeiro e o terceiro símbolos são iguais.



[Voltar para o índice](#)

5) (2.0 pontos) Construa um AFD e uma gramática regular equivalente à expressão regular $(a^* \mid b^*)d^*(a|b)$



$S \rightarrow aA \mid bB \mid a \mid b$

$A \rightarrow aA \mid a \mid b \mid dD$

$B \rightarrow bB \mid a \mid b \mid dD$

$D \rightarrow a \mid b \mid dD$

1)(2 pontos) Defina uma CFG que gere sentenças em $\{a, b\}^*$ tal que cada sentença contenha igual número de a's e b's.

resposta:

$$S \rightarrow aB \mid bA \mid \epsilon$$

$$A \rightarrow aS \mid bAA$$

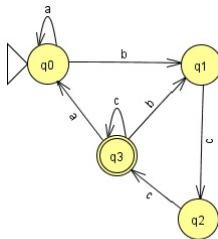
$$B \rightarrow bS \mid aBB$$

2)(3 pontos) Seja uma gramática regular G definida por

$$G = (\{S, B, C\}, \{a, b, c\}, P, S)$$

$$P = \{S \rightarrow aS \mid S \rightarrow bB \mid B \rightarrow cB \mid B \rightarrow cC \mid C \rightarrow cS \mid C \rightarrow c\}$$

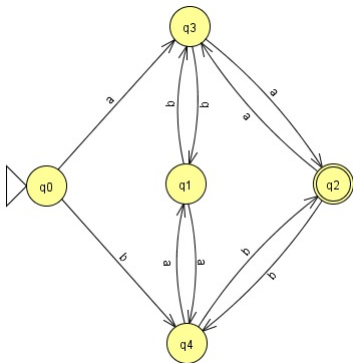
Construa um autômato finito determinístico (2 pontos) e a expressão regular (1 ponto) equivalente.



$S \rightarrow (a|b(ccc*b)ccc*a)b(ccc*b)cc$

[Voltar para o índice](#)

3)(2 pontos) Defina um autômato finito (1 ponto) que reconheça sentenças em $\{a, b\}^+$ que contenham um número par de a 's e um número par de b 's. Escreva uma gramática regular equivalente (1 ponto).



$$S \rightarrow aA | bB$$

$$A \rightarrow aD | a | bC$$

$$B \rightarrow bD | b | aC$$

$$C \rightarrow bA | aB$$

$$D \rightarrow aA | bB$$

- 4)(3 pontos) Considere os seguintes operadores utilizados nas expressões relacionais na linguagem algorítmica utilizada na disciplina de algoritmos.

Operador	Precedência	Associatividade
$\wedge$ (potenciação)	0	$\leftarrow$
$*$ /	1	$\rightarrow$
$+$ -	2	$\rightarrow$
relacionais	3	não associativos

Escreva uma gramática não ambígua que gere expressões relacionais na linguagem de algoritmos incluindo o uso de parênteses e permitindo como elementos da expressão constantes, variáveis ou chamadas de função com argumentos.

$R \rightarrow E > E \mid E < E \mid E = E \mid E < > E \mid E > = E \mid E < = E$

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow K \wedge F \mid K$

$K \rightarrow (E) \mid \text{cte} \mid \text{id} \mid \text{id}(A)$

$A \rightarrow E \mid L$

$L \rightarrow , E L \mid \epsilon$

[Voltar para o índice](#)

1) Encontre uma E.R. equivalente ao autômato a seguir (1.5 pontos):

Estados : $\{q_0, q_1\}$

Estado Inicial : q_0

Estados Finais : q_0 e q_1

Função de transição:

$\delta(q_0, 0) = q_1$

$\delta(q_0, 1) = q_0$

$\delta(q_1, 1) = q_0$

resposta: $(1|01)^*(0|\epsilon)$

2) (1.5 pontos) Apresente uma gramática regular que gere a linguagem formada por todas as sentenças sobre $\{a, b\}$ onde ocorre pelo menos uma vez o string aa .

resposta:

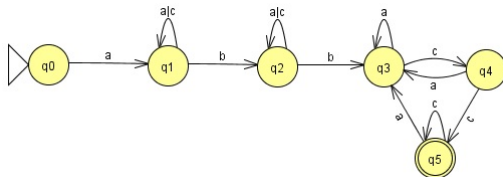
$$S \rightarrow aS | bS | aA$$

$$A \rightarrow aB$$

$$B \rightarrow aB | bB | a | b$$

3) (1.5 pontos) Apresente um NFA sobre $\{a, b\}$ que reconheça sentenças em que cada subpalavra de tamanho 4 tem exatamente UM b , ou seja, entre dois b 's há exatamente 3 a 's. Ex: $aaabaaa$, $baaab$, a , ab , aba , $abaa$, $aaba$, $abaaab$, etc..

4) (1.5 pontos) Apresente um DFA sobre $\{a, b, c\}$ que reconheça sentenças que iniciem por a, tenham exatamente 2 b's e terminem por cc. Ex: acbaacbacc



[Voltar para o índice](#)

5) (2 pontos) Considere a gramática a seguir:

$$S \rightarrow Aa$$

$$A \rightarrow Sb|Ac|d$$

Gere uma gramática equivalente sem recursão direta ou indireta.

resposta:

$$S \rightarrow dA'aS'$$

$$S' \rightarrow bA'aS'|\epsilon$$

$$A' \rightarrow cA'|\epsilon$$

6) Gere o autômato mínimo equivalente ao autômato a seguir(2 pontos):

Estados : $\{q_0, q_1, q_2, q_3, q_4, q_5\}$

Estado Inicial : q_0

Estado Final : q_5

Função de transição:

$\delta(q_0, a) = q_1$ $\delta(q_0, b) = q_3$ $\delta(q_1, a) = q_1$ $\delta(q_1, b) = q_2$

$\delta(q_2, a) = q_2$ $\delta(q_2, b) = q_5$ $\delta(q_3, a) = q_3$ $\delta(q_3, b) = q_4$

$\delta(q_4, a) = q_4$ $\delta(q_4, b) = q_5$ $\delta(q_5, a) = q_5$ $\delta(q_5, b) = q_5$

[Voltar para o índice](#)

1) (2 pontos) Apresente uma gramática regular que gere a linguagem formada por todas as sentenças sobre $\{0, 1\}^*$ onde não ocorre o string 00.

resposta:

$$S \rightarrow 1S|\epsilon|0F|0$$

$$F \rightarrow 1S$$

2) (2 pontos) Escreva uma gramática G livre de contexto que gere a linguagem $L(G) = \{0^m 1^n 2^o\}$, $m \geq 0$, $o \geq 0$, $n = m + 0$.

resposta:

$$S \rightarrow AB$$

$$A \rightarrow 0A1|\epsilon$$

$$B \rightarrow 1B2|\epsilon$$

4) (2 pontos) Apresente um DFA sobre $\{0, 1, 2\}$ que reconheça sentenças que iniciem por 0 e tenham exatamente dois dígitos 1 e dois dígitos 2.

5) (2 pontos) Escreva um AFD, Expressão Regular e Gramática Regular que reconheça sentenças em $\{0, 1\}^+$ em que o último símbolo da sentença já ocorreu antes na mesma.

[Voltar para o índice](#)

Primeira Prova de Linguagens Formais - 23/05/02

1) (2.0 pontos) Escreva uma gramática G livre de contexto que gere a linguagem $L(G) = \{0^n 1^m\}$, $m > n > 0$

resposta:

$$S \rightarrow AB$$

$$A \rightarrow 0A1 \mid 01$$

$$B \rightarrow 1B \mid 1$$

2) (2.0 pontos) Escreva uma gramática regular equivalente à expressão regular

$$(a|b)^*(c|d)^* eef^*(h|i)^*$$

resposta: aplicando a regra do fecho

$$S \rightarrow aS \mid bS \mid A$$

$$A \rightarrow cA \mid dA \mid eB$$

$$B \rightarrow eC$$

$$C \rightarrow f^*(h|i)^*$$

aplicando a regra do fecho

$$S \rightarrow aS \mid bS \mid A$$

$$A \rightarrow cA \mid dA \mid eB$$

$$B \rightarrow eC$$

$$C \rightarrow fC \mid D$$

$$D \rightarrow hD \mid iD \mid \epsilon$$

[Voltar para o índice](#)

3) Encontre uma E.R. equivalente ao autômato a seguir (2.0 pontos):

Vocabulário: $\{a, b\}$

Estado inicial : q_0

Conjunto de estados : $\{q_0, q_1, q_2\}$

Conjunto de estados finais : $\{q_2\}$

Função de Transição:

$$d(q_0, a) = q_1 \quad d(q_0, b) = q_0$$

$$d(q_1, a) = q_1 \quad d(q_1, b) = q_2$$

$$d(q_2, a) = q_1 \quad d(q_2, b) = q_0$$

resposta:

$$S \rightarrow (b|a(a|ba)^*bb)^*a(a|ba)^*b$$

5) (2.0 pontos) Apresente um DFA sobre $\{a, b\}$ que reconheça sentenças em que o número de a's mais o dobro do número de b's é múltiplo de 3. P.ex. a sentença abaaa deve ser reconhecida, pois o número de a's (4) mais o dobro do número de b's (2) é múltiplo de 3.

[Voltar para o índice](#)

6) (2 pontos) Gere o autômato mínimo equivalente ao autômato a seguir(2 pontos):

Estados : $\{q_0, q_1, q_2, q_3, q_4, q_5\}$

Estado Inicial : q_0

Estado Final : q_5

Função de transição:

$$\begin{array}{llll} \delta(q_0, a) = q_1 & \delta(q_0, b) = q_3 & \delta(q_1, a) = q_1 & \delta(q_1, b) = q_2 \\ \delta(q_2, a) = q_2 & \delta(q_2, b) = q_5 & \delta(q_3, a) = q_3 & \delta(q_3, b) = q_4 \\ \delta(q_4, a) = q_4 & \delta(q_4, b) = q_5 & \delta(q_5, a) = q_5 & \delta(q_5, b) = q_5 \end{array}$$

1)(2 pontos) Remova toda a recursão direta e indireta da gramática a seguir:

$$A \rightarrow BC \mid a$$

$$B \rightarrow CA$$

$$C \rightarrow AB \mid CC \mid a$$

■ substituindo B em A e C

$$A \rightarrow CAC \mid a$$

$$C \rightarrow ACA \mid CC \mid a$$

■ removendo recursão em C

$$A \rightarrow CAC \mid a$$

$$C \rightarrow ACAC' \mid aC'$$

$$C' \rightarrow CC' \mid \epsilon$$

■ substituindo C em A

$$A \rightarrow ACAC'AC \mid aC'AC \mid a$$

$$C \rightarrow ACAC' \mid aC'$$

$$C' \rightarrow CC' \mid \epsilon$$

■ removendo recursão em A

$$A \rightarrow aC'ACA' \mid aA'$$

$$A' \rightarrow CAC'ACA' \mid \epsilon$$

$$C \rightarrow ACAC' \mid aC'$$

$$C' \rightarrow CC' \mid \epsilon$$

2) (1.5 pontos) Escreva o AFD mínimo equivalente à gramática a seguir

$$S \rightarrow 0S \mid 1S \mid 0A \mid 0C \mid 1B$$

$$A \rightarrow 0A \mid 0$$

$$B \rightarrow 1B \mid 1$$

$$C \rightarrow 0A \mid 0$$

- 3) (1.5 pontos) Escreva uma GLC que gere a linguagem $\{a^n b^n c | n \geq 1\} \cup \{a^n b^{2n} d | n \geq 1\}$

- resposta:

$$S \rightarrow Ec \mid Fd$$

$$E \rightarrow aEb \mid ab$$

$$F \rightarrow aFbb \mid abb$$

- 5) (1.5 pontos) Construa um AFD e uma gramática regular equivalente à expressão regular $a(b|ca)d^*(a|b)^+$

$$S \rightarrow aA$$

$$A \rightarrow (b|ca)d^*(a|b)^+$$

- dividindo o primeiro termo de A

$$S \rightarrow aA$$

$$A \rightarrow bB$$

$$A \rightarrow caB$$

$$B \rightarrow d^*(a|b)^+$$

■ separando caB e aplicando regra do fecho

$$S \rightarrow aA$$

$$A \rightarrow bB \mid cC$$

$$C \rightarrow aB$$

$$B \rightarrow dB$$

$$B \rightarrow (a|b)^+$$

■ "quebrando" o fecho positivo

$$S \rightarrow aA$$

$$A \rightarrow bB \mid cC$$

$$C \rightarrow aB$$

$$B \rightarrow dB$$

$$B \rightarrow (a|b)(a|b)^*$$

■ separando $a|b$

$$S \rightarrow aA$$

$$A \rightarrow bB \mid cC$$

$$C \rightarrow aB$$

$$B \rightarrow dB$$

$$B \rightarrow aD \mid bD$$

$$D \rightarrow (a|b)^*$$

■ aplicando a regra do fecho

$$S \rightarrow aA$$

$$A \rightarrow bB \mid cC$$

$$C \rightarrow aB$$

$$B \rightarrow dB \mid aD \mid bD$$

$$D \rightarrow (a|b)D \mid \epsilon$$

■ separando a | b

$$S \rightarrow aA$$

$$A \rightarrow bB \mid cC$$

$$C \rightarrow aB$$

$$B \rightarrow dB \mid aD \mid bD$$

$$D \rightarrow aD \mid bD \mid \epsilon$$

[Voltar para o índice](#)

- 3) Encontre uma E.R. equivalente ao autômato a seguir (2.0 pontos):

- Vocabulário: $\{a, b\}$
- Estado inicial : q_0
- Conjunto de estados : $\{q_0, q_1, q_2\}$
- Conjunto de estados finais : $\{q_2\}$
- Função de Transição:
 $d(q_0, a) = q_1$ $d(q_0, b) = q_0$
 $d(q_1, a) = q_1$ $d(q_1, b) = q_2$
 $d(q_2, b) = q_0$

resposta: $(b|aa^*bb)^*aa^*b$

4) (2.0 pontos) Escreva o AFD mínimo equivalente à gramática a seguir

$$S \rightarrow 0S \mid 1S \mid 0A \mid 0C \mid 1B$$

$$A \rightarrow 0A \mid 0$$

$$B \rightarrow 1B \mid 1$$

$$C \rightarrow 0A \mid 0$$

[Voltar para o índice](#)

5) (2.0 pontos) Apresente um DFA sobre $\{a,b\}$ que reconheça sentenças em que o número de a's mais o dobro do número de b's é múltiplo de 3. P.ex. a sentença abaaa deve ser reconhecida, pois o número de a's (4) mais o dobro do número de b's (2) é múltiplo de 3. Considere que a sentença vazia não deve ser reconhecida (consideramos para esse problema que 0 não é múltiplo de 3).

[Voltar para o índice](#)

Primeira Prova - Parte 2 - 16/10/12

- 2) (1.5 pontos) Escreva o AFD mínimo equivalente à gramática a seguir

$$S \rightarrow 0S \mid 1S \mid 0A \mid 0C \mid 1B$$

$$A \rightarrow 0A \mid 0$$

$$B \rightarrow 1B \mid 1$$

$$C \rightarrow 0A \mid 0$$

[Voltar para o índice](#)

- 3) (1 ponto) Considere a gramática a seguir:

$$S \rightarrow Aa$$

$$A \rightarrow Sb \mid Ac \mid d$$

Gere uma gramática equivalente sem recursão direta e indireta.

retirando a recursão de A

$$S \rightarrow Aa$$

$$A \rightarrow SbA' \mid dA'$$

$$A' \rightarrow cA' \mid \epsilon$$

substituindo A em S

$$S \rightarrow SbA'a \mid dA'a$$

$$A' \rightarrow cA' \mid \epsilon$$

removendo a recursão de S

$$S \rightarrow dA'aS'$$

$$S' \rightarrow bA'aS' \mid \epsilon$$

$$A' \rightarrow cA' \mid \epsilon$$

Primeira Prova - 18/10/12

- 2)(1 ponto) Construa um automato finito determinístico que reconheça a linguagem $a^*b|ab^*$
- 3)(1 ponto) Obtenha o autômato mínimo equivalente ao autômato a seguir:
- $(\{q_0, q_1, q_2, q_3\}, \{a, b, c\}, q_0, \{q_0, q_1, q_2, q_3\})$
 - $d(q_0, a) = q_0$ $d(q_0, b) = q_1$ $d(q_0, c) = q_2$
 - $d(q_1, a) = q_0$ $d(q_1, b) = q_3$ $d(q_1, c) = q_2$
 - $d(q_2, a) = q_0$ $d(q_2, b) = q_1$ $d(q_2, c) = q_2$
 - $d(q_3, a) = q_0$ $d(q_3, b) = q_3$ $d(q_3, c) = q_2$

[Voltar para o índice](#)

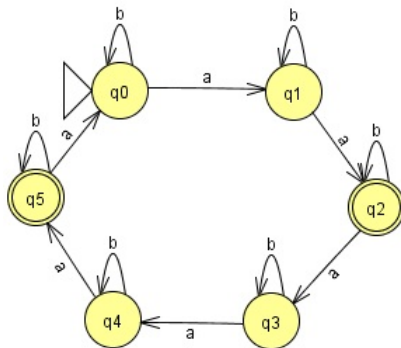
- 4)(1 ponto) Encontre uma E.R. equivalente ao autômato a seguir:

- Vocabulário: $\{a,b\}$
- Estado inicial : q_0
- Conjunto de estados : $\{q_0, q_1, q_2\}$
- Conjunto de estados finais : $\{q_2\}$
- Função de Transição:
- $d(q_0, a) = q_1$ $d(q_0, b) = q_0$
- $d(q_1, a) = q_1$ $d(q_1, b) = q_2$
- $d(q_2, a) = q_1$ $d(q_2, b) = q_2$

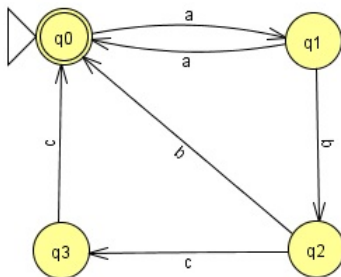
resposta: $b^* a(a|bb^* a)bb^*$

Avaliação Primeira Área - Parte 1 - 06/10/14

2)(1 ponto) Verifique se o autômato abaixo é mínimo e, se não for, encontre o autômato mínimo equivalente.



3)(1.5 pontos) Construa, para o autômato abaixo, a expressão regular equivalente:



resposta: $(aa|abb|abcc)^*$

1)(1 ponto) Considere a gramática abaixo, definida sobre o alfabeto $\{a,b,c,\epsilon,+,*,(,)\}$:

$$S \rightarrow SS \mid S+S \mid S^* \mid (S) \mid a \mid b \mid c$$

Essa gramática gera todas as expressões regulares para o alfabeto $\{a,b,c,\epsilon,+,*,(,)\}$, sendo que nela o símbolo '+' é utilizado no lugar do símbolo $\mid$ para representar a união. Construa uma gramática não ambígua equivalente a ela.

resposta:

$$S \rightarrow S + T \mid T$$

$$T \rightarrow TF \mid F$$

$$F \rightarrow G^* \mid G$$

$$G \rightarrow (S) \mid a \mid b \mid c$$

2)(1 ponto) Elimine a recursão à esquerda (direta e indireta) da gramática a seguir:

$$S \rightarrow A0 \mid 0B1 \mid 11C$$

$$A \rightarrow 01 \mid 0BC \mid S0$$

$$B \rightarrow B01 \mid 01A$$

$$C \rightarrow 01 \mid 10$$

eliminando a recursão em B:

$$S \rightarrow A0 \mid 0B1 \mid 11C$$

$$A \rightarrow 01 \mid 0BC \mid S0$$

$$B \rightarrow 01AB'$$

$$B' \rightarrow 01B' \mid \epsilon$$

$$C \rightarrow 01 \mid 10$$

substituindo A em S:

$$S \rightarrow S00 \mid 010 \mid 0BC0 \mid 0B1 \mid 11C$$

$$A \rightarrow 01 \mid 0BC \mid S0$$

$$B \rightarrow 01AB'$$

$$B' \rightarrow 01B' \mid \epsilon$$

$$C \rightarrow 01 \mid 10$$

removendo a recursão em S:

$$S \rightarrow 010S' \mid 0BC0S' \mid 0B1S' \mid 11CS'$$

$$S' \rightarrow 00S' \mid \epsilon$$

$$A \rightarrow 01 \mid 0BC \mid S0$$

$$B \rightarrow 01AB'$$

$$B' \rightarrow 01B' \mid \epsilon$$

$$C \rightarrow 01 \mid 10$$

3)(1.5 pontos) Construa uma gramática livre de contexto que gere a linguagem $L = \{a^{k+1}bc^{2k}, k \geq 1\}$

resposta: $S \rightarrow aScc \mid aabcc$

[Voltar para o índice](#)

Recuperação da Primeira Área - Parte 1 - 08/12/14

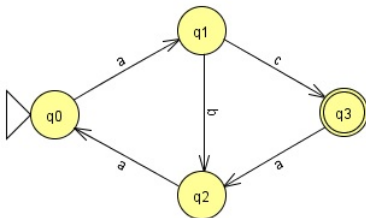
1)(1 ponto) Escreva uma expressão regular sobre o alfabeto $\{a, b, c\}$ que gere a linguagem cujas sentenças contenham no mínimo 1 e no máximo 3 símbolos **a**.

resposta: $(b|c)^*a(b|c)^*(a|\epsilon)(b|c)^*(a|\epsilon)(b|c)^*$

2)(1 ponto) Mostre um autômato finito determinístico sobre o alfabeto $\{a, b\}$ que reconheça as sentenças que terminam em **ab**.

[Voltar para o índice](#)

3)(1.5 pontos) Construa, para o autômato abaixo, a expressão regular equivalente:



resposta: $S \rightarrow (aba|aca)^*ac$

4)(1.5 pontos) Encontre um autômato finito determinístico equivalente à expressão regular $a^*b|b^*a$.

Prova 1 - Parte 2 - 07/10/15

1)(1.5 pontos) Converta a gramática regular a seguir para uma expressão regular:

$$S \rightarrow 1S \mid 2S \mid 0B$$

$$B \rightarrow 0B \mid 1C \mid 2S$$

$$C \rightarrow 0B \mid \epsilon$$

resposta: $S \rightarrow (1 \mid 2 \mid 0(0 \mid 10)^* 2)^* 0(0 \mid 10)^* 1$

3)(1 ponto) Construa uma gramática livre de contexto que gere a linguagem $L = \{a^k b^l, k \geq 1, l \geq k\}$

resposta:

$$S \rightarrow EB$$

$$E \rightarrow aEb \mid ab$$

$$B \rightarrow bB \mid \epsilon$$

4)(1 ponto) Apresente uma gramática regular que gere a linguagem formada por todas as sentenças sobre $\{a, b, c\}^*$ onde não ocorre o string abc .

resposta:

$$\begin{aligned} S &\rightarrow bS \mid cS \mid aA \mid \epsilon \\ A &\rightarrow bB \mid cS \mid aA \mid \epsilon \\ B &\rightarrow bS \mid aA \mid \epsilon \end{aligned}$$

[Voltar para o índice](#)

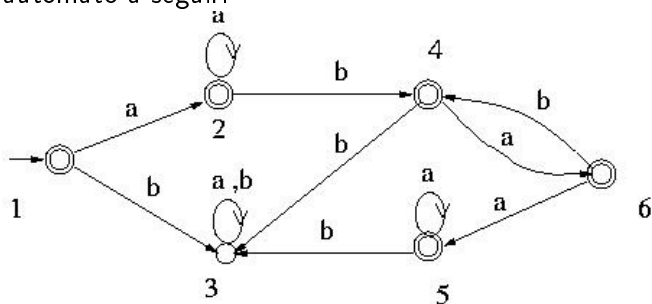
1)(1 ponto) Escreva uma expressão regular sobre o alfabeto $\{a, b, c\}$ que gere a linguagem cujas sentenças não contenham o substring bb .

resposta: $(a|c|ba|bc)^*(b|\epsilon)$

2)(1 ponto) Encontre um autômato finito determinístico que reconheça a linguagem gerada pela expressão $(1|0)^*00$.

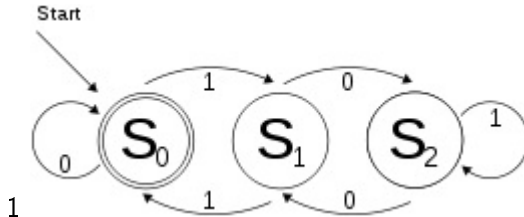
[Voltar para o índice](#)

3)(1.5 pontos) Encontre o autômato mínimo equivalente ao autômato a seguir.



[Voltar para o índice](#)

4)(1.5 pontos) O autômato abaixo reconhece números binários múltiplos de 3. Encontre a expressão regular equivalente:



[Voltar para o índice](#)

1)(2.5 pontos) Escreva um parser descendente recursivo com retrocesso para a seguinte gramática:

$$S \rightarrow aC \mid aD$$

$$C \rightarrow cC \mid c$$

$$D \rightarrow cE$$

$$E \rightarrow dE \mid d$$

2)(2.5 pontos) Redefina a gramática da questão 1 e escreva um parser descendente recursivo preditivo para a gramática da questão 1.

[Voltar para o índice](#)

3)(2.5 pontos) Defina a tabela de precedência de operadores para a gramática a seguir:

$S \rightarrow \text{if } c \text{ then } S \text{ else } S$

$S \rightarrow \text{if } c \text{ then } S$

$S \rightarrow s$

4)(2.5 pontos) Defina as funções de precedência de operadores $f()$ e $g()$ para a tabela de precedência a seguir:
 $\text{orbor} = b > > \ll$

[Voltar para o índice](#)

Segunda Prova de Linguagens Formais - 03/07/97

- 1)(3 pontos) Escreva um analisador tabular preditivo que reconheça a linguagem $a^*b^*c^*$.
- 2)(4 pontos) Escreva uma gramática de precedência de operadores que reconheça a linguagem gerada pela gramática seguinte:

$\text{COM_FOR} \rightarrow \text{for id} := \text{EXP to EXP do COM_COMP}$

$\text{COM_COMP} \rightarrow \text{COMANDO} \mid \text{begin LISTA_COM end}$

$\text{COMANDO} \rightarrow \text{COM_FOR} \mid \text{COM_ATRIB}$

$\text{COM_ATRIB} \rightarrow \text{id} := \text{EXP}$

$\text{EXP} \rightarrow k$

$\text{LISTA_COM} \rightarrow \text{COMANDO} \mid \text{COMANDO} ; \text{LISTA_COM}$

1)(2.5 pontos) Encontre a coleção canônica de conjuntos de itens LR(0) para a gramática a seguir:

$$S \rightarrow a \mid (L)$$

$$L \rightarrow S, L \mid S$$

2)(2.5 pontos) Construa um analisador sintático descendente recursivo preditivo para reconhecer expressões regulares sobre $\{a, b\}$.

3)(2.5 pontos) Construa a tabela de precedência de operadores para reconhecer a linguagem gerada pela gramática a seguir (obs : lembre-se que para isso a gramática deve obedecer certas restrições, portanto...)

$$S \rightarrow aBc$$

$$B \rightarrow bB \mid$$

4)(2.5 pontos) Construa a tabela de precedência de operadores para a gramática a seguir:

Comando $\rightarrow$ while exp do comando | begin lista end | atrib

Lista $\rightarrow$ lista , comando | comando

Segunda Prova de Linguagens Formais - 04/07/01

1)(2 pontos) Escreva um parser descendente recursivo que reconheça a linguagem gerada pela gramática (se necessário altere a gramática):

$$S \rightarrow Sb \mid A$$
$$A \rightarrow Aa \mid a$$

2)(3 pontos) Mostre qual a relação de precedência entre os operadores marcados com x na tabela de precedência de operadores para a gramática a seguir:

$$S \rightarrow id := E$$
$$S \rightarrow \text{repeat } L \text{ until } E$$
$$L \rightarrow S \mid S;L$$
$$E \rightarrow id \mid id (A) \mid (E) \mid id + E$$
$$A \rightarrow E \mid E,A$$

[Voltar para o índice](#)

3) (2.5 pontos) Faça uma rotina de análise léxica equivalente à função prox-token do trabalho, que reconheça tokens com 0's e 1's que contenham um número par de 0's e um número par de 1's, ou um número ímpar de 0's e um número ímpar de 1's. Descreva qualquer outra função utilizada pela rotina.

4)(2.5 pontos) Encontre os conjuntos First e Follow para os não terminais da gramática a seguir:

$$S \rightarrow ABCd \mid \epsilon$$

$$A \rightarrow aA \mid aSBC \mid \epsilon$$

$$B \rightarrow bB \mid eC \mid \epsilon$$

$$C \rightarrow Cc$$

Segunda Prova de Linguagens Formais - 01/07/02

1. (2.5 pontos) Mostre qual a relação de precedência entre os operadores marcados com x na tabela de precedência de operadores para a gramática a seguir:

$$S \rightarrow \text{id} := E$$
$$S \rightarrow \text{repeat } L \text{ until } E$$
$$L \rightarrow S \mid S; L$$
$$E \rightarrow \text{id} \mid \text{id } (A) \mid (E) \mid \text{id} + E$$
$$A \rightarrow E \mid E, A$$

1. (2.5 pontos) Encontre os conjuntos First e Follow para os não terminais da gramática a seguir:

$$S \rightarrow ABCd \mid \epsilon$$
$$A \rightarrow aA \mid aSBC \mid \epsilon$$
$$B \rightarrow bB \mid eC \mid \epsilon$$
$$C \rightarrow Cc \mid \epsilon$$

1. (2.5 pontos) Escreva uma gramática que reconheça declarações de arrays de inteiros de qualquer dimensão em pascal e faça a tabela de precedência de operadores para a mesma. Ex. de sentenças a serem reconhecidas: `Var a,b:array[1..10,2..20] of integer;`
1. (2.5 pontos) Faça um analisador descendente recursivo (preditivo ou com retrocesso) para a gramática a seguir. Se necessário altere a gramática.

$$A \rightarrow Bc \mid De$$
$$B \rightarrow Bb \mid b$$
$$D \rightarrow be \mid f$$

1)(1.0 pontos) Encontre a coleção canônica de conjuntos de itens LR(0) para a gramática a seguir:

$$S \rightarrow aS \mid a$$

2)(1.5 pontos) Construa a tabela de precedência de operadores para reconhecer a linguagem gerada pela gramática a seguir (obs : lembre-se que para isso a gramática deve obedecer certas restrições, portanto...)

$$S \rightarrow aBc$$
$$B \rightarrow bB \mid e$$

[Voltar para o índice](#)

3) (1.0 ponto) Dada a gramática de expressões aritméticas abaixo e a tabela de análise LR, mostre o estado da pilha durante o reconhecimento da sentença $((id))\$$

1) $E \rightarrow E + T$

2) $E \rightarrow T$

3) $T \rightarrow T * F$

4) $T \rightarrow F$

5) $F \rightarrow (E)$

6) $F \rightarrow id$

[Voltar para o índice](#)

	id	+	*	(	)	\$	E	T	F
0	e5			e4			1	2	3
1		e6				AC			
2		r2	e7		r2	r2			
3		r4	r4		r4	r4			
4	e5			e4			8	2	3
5		r6	r6		r6	r6			
6	e5			e4				9	3
7	e5			e4					10
8		e6			e11				
9		r1	e7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

[Voltar para o índice](#)

4)(1.5 pontos) Construa a tabela de precedência e a função de precedência para a gramática

$S \rightarrow aS \mid a$

[Voltar para o índice](#)

11/12/12 - Recuperação da Prova 2 - Parte 2

1)(1.5 pontos) Encontre a coleção canônica de conjuntos de itens LR(0) para a gramática a seguir:

$$S \rightarrow aB \mid a$$

$$B \rightarrow bS \mid b$$

2) (1.5 pontos) Dada a gramática de expressões aritméticas abaixo e a tabela de análise LR, mostre o estado da pilha durante o reconhecimento da sentença $(id+id)*id\$$

$$1) E \rightarrow E + T \quad 4) T \rightarrow F$$

$$2) E \rightarrow T \quad 5) F \rightarrow (E)$$

$$3) T \rightarrow T * F \quad 6) F \rightarrow id$$

[Voltar para o índice](#)

	id	+	*	(	)	\$	E	T	F
0	e5			e4			1	2	3
1		e6				AC			
2		r2	e7		r2	r2			
3		r4	r4		r4	r4			
4	e5			e4			8	2	3
5		r6	r6		r6	r6			
6	e5			e4				9	3
7	e5			e4					10
8		e6			e11				
9		r1	e7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

[Voltar para o índice](#)

3)(2 pontos) Construa a tabela de precedência e a função de precedência para a gramática

$$S \rightarrow aSa \mid bCb$$

$$C \rightarrow cC \mid c$$

[Voltar para o índice](#)

Segunda Prova - Primeira Parte - 29/11/12

1)(1.5 pontos) Escreva um parser descendente recursivo com retrocesso para a seguinte gramática:

$$S \rightarrow a \mid aab$$

2)(2.0 pontos) Escreva um parser descendente recursivo preditivo para a gramática a seguir. Faça as alterações necessárias na gramática.

$$S \rightarrow Cd \mid a$$

$$C \rightarrow Cbd \mid Sbd \mid c$$

[Voltar para o índice](#)

3)(1.5 pontos) Construa a tabela do analisador tabular preditivo para a gramática a seguir:

$$S \rightarrow ASB \mid s$$

$$B \rightarrow e \mid b \mid bA$$

$$A \rightarrow e \mid a \mid aB$$

[Voltar para o índice](#)

Questões de Exames

1) (2001/2 - 3 pontos) Encontre os conjuntos FIRST e FOLLOW para os não terminais da linguagem a seguir e monte a tabela de produções para um analisador tabular preditivo

$$S \rightarrow ABCEf$$

$$A \rightarrow aA \mid aB \mid \epsilon$$

$$B \rightarrow bB \mid gB \mid \epsilon$$

$$C \rightarrow cC \mid h \mid \epsilon$$

$$E \rightarrow eE \mid f$$

[Voltar para o índice](#)

1) (2001/2 - 3 pontos) Mostre qual a relação de precedência entre os operadores marcados com x na tabela de precedência de operadores para a gramática a seguir:

$$S \rightarrow \text{id} := E$$
$$S \rightarrow \text{while } E \text{ do } S$$
$$S \rightarrow \text{begin } L \text{ end}$$
$$L \rightarrow S \mid S; L$$
$$E \rightarrow \text{id} \mid \text{id } (A) \mid (E) \mid \text{id} + E$$
$$A \rightarrow E \mid E, A$$

[Voltar para o índice](#)

1. (99/2 - 2 pontos) Encontre a coleção canônica de conjuntos de itens LR(0) para a gramática a seguir:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow TF \mid F$$

$$F \rightarrow F^* \mid a \mid b$$

[Voltar para o índice](#)

Recuperação da Segunda Área - 10/12/13

- 1)(2 pontos) Encontre os 4 primeiros conjuntos de itens da coleção canônica de conjuntos de itens LR(0) para a gramática a seguir:

$$S \rightarrow SaB \mid aB$$
$$B \rightarrow bB \mid b$$

[Voltar para o índice](#)

- 3) Considere a gramática a seguir para declaração de switches simplificados (com apenas um comando em cada entrada) em C (terminais em minúscula, não-terminais em maiúscula):

COMSWITCH $\rightarrow$ switch (id) { LISTA_C }

LISTA_C $\rightarrow$ C LISTA_C | C

C $\rightarrow$ case cte : COM ;

COM $\rightarrow$ break | id = cte

- a)(2 pontos) Implemente um analisador descendente recursivo preditivo para ela. Faça as alterações que forem necessárias na gramática.
- b)(2 pontos) Implemente um analisador de precedência de operadores para ela. Faça as alterações que forem necessárias na gramática.

- 4) (2 pontos) Considere a gramática a seguir para declaração de funções em C (terminais em minúscula, não-terminais em maiúscula):

$\text{DecFunc} \rightarrow \text{Tipo id (ListaPar) \{ ListaC\}}$

$\text{ListaPar} \rightarrow \text{Tipo id RLista} \mid \epsilon$

$\text{RLista} \rightarrow , \text{id RLista} \mid \epsilon$

$\text{Tipo} \rightarrow \text{float} \mid \text{int}$

$\text{ListaC} \rightarrow \text{Com ListaC} \mid \epsilon$

$\text{Com} \rightarrow \text{return;} \mid \text{id} = \text{cte};$

- Implemente um analisador tabular preditivo para ela. Faça as alterações que forem necessárias na gramática.

Segunda Prova - Parte 1 - 01/12/14

- 1)(1.5 pontos) Escreva um parser descendente recursivo preditivo ou com retrocesso para reconhecer a linguagem $L=\{a^n b^n \mid n>0\}$
- 2)(2.0 pontos) Escreva um parser descendente recursivo preditivo para a gramática a seguir. Faça as alterações necessárias na gramática.
 - $S \rightarrow Cd \mid a$
 - $C \rightarrow Cbd \mid Sbd \mid c$

[Voltar para o índice](#)

- 3)(1.5 pontos) Construa a tabela do analisador tabular preditivo para a linguagem descrita pela gramática a seguir. Faça as alterações necessárias na gramática:

$A \rightarrow id = E$

$E \rightarrow E + E \mid id \mid cte \mid id (E)$

[Voltar para o índice](#)

- 1)(1.5 pontos) Encontre a coleção canônica de conjuntos de itens LR(0) para a gramática a seguir:

$$L \rightarrow aLb \mid \epsilon$$

[Voltar para o índice](#)

4)(1.5 pontos) A partir da gramática e coleção canônica de conjuntos de itens a seguir, construa as tabelas de ação e de transição SLR.

$$I \rightarrow alb|ab$$

$$I_0: \{I' \rightarrow \bullet I$$

$$I \rightarrow \bullet alb$$

$$I \rightarrow \bullet ab\}$$

$$I_1: \{I' \rightarrow I \bullet\}$$

$$I_2: \{I \rightarrow \bullet alb$$

$$I \rightarrow a \bullet Ib$$

$$I \rightarrow a \bullet b$$

$$I \rightarrow \bullet ab\}$$

$$I_3: \{I \rightarrow al \bullet b\}$$

$$I_4: \{I \rightarrow ab \bullet\}$$

$$I_5: \{I \rightarrow alb \bullet\}$$

Recuperação da Segunda Prova - Parte 1 - 08/12/14

- 1)(1.5 pontos) Escreva um parser descendente recursivo preditivo ou com retrocesso para reconhecer a linguagem $L = \{a^n b^m \mid n \neq m\}$
- 2)(2.0 pontos) Escreva um parser descendente recursivo preditivo para a gramática a seguir. Faça as alterações necessárias na gramática.
 $S \rightarrow Aa \mid a$
 $A \rightarrow Bb$
 $B \rightarrow Sc \mid Bc \mid d$
- 3)(1.5 pontos) Construa a tabela do analisador tabular preditivo para a linguagem descrita pela gramática a seguir. Faça as alterações necessárias na gramática:
 $E \rightarrow E ? E : E$
 $E \rightarrow id \mid id (E)$

- 1)(1.5 pontos) Encontre a coleção canônica de conjuntos de itens LR(0) para a gramática a seguir:

$$L \rightarrow aLb \mid \epsilon$$

- 2)(1.5 pontos) Encontre a coleção canônica de conjuntos de itens LR(1) para a gramática a seguir:

$$L \rightarrow aLb \mid \epsilon$$

[Voltar para o índice](#)

3)(1.5 pontos) A partir da gramática e conjuntos de itens a seguir, construa as tabelas de ação e de transição SLR.

$$I \rightarrow alb|ab$$

$$I_0: \{I' \rightarrow \bullet I$$

$$I \rightarrow \bullet alb$$

$$I \rightarrow \bullet ab\}$$

$$I_1: \{I' \rightarrow I \bullet\}$$

$$I_2: \{I \rightarrow \bullet alb$$

$$I \rightarrow a \bullet Ib$$

$$I \rightarrow a \bullet b$$

$$I \rightarrow \bullet ab\}$$

$$I_3: \{I \rightarrow al \bullet b\}$$

$$I_4: \{I \rightarrow ab \bullet\}$$

$$I_5: \{I \rightarrow alb \bullet\}$$

4)(1.5 pontos) A partir da gramática e conjuntos de itens a seguir, construa as tabelas de ação e de transição LR(1).

$$I \rightarrow alb|ab$$

$$I_0: \{I' \rightarrow \bullet I$$

$$I \rightarrow \bullet alb$$

$$I \rightarrow \bullet ab\}$$

$$I_1: \{I' \rightarrow I \bullet\}$$

$$I_2: \{I \rightarrow \bullet alb$$

$$I \rightarrow a \bullet Ib$$

$$I \rightarrow a \bullet b$$

$$I \rightarrow \bullet ab\}$$

$$I_3: \{I \rightarrow al \bullet b\}$$

$$I_4: \{I \rightarrow ab \bullet\}$$

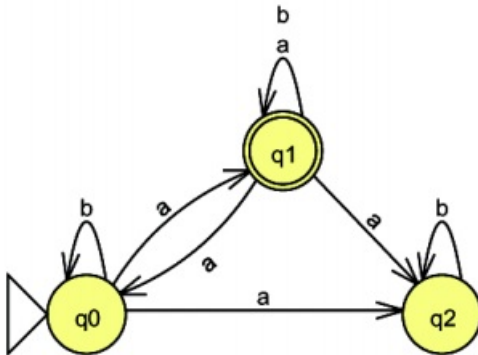
$$I_5: \{I \rightarrow alb \bullet\}$$

- 5) Identifique 5 relações de precedência entre terminais da gramática a seguir:
- 6) Identifique quais conjuntos de itens devem ser agrupados nos conjuntos a seguir, para gerar um analisador LALR(1)
- 7) O conjunto LR(0) a seguir representa uma gramática LR(0)?

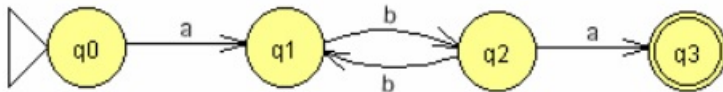
[Voltar para o índice](#)

- 1)(1 ponto) Escreva uma expressão regular sobre o alfabeto $\{a, b, c\}$ que gere a linguagem cujas sentenças não contenham o substring abc .
- 2)(1 ponto) Encontre um autômato finito determinístico que reconheça a linguagem gerada pela expressão $a^*b^*cd^*$.

3)(1.5 pontos) Converta o autômato a seguir para um autômato finito determinístico equivalente.



4)(1.5 pontos) Encontre a expressão regular equivalente ao autômato a seguir:



Avaliação Primeira Área - Parte 2 - 07/10/15

1)(1 ponto) Converta a gramática regular a seguir para uma expressão regular:

$$S \rightarrow 1S \mid 2S \mid 0B$$

$$B \rightarrow 0B \mid 1C \mid 2S$$

$$C \rightarrow 0B \mid \epsilon$$

2)(1 ponto) Elimine a recursão à esquerda (direta e indireta) da gramática a seguir:

$$S \rightarrow A0 \mid 0B1 \mid 11$$

$$A \rightarrow 01 \mid 0BC \mid S0$$

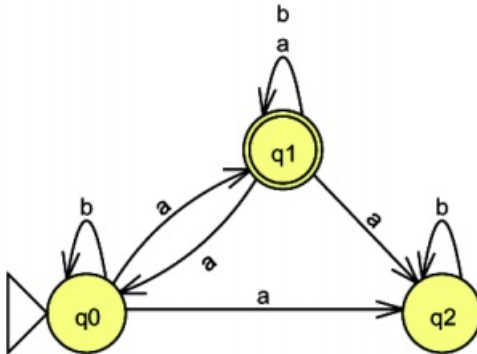
$$B \rightarrow B01 \mid 01A$$

3)(1.5 pontos) Construa uma gramática livre de contexto que gere a linguagem $L = \{a^k b^l, k \geq 1, l \geq k\}$

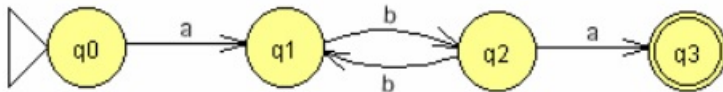
4)(1.5 pontos) Apresente uma gramática regular que gere a linguagem formada por todas as sentenças sobre $\{a, b, c\}^*$ onde não ocorre o string abc .

- 1)(1 ponto) Escreva uma expressão regular sobre o alfabeto $\{a, b, c\}$ que gere a linguagem cujas sentenças não contenham o substring ab .
- 2)(1 ponto) Encontre um autômato finito determinístico que reconheça a linguagem gerada pela expressão $a^*b^*cd^*$.

3)(1.5 pontos) Converta o autômato a seguir para o autômato finito determinístico mínimo equivalente.

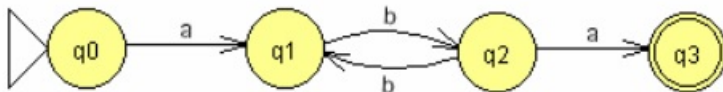


4)(1.5 pontos) Encontre a expressão regular equivalente ao autômato a seguir:



Avaliação Primeira Área - 16/10/17

- 1)(2 pontos) Escreva uma gramática regular sobre o alfabeto $\{a, b, c\}$ que gere a linguagem cujas sentenças não contenham o substring abc .
- 2)(2 pontos) Encontre um autômato finito determinístico que reconheça a linguagem gerada pela expressão $a^*b^*cd^*$.
- 3)(2 pontos) Encontre a expressão regular equivalente ao autômato a seguir:



4)(2.0 pontos) Escreva uma gramática G livre de contexto que gere a linguagem $L(G) = \{0^n 1^m, m > n > 0\}$

5)(2 pontos) Considere a gramática a seguir:

$$S \rightarrow Aa$$

$$A \rightarrow Sb \mid Ac \mid d$$

Gere uma gramática equivalente sem recursão imediata e não imediata à esquerda.

- 1)(1.5 pontos) Escreva um parser descendente recursivo com retrocesso para a gramática a seguir. As funções do parser devem corresponder exatamente às produções da gramática original, sem alterações:

$$S \rightarrow X|Y$$
$$X \rightarrow \mathbf{abC}$$
$$Y \rightarrow \mathbf{abD}$$
$$C \rightarrow \mathbf{cC|f}$$
$$D \rightarrow \mathbf{cD|\epsilon}$$

- 2)(1.5 pontos) Escreva um parser descendente recursivo preditivo para a linguagem definida pela gramática a seguir. Efetue as alterações necessárias na gramática:

$S \rightarrow X|Y$

$X \rightarrow \mathbf{ab}C$

$Y \rightarrow \mathbf{ab}D$

$C \rightarrow \mathbf{c}C|\mathbf{f}$

$D \rightarrow \mathbf{c}D|\epsilon$

- 3)(2 pontos) Construa a tabela do analisador tabular preditivo para a linguagem descrita pela gramática a seguir. Faça as alterações necessárias na gramática para que o parser reconheça a linguagem. Os terminais são **a**, **b**, **c** e **f**:

$S \rightarrow X|Yf$

$X \rightarrow abC$

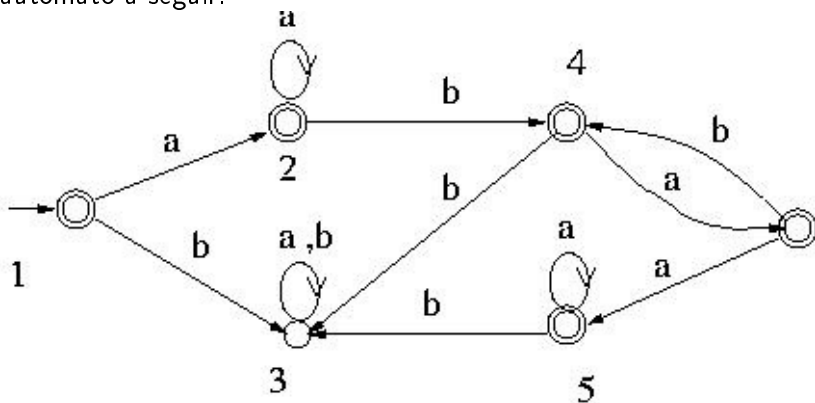
$Y \rightarrow abD$

$C \rightarrow cC|f$

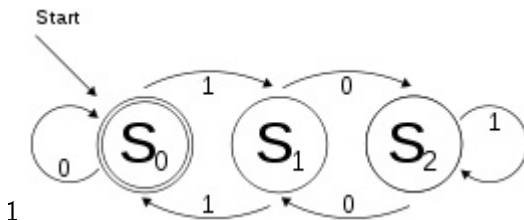
$D \rightarrow cD|\epsilon$

- 1)(1 ponto) Escreva uma expressão regular sobre o alfabeto $\{a, b, c\}$ que gere a linguagem cujas sentenças não contenham o substring bb .
- 2)(1 ponto) Encontre um autômato finito determinístico que reconheça a linguagem gerada pela expressão $(1|0)^*00$.

3)(1.5 pontos) Encontre o autômato mínimo equivalente ao autômato a seguir.



4)(1.5 pontos) O autômato abaixo reconhece números binários múltiplos de 3. Encontre a expressão regular equivalente:



- 1)(1.5 pontos) Escreva um parser descendente recursivo com retrocesso para a gramática a seguir. O parser deve implementar exatamente a gramática fornecida, sem alterações:

$A \rightarrow \mathbf{abC} \mid \mathbf{abD}$

$C \rightarrow \mathbf{cC} \mid \mathbf{f}$

$D \rightarrow \mathbf{cD} \mid \mathbf{d}$

- 2)(1.5 pontos) Escreva um parser descendente recursivo preditivo para a produção de C na gramática a seguir. Considere que o *else* deve estar sempre associado ao *if* mais interno. Considere que já existe uma função para o símbolo E.
 $C \rightarrow \mathbf{if\ E\ then\ C\ else\ C \mid if\ E\ then\ C \mid id = cte;}$

- 3)(2 pontos) Construa a tabela do analisador tabular preditivo para a linguagem descrita pela gramática a seguir. Faça as alterações necessárias na gramática:

$$E \rightarrow EE \mid b \mid a$$

Avaliação Segunda Área - Parte 2 - 09/12/15

1)(1.5 pontos) Encontre os 3 primeiros conjuntos de itens LR(0) para a gramática a seguir:

$$L \rightarrow aLb \mid ab$$

2)(1.5 pontos) A partir da gramática e coleção canônica de conjuntos de itens a seguir, construa as 3 primeiras linhas da tabela de ação e de transição SLR.

$$I \rightarrow alb|ab$$

$I_0 :$ $\{I' \rightarrow \bullet I$
 $I \rightarrow \bullet aIb$
 $I \rightarrow \bullet ab\}$
 $I_1 :$ $\{I' \rightarrow I\bullet\}$
 $I_2 :$ $\{I \rightarrow \bullet aIb$
 $I \rightarrow a \bullet Ib$
 $I \rightarrow a \bullet b$
 $I \rightarrow \bullet ab\}$
 $I_3 :$ $\{I \rightarrow aI \bullet b\}$
 $I_4 :$ $\{I \rightarrow ab\bullet\}$
 $I_5 :$ $\{I \rightarrow alb\bullet\}$

3)(2 pontos) Encontre os 3 primeiros conjuntos de itens LR(1) para a gramática estendida a seguir:

$I \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow id$

Avaliação Segunda Área - Parte 2 - 09/12/15

1)(1.5 pontos) Encontre os 3 primeiros conjuntos de itens LR(0) para a gramática a seguir:

$$L \rightarrow aLb \mid ab$$

2)(1.5 pontos) A partir da gramática e coleção canônica de conjuntos de itens a seguir, construa as 3 primeiras linhas da tabela de ação e de transição SLR.

$$I \rightarrow alb|ab$$

$I_0 :$ $\{I' \rightarrow \bullet I$
 $I \rightarrow \bullet aIb$
 $I \rightarrow \bullet ab\}$
 $I_1 :$ $\{I' \rightarrow I\bullet\}$
 $I_2 :$ $\{I \rightarrow \bullet aIb$
 $I \rightarrow a \bullet Ib$
 $I \rightarrow a \bullet b$
 $I \rightarrow \bullet ab\}$
 $I_3 :$ $\{I \rightarrow aI \bullet b\}$
 $I_4 :$ $\{I \rightarrow ab\bullet\}$
 $I_5 :$ $\{I \rightarrow alb\bullet\}$

Avaliação da Primeira Área - 17/10/18

1) (2.0 pontos) Escreva uma gramática G livre de contexto que gere a linguagem $L(G) = \{0^n 1^n 0^m 1^m, n > 0, m > 0\}$

2) (2.0 pontos) Escreva uma gramática regular equivalente à expressão regular

$(a|b)^*(c|d)ef^*$

3) (2.0 pontos) Encontre uma E.R. equivalente ao autômato a seguir:

Vocabulário: $\{a, b\}$

Estado inicial : q_0

Conjunto de estados : $\{q_0, q_1, q_2\}$

Conjunto de estados finais : $\{q_2\}$

Função de Transição:

$d(q_0, a) = q_1$ $d(q_0, b) = q_0$

$d(q_1, a) = q_1$ $d(q_1, b) = q_2$

$d(q_2, b) = q_0$

4) (2.0 pontos) Gere uma gramática equivalente à gramática a

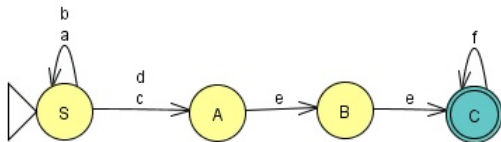
Avaliação da Primeira Área - 17/10/18

1) (2.0 pontos) Escreva uma gramática G livre de contexto que gere a linguagem $L(G) = \{0^n 1^n 0^m 1^m, n > 0, m > 0\}$

- $S \rightarrow AA$
- $A \rightarrow 0A1 \mid 01$

2) (2.0 pontos) Escreva uma gramática regular equivalente à expressão regular

$(a|b)^*(c|d)eef^*$



$S \rightarrow aS \mid bS \mid cA \mid dA$

$A \rightarrow eB$

$B \rightarrow eC$

$C \rightarrow fC \mid \epsilon$

- substituindo $C \rightarrow \epsilon$ nas outras...

$S \rightarrow aS \mid bS \mid cA \mid dA$

$A \rightarrow eB$

$B \rightarrow eC \mid e$

$C \rightarrow fC \mid f$

3) (2.0 pontos) Encontre uma E.R. equivalente ao autômato a seguir:

Vocabulário: $\{a, b\}$

Estado inicial : q_0

Conjunto de estados : $\{q_0, q_1, q_2\}$

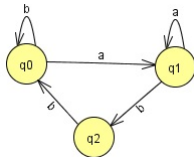
Conjunto de estados finais : $\{q_2\}$

Função de Transição:

$d(q_0, a) = q_1$ $d(q_0, b) = q_0$

$d(q_1, a) = q_1$ $d(q_1, b) = q_2$

$d(q_2, b) = q_0$



4) (2.0 pontos) Gere uma gramática equivalente à gramática a seguir, mas sem recursão à esquerda, imediata e não imediata.

$$S \rightarrow Ab \mid Sb \mid c$$

$$A \rightarrow Sc \mid Ad \mid f$$

5) (2.0 pontos) Defina um autômato finito determinístico que reconheça todas as sentenças sobre $\{a,b,c\}^*$ que não contenham o substring "ab".

Comentários sobre a gramática de referência - AnsiC.y

- O documento inicia especificando a lista de tokens compostos por mais de um caracter. Os tokens compostos de apenas um caracter são identificados nas regras sintáticas e no léxico pelo próprio caracter.
- A cláusula token associa uma numeração a partir de 258 a cada token definido (as numerações de 1 a 255 são utilizadas para os tokens de um caracter). A numeração 0 representa fim-de-arquivo.
- `%token IDENTIFIER CONSTANT STRING_LITERAL SIZEOF`
- `%token PTR_OP INC_OP DEC_OP LEFT_OP RIGHT_OP LE_OP GE_OP EQ_OP
NE_OP`

- A lista de tokens é colocada no arquivo ytab.h gerado, na forma de um enum, e é compartilhada pelo léxico:

```
enum yytokentype {  
    IDENTIFIER = 258,  
    CONSTANT = 259,...
```

- %token AND_OP OR_OP MUL_ASSIGN DIV_ASSIGN MOD_ASSIGN ADD_ASSIGN
- %token SUB_ASSIGN LEFT_ASSIGN RIGHT_ASSIGN AND_ASSIGN
- %token XOR_ASSIGN OR_ASSIGN TYPE_NAME
- %token TYPEDEF EXTERN STATIC AUTO REGISTER
- %token CHAR SHORT INT LONG SIGNED UNSIGNED FLOAT DOUBLE CONST VOLATILE VOID
- %token STRUCT UNION ENUM ELLIPSIS
- %token CASE DEFAULT IF ELSE SWITCH WHILE DO FOR GOTO CONTINUE BREAK RETURN

- A seção de definições termina definindo o ponto de entrada da gramática, com a cláusula `%start translation_unit`
- Por default o yacc utiliza como ponto de entrada a primeira regra de produção da seção de regras, a cláusula `start` permite definir outro ponto de entrada.
- Nessa gramática, `translation_unit` designa um programa fonte, que é uma lista de declarações de funções, tipos e variáveis.

- A primeira parte da gramática apresenta as expressões.
- `primary_expression` e `postfix_expression` define os operandos de expressões:
 - **`primary_expression`** define constantes (numéricas e chars), variáveis e expressões entre parênteses.
 - **`postfix_expression`** define referências a pointers, elementos de matrizes e vetores, chamadas de funções, referências a campos de structs e operandos com pós incremento e decremento.
- `postfix_expression` `(' argument_expression_list ')` designa uma chamada de funções e `argument_expression_list` a lista de parâmetros com ao menos um parâmetro.
- E em `postfix_expression` `PTR_OP IDENTIFIER`, `PTR_OP` é o operador `->`

- A produção:
 - `argument_expression_list`
 - : `assignment_expression`
 - | `argument_expression_list` `,` `assignment_expression`
 - ;
- Descreve listas de argumentos na chamada de funções. Como essa gramática é para o YACC, que usa um analisador redutivo, a recursão imediata à esquerda não é um problema
- De modo geral, em todos os pontos da gramática em que são geradas listas de elementos (identificadores, parâmetros, comandos...) o modelo seguido é o mesmo:
 - `A : s`
 - `A : A s`

- Para implementação de um analisador descendente recursivo, esse modelo de listas de elementos deve ser modificado para um modelo que não contenha recursão à esquerda, conforme visto em aula. A lista de argumentos, por exemplo, poderia ser implementada como:
 - $\text{ListaArg} \rightarrow \text{ExpressaoAtrib RestoListaArg}$
 - $\text{RestoListaArg} \rightarrow , \text{ExpressaoAtrib RestoListaArg}$
 - $\text{RestoListaArg} \rightarrow \epsilon$

- Sendo cada argumento da lista uma expressão com quaisquer operadores com precedência maior ou igual aos operadores de atribuição.
- O operador vírgula é deixado de fora para eliminar a ambiguidade entre a vírgula como operador em uma expressão e a vírgula separadora de parâmetros (Isso é uma definição da linguagem, e não apenas dessa gramática).
 - Ex: `fat(a,b)` é uma chamada de função com dois parâmetros ou com uma única expressão contendo vírgula?

Tabela de precedência de operadores em C

Prec.	Símbolo	Função	Associatividade
1	++ --	Incremento e decremento pósfixo	esquerda para direita
1	()	Parênteses (chamada de função)	
1	[]	Elemento de array	
1	.	Seleção de campo de structure	
1	->	Seleção de campo de structure a partir de ponteiro	
2	++ --	Incremento e decremento prefixo	direita para a esquerda
2	+ -	Adição e subtração unária	
2	! ~	Não lógico e complemento	
2	(tipo)	Conversão de tipo de dado	
2	*	Desreferência	
2	&	Referência (endereço de elemento)	
3	* / %	Multiplicação, divisão, e módulo (resto)	esquerda para a direita
4	+	Adição e subtração	
5	<< >>	Deslocamento de bits à esquerda e à direita	
6	< <=	"menor que" e "menor ou igual que"	
6	> >=	"maior que" e "maior ou igual que"	
7	= = !=	"Igual a" e "diferente de"	
8	&	E bit a bit	
9	^	Ou exclusivo para bits	
10		Ou para bits	
11	&&	E lógico	
12		Ou lógico	
13	c ? t : f	Condição ternária	direita para esquerda
14	=	Atribuição	
14	+= -=	Atribuição por adição ou subtração	
14	*= /= %=	Atribuição por multiplicação, divisão ou módulo (resto)	
14	<<= >>=	Atribuição por deslocamento de bits	
14	&= ^= =	Atribuição por operações lógicas	esquerda para direita
15	,	vírgula	

- A próximas 3 produções (`unary_expression`, `unary_operator` e `cast_expression`) descrevem os operadores de nível 2, basicamente operadores unários, pré incremento e decremento, `sizeof` e `cast`.
- As produções de `multiplicative_expression` a `logical_or_expression` geram as expressões com os operadores de cada nível de precedência.
- Como todos os operadores dessas faixas de precedência são associativos à esquerda, as produções são recursivas à esquerda, remetendo para o nível seguinte de precedência.
- Para implementar o analisador descendente recursivo é necessário transformar cada uma em uma produção equivalente sem recursão à esquerda.

- A produção seguinte define expressões com operador ternário:

```
■ conditional_expression
■   : logical_or_expression
■   | logical_or_expression '?' expression ':' conditional_expression
■   ;
```

- O segundo operando do ternário pode ser uma expressão com qualquer operador, inclusive o de atribuição, que tem menor prioridade que o ternário.
- A expressão do primeiro operando só pode conter operadores de maior prioridade, a partir do "ou lógico".
- A possibilidade do ternário no primeiro operando geraria uma ambiguidade na expressão: "a?b:c?d:e" que poderia ser interpretada como "(a?b:c)?d:e" e "a?b:(c?d:e)"

- As próximas duas produções definem expressões de atribuição. Como os operadores de atribuição são associativos à direita, a produção é recursiva à direita.

```
assignment_expression  
    : conditional_expression  
    | unary_expression assignment_operator  
assignment_expression  
    ;
```

- E **assignment_operator** é qualquer um dos operadores de atribuição (=, +=, -=, *=, ...);
- À esquerda do símbolo de atribuição só são permitidas expressões unárias, ou seja, expressões formadas apenas por operandos e operadores unários.
- A razão disso é que o operando da esquerda da atribuição deve ser um "l-value" ("left value"), expressão que resulte em um endereço, o que não ocorre com os operadores de dois operandos.

- A próxima produção define expressões com o operador vírgula. É o operador de menor precedência, e permite combinar duas expressões em uma única expressão. Isso ocorre, por exemplo, na lista de inicializações e incrementos do **for**.
- Na avaliação da expressão com vírgula, a primeira subexpressão é avaliada e seu resultado é descartado, e o resultado da expressão com vírgula é o resultado da segunda subexpressão.
- Na maioria dos pontos da gramática onde pode ocorrer uma expressão (na condição de **if**, **while** e **for**, comando-expressão...) remete-se para esse nível de expressão.

- A próxima produção define **constant_expression**, uma expressão a partir de expressão ternária, cujo resultado possa ser resolvido já na compilação.
- É utilizada na declaração de dimensões de arrays, labels de switch e definição de enum.

```
constant_expression  
    : conditional_expression  
    ;
```

- As próximas 31 produções definem declarações de variáveis, funções e tipos e serão comentadas mais adiante.
- As produções seguintes definem os comandos. A produção `statement` classifica os comandos em 6 categorias:
 - `labeled_statement` - comandos com label
 - `compound_statement` - comando composto. Lista de comandos delimitada por par de chaves.
 - `expression_statement` - comando-expressão
 - `selection_statement` - comando de seleção: `if` e `switch`
 - `iteration_statement` - comando de repetição: `for`, `while` e `do`
 - `jump_statement` - desvios: `break`, `continue`, `return` e `goto`

- A produção `labeled_statement` define comandos com label, podendo esse label ser um identificador, como em: `"Label1: a=0;"`, bem como `"case n:"` e `"default"`, usados no comando `switch`.
- No caso do label `"case"`, o valor do label deve ser uma expressão constante, cujo valor já possa ser resolvido na compilação.

- A produção `compound_statement` define comando composto, também conhecido como "bloco de comandos".
- É um par de chaves com uma lista de 0 ou mais comandos e declarações.
- Nessa gramática as declarações devem aparecer obrigatoriamente antes dos comandos, como era a definição original da linguagem C.
- A possibilidade de colocar declarações após comandos dentro de um bloco surgiu no C99.
- Em seguida são apresentadas as produções para lista de comandos e lista de declarações.

- A produção seguinte apresenta o comando-expressão, que é uma expressão acrescentada de um ponto-e-vírgula, ou simplesmente um ponto-e-vírgula.
- Uma expressão em C não pode ser vazia, mas um comando expressão pode conter somente o ponto-e-vírgula.
- Em C, qualquer expressão, acrescentada de um ponto-e-vírgula, é um comando válido, mesmo que não resulte em nenhuma ação.
- Assim, "a+b;" é um comando válido.
- Chamadas de funções ou o "comando de atribuição" são derivados de comando-expressão.

- A próxima produção define os comandos **if** e **switch**.
- O comando **if** tem uma produção com o **else** e uma produção sem o **else**, que gerará um conflito empilha-reduz quando analisar um if encadeado, resolvido pelo analisador priorizando o empilhamento, o que faz com que o else seja associado ao if mais próximo.
- O comando **switch** tem um único comando que, normalmente, mas não obrigatoriamente será um comando composto em que alguns dos comandos tem labels associados. O que o switch faz é avaliar a expressão e desviar para o label resultante, se existir.

- Na próxima produção são definidos os comandos iterativos. As expressões do do-while e do while podem conter quaisquer operadores e as chaves no bloco não são obrigatórios, sintaticamente contendo apenas um único comando .
- O cabeçalho do for tem três elementos, sendo os dois primeiros comando-expressão, o que obriga os dois ponto-e-vírgula, mas não as expressões. O terceiro elemento é uma expressão ou vazio.

- A produção seguinte define comandos de desvio: `break`, `continue`, `goto`, e `return`, com e sem valor de retorno.
- A próxima produção é o ponto de entrada da gramática, `translation_unit`, que gera uma lista de declarações (`external_declaration`), sendo que declarações podem ser definição de função ou declaração de variáveis ou tipos.